UNCUYO
UNIVERSIDAD NACIONAL DE CUYO

FACULTAD DE INGENIERÍA

Semáforos Inteligentes

Un Enfoque con Aprendizaje por Refuerzo Multiagente

Tesina

Licenciatura en Ciencias de la Computación

Autor

Juan Martín Morales

Directora

Dra. Ana Carolina Olivera

Ciudad de Mendoza, Argentina

2026

Resumen

Este trabajo aborda la congestión vehicular urbana mediante el control semafórico con aprendizaje por refuerzo multiagente. Cada intersección se representa por un agente que selecciona fases a partir de una observación local del tránsito. Se consideran MAPPO y HAPPO con entrenamiento centralizado y ejecución descentralizada, e IPPO e IDQN con aprendizaje independiente; además, se describe una adaptación de HAPPO con actualización secuencial de los actores. La espera acumulada por carril se codifica mediante una transformación logarítmico-lineal acotada y discretizada, con mayor resolución en valores bajos. La búsqueda de hiperparámetros se organiza en dos fases, una amplia y otra refinada, para los cuatro algoritmos en las tres redes sintéticas y en una red del microcentro de Mendoza, con dominios propios de cada escenario. La comparación local y vecinal de MAPPO se circunscribe a las redes sintéticas. La red urbana se construye con información cartográfica y conteos parciales, y su comprobación con los programas de SUMO caracteriza su operación, no el desempeño de políticas aprendidas.

Abstract

This work addresses urban traffic congestion through multi-agent reinforcement learning for traffic-signal control. Each intersection is represented by an agent that selects phases from a local observation of traffic. The algorithms considered are MAPPO and HAPPO with centralized training and decentralized execution, and IPPO and IDQN with independent learning; the method also describes an adaptation of HAPPO with sequential actor updates. Accumulated lane waiting is represented by a bounded, discretized log-linear transformation with finer resolution at lower values. Hyperparameter search is organized in two phases, a broad exploration and a refined search, for the four algorithms across three synthetic networks and an urban network in downtown Mendoza, using scenario-specific domains. The MAPPO comparison of local and neighborhood variants is limited to the synthetic networks. The urban network is built from cartographic information and partial traffic counts, and its operational check with SUMO signal programs characterizes network operation rather than learned-policy performance.

Agradecimientos

Página de agradecimientos

Índice general

1. Introducción **1**
1.1. Motivación . . . 3
1.2. Organización del documento . . . 4

2. Marco teórico **5**
2.1. Ingeniería de tránsito y simulación . . . 5
2.1.1. Terminología de tránsito . . . 6
2.1.2. Fundamentos de la dinámica vehicular . . . 8
2.1.3. Sistemas de control de tránsito . . . 14
2.1.4. Simulación de tránsito . . . 16
2.2. Aprendizaje por refuerzo (RL) . . . 22
2.2.1. Procesos de decisión de Markov (MDP) . . . 23
2.2.2. Métodos basados en valor y basados en política . . . 25
2.2.3. Aprendizaje profundo y redes neuronales . . . 29
2.2.4. Aprendizaje por refuerzo profundo (DRL) . . . 32
2.3. Aprendizaje por refuerzo multiagente (MARL) . . . 39
2.3.1. Fundamentos teóricos: juegos estocásticos, equilibrio de Nash y Dec-POMDP . . . 40
2.3.2. Desafíos del aprendizaje multiagente . . . 42
2.3.3. Paradigmas de entrenamiento y ejecución . . . 43
2.3.4. MADRL y líneas de base . . . 44
2.3.5. MAPPO . . . 45
2.3.6. HAPPO . . . 48
2.3.7. Antecedentes recientes del control semafórico multiagente . . . 53

3. Metodología **59**
3.1. Diseño experimental . . . 60
3.2. Escenarios de simulación . . . 61
3.2.1. Preparación de las redes de tráfico . . . 61
3.2.2. Redes de tráfico sintéticas . . . 62
3.2.3. Red de tráfico de la Ciudad de Mendoza . . . 67

3.3. Sistema de control multiagente 72
3.3.1. Entorno de simulación 73
3.3.2. Observación, acciones y recompensas 73
3.3.3. Agentes y algoritmos de aprendizaje 77
3.4. Entrenamiento y búsqueda de hiperparámetros 79
3.4.1. Condiciones de simulación y entrenamiento 80
3.4.2. Función objetivo de la búsqueda 81
3.4.3. Fase 1: búsqueda amplia 82
3.4.4. Fase 2: búsqueda con dominios reducidos 83
3.4.5. Criterios de selección de configuraciones 84
3.5. Comparación de variantes locales y vecinales de MAPPO 86
3.5.1. Prueba preliminar de formulaciones 86
3.5.2. Entrenamiento de las variantes en las tres redes sintéticas 87
3.6. Evaluación y análisis de políticas 88
3.6.1. Métricas de desempeño y vehículos evaluados 88
3.6.2. Entrenamiento y evaluación de los modelos finales 90
3.6.3. Evaluación de variantes de MAPPO 91
3.6.4. Análisis por pares y criterio de decisión 92
3.7. Reproducibilidad y alcance 94
3.7.1. Entorno de cómputo y envío de trabajos 94
3.7.2. Limitaciones del diseño y alcance de la inferencia 95

4. Análisis y discusión de resultados **96**
4.1. Resultados del ajuste de hiperparámetros 96
4.1.1. Estado de la consolidación y de las búsquedas 97
4.1.2. Resultados del ajuste y análisis de sensibilidad 97
4.1.3. Selección de configuraciones de HPO 97
4.2. Evaluación de políticas y comprobación de escenarios 97
4.2.1. Criterios de evaluación y métricas de comparación 97
4.2.2. Desempeño en escenarios sintéticos: 3×1, 3×3, 4×4 heterogéneo 97
4.2.3. Comprobación operacional del escenario de la Ciudad de Mendoza . 97
4.3. Discusión e interpretación de resultados 98

5. Conclusiones **99**

A. Arquitectura de software y reproducción experimental **100**
A.1. Diseño orientado a objetos del entorno de simulación 100
A.2. Jerarquía de funciones de recompensa y patrón Strategy 102
A.3. Patrones de diseño de la arquitectura 104
A.4. Parámetros de SUMO y programas semafóricos de referencia 105

A.4.1. Identificadores y procedencia del escenario urbano de Mendoza . . . 106
A.5. Especificaciones para reproducir los experimentos . . . 110
A.5.1. Infraestructura y almacenamiento de las ejecuciones . . . 110
A.5.2. Comprobaciones preliminares . . . 110
A.5.3. Detalles de las redes sintéticas . . . 111
A.5.4. Espacio de búsqueda inicial de fase 1 . . . 111
A.5.5. Semillas de fase 1 en las redes sintéticas . . . 115
A.5.6. Arquitecturas y semillas de la búsqueda refinada sintética . . . 116
A.5.7. Dominios declarados y significado de los parámetros . . . 118
A.5.8. Protocolo urbano de Mendoza . . . 125
A.5.9. Métricas y análisis por pares de A/C . . . 127
A.6. Detalles del aprendizaje y de la codificación . . . 127
A.6.1. Desarrollo algebraico del codificador . . . 128
A.6.2. Aplicación de la máscara de acciones . . . 130
A.6.3. Configuración fija de la comparación multiescenario . . . 130
A.6.4. Calibraciones y semillas de cooperación . . . 132
A.6.5. Preparación de las figuras de escenarios . . . 132

Índice de figuras

1.1. Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC. 1

2.1. Movimientos vehiculares, fases y cruces peatonales 7
2.2. Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada. 8
2.3. Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s). 11
2.4. Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de agrupamientos vehiculares. Las curvas representan situaciones conceptuales y no una distribución empírica particular. 13
2.5. Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada. 15
2.6. Correspondencia entre la circulación real y su representación en SUMO . . 17
2.7. Interfaz sumo-gui durante una simulación microscópica 18
2.8. Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos. 20
2.9. Interacción agente-entorno en un proceso de decisión de Markov [SB18]. . . 24
2.10. Esquema de un método basado en valor 26
2.11. Esquema de un método basado en política 27
2.12. Arquitectura actor-crítico aplicada al control semafórico 29
2.13. Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación. . 30
2.14. Arquitectura conceptual de una red neuronal *feedforward* con dos capas ocultas. 31

2.15. Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de *bootstrapping* del valor objetivo diferencia temporal (*Temporal-Difference*, TD). Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS+15]. 34
2.16. Red de política para un espacio discreto de acciones 36
2.17. Interacción agente-entorno en MARL cooperativo 41
2.18. Evolución de las políticas de dos agentes bajo WoLF-PHC en Piedra, Papel o Tijera 43
2.19. Paradigma de entrenamiento centralizado con ejecución descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales. 45
2.20. Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m}, que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia. 51

3.1. Flujo de las etapas experimentales. Los escenarios preparados alimentan la búsqueda de hiperparámetros y los estudios de variantes. Las fases 1 y 2 abarcan los cuatro escenarios; A–E y A/C utilizan configuraciones fijas en las redes sintéticas. Las flechas representan dependencias, no el orden cronológico de las campañas; la selección final utiliza los resultados de la búsqueda. 61
3.2. Partición temporal de los programas semafóricos de referencia en las redes de tráfico sintéticas. Cada barra representa un ciclo de 90 segundos y distingue los intervalos de verde (V), cambio amarillo (A) y todo rojo (R); las barras resumen duraciones y no desagregan el estado de cada movimiento. 62
3.3. Red 3×1 en SUMO: tres intersecciones semaforizadas y dos carriles por sentido. 63
3.4. Red 3×3 en SUMO: nueve intersecciones semaforizadas y dos carriles por sentido. 64
3.5. Red $4 \times 4_H$ en SUMO: dieciséis intersecciones semaforizadas y tramos de uno a cuatro carriles por sentido de circulación. 65
3.6. Red de tráfico de una porción del microcentro de Mendoza en SUMO. . . . 68
3.7. Zonas de análisis de tránsito de Mendoza en SUMO. Los tramos asociados a las zonas se destacan en celeste y los recuadros rojos permiten localizarlos en la vista. 70

3.8. Flujo entre la simulación, el entorno y el aprendizaje. El estado de SUMO se almacena aparte del flujo de aprendizaje. 72
3.9. Codificación de la espera por carril con $I = 22$ y límite de escala $M_{\text{lane}} =$ 1136 veh·s, como ejemplo ilustrativo. La fase activa se conserva y el nivel máximo puede alcanzarse antes de llegar a ese límite. 75
3.10. Información de los algoritmos durante el entrenamiento y la ejecución. Cada algoritmo y agente tiene su propia política; el crítico centralizado de MAPPO y HAPPO no interviene al ejecutar. En la variante vecinal de MAPPO, el actor recibe además la espera media por carril codificada de cada vecino directo. 79
3.11. Búsqueda en dos fases para las tres redes sintéticas y Mendoza. En cada escenario, la exploración amplia orienta la reducción de sus dominios; las condiciones sintéticas y urbanas se detallan por separado en el apéndice. . 84
3.12. Análisis por pares: A y C comparten la semilla de entrenamiento y las cinco semillas de evaluación; el remuestreo conserva cada par completo. 93

A.1. Diagrama de clases del entorno de simulación y de sus relaciones principales. 102
A.2. Especificación UML del módulo de recompensas. 103
A.3. Ejemplo de codificación de observaciones. 130

Índice de tablas

2.1. Variables de tránsito macroscópicas y microscópicas 10
2.2. Archivos de entrada y configuración empleados en una simulación de SUMO. 21
2.3. Comparación sintética de familias algorítmicas PPO en entornos multiagente cooperativos. 53

3.1. Duraciones de los programas semafóricos de referencia en las redes de tráfico sintéticas (segundos). 62
3.2. Perfiles de vehículos de las redes de tráfico sintéticas. 63
3.3. Topología, control y demanda de las redes sintéticas. Se excluyen los tramos y carriles internos de los cruces. 66
3.4. Observaciones y recompensas de las búsquedas sintéticas y de la comparación A/C. 77
3.5. Calentamiento y duración del control por procedimiento. 80
3.6. Variantes examinadas en la prueba preliminar. 87

4.1. Métricas operacionales del escenario urbano de la Ciudad de Mendoza con los programas semafóricos de la red. 97

A.1. Herramientas y responsabilidades en el sistema experimental. 101
A.2. Duraciones nominales de las fases en el archivo de red de Mendoza, en segundos. 105
A.3. Resumen de dos versiones documentadas del perfil de demanda de Mendoza. 106
A.4. Archivos que sustentan la procedencia del escenario urbano de Mendoza. . 107
A.5. Espacio inicial de fase 1 para MAPPO, IPPO y HAPPO. Las excepciones por algoritmo se indican en las filas. 112
A.6. Espacio inicial de fase 1 para IDQN. Los contadores se expresan en pasos según la interfaz del algoritmo. 113
A.7. Factores de estudio y valores fijos documentados para la fase 1 sintética. . 114
A.8. Correspondencia entre los parámetros de búsqueda y sus claves de configuración. 115
A.9. Semillas por escenario sintético en la fase 1. 115

A.10.Arquitecturas y semillas de la búsqueda refinada para las doce combinaciones sintéticas. La semilla de evaluación es común a los algoritmos del mismo escenario; las semillas de TPE no son réplicas de entrenamiento. . . 117
A.11.Dominios declarados para la búsqueda `phase2_refined` de HAPPO por escenario y arquitectura. 120
A.12.Dominios declarados para la búsqueda `phase2_refined` de IDQN por escenario y arquitectura. 121
A.13.Dominios declarados para la búsqueda refinada de IPPO por escenario y arquitectura. 122
A.14.Dominios declarados para la búsqueda refinada de MAPPO por escenario y arquitectura. 123
A.15.Hiperparámetros fijos de aprendizaje de MAPPO. 131

Siglas

A2C Actor–crítico con ventaja (*Advantage Actor-Critic*).

AFAC Asociación de Fábricas Argentinas de Componentes.

API Interfaz de programación de aplicaciones (*Application Programming Interface*).

ASHA Algoritmo asíncrono de reducción sucesiva a la mitad (*Asynchronous Successive Halving Algorithm*).

BFS Búsqueda en anchura (*Breadth-First Search*).

CCAD Centro de Cómputo de Alto Desempeño.

CCAR Centro de Cómputo de Alto Rendimiento.

CI Tasa de completitud de vehículos insertados (*Completion of Inserted Vehicles*).

CLIP Objetivo sustituto recortado (*Clipped surrogate objective*).

CO Monóxido de carbono.

CO2 Dióxido de carbono.

CONICET Consejo Nacional de Investigaciones Científicas y Técnicas.

CPU Unidad central de procesamiento (*Central Processing Unit*).

CR Tasa de completitud (*Completion Rate*).

CTDE Entrenamiento centralizado con ejecución descentralizada (*Centralized Training with Decentralized Execution*).

CVaR Valor en riesgo condicional (*Conditional Value at Risk*).

DAG Grafo acíclico dirigido (*Directed Acyclic Graph*).

DDPG Gradiente de política determinista profundo (*Deep Deterministic Policy Gradient*).

Dec-POMDP Proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*).

DLR Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*).

DQN Red Q profunda (*Deep Q-Network*).

DRL Aprendizaje por refuerzo profundo (*Deep Reinforcement Learning*).

E/S Entrada/salida (*Input/Output*).

FCEFN Facultad de Ciencias Exactas, Físicas y Naturales.

FHWA Administración Federal de Carreteras (*Federal Highway Administration*).

FLIR Infrarrojo de barrido frontal (*Forward Looking Infrared*).

GAE Estimación generalizada de ventaja (*Generalized Advantage Estimation*).

GEH Estadístico de comparación de flujos de Geoffrey E. Havers.

GPU Unidad de procesamiento gráfico (*Graphics Processing Unit*).

GUI Interfaz gráfica de usuario (*Graphical User Interface*).

HAML Aprendizaje espejo con agentes heterogéneos (*Heterogeneous-Agent Mirror Learning*).

HAPPO Optimización de política proximal con agentes heterogéneos (*Heterogeneous-Agent Proximal Policy Optimization*).

HARL Aprendizaje por refuerzo con agentes heterogéneos (*Heterogeneous-Agent Reinforcement Learning*).

HATRPO Optimización de política por región de confianza con agentes heterogéneos (*Heterogeneous-Agent Trust Region Policy Optimization*).

HBEFA Manual de factores de emisión para transporte por carretera (*Handbook Emission Factors for Road Transport*).

HC Hidrocarburos no combustionados.

HPC Computación de alto rendimiento (*High-Performance Computing*).

HPO Optimización de hiperparámetros (*Hyperparameter Optimization*).

IA Inteligencia artificial (*Artificial Intelligence*).

IDQN Red Q profunda independiente (*Independent Deep Q-Network*).

IL Aprendizaje independiente (*Independent Learning*).

IPPO Optimización de política proximal independiente (*Independent Proximal Policy Optimization*).

IQM Media intercuartílica (*Interquartile Mean*).

IR Tasa de inserción (*Insertion Rate*).

ITE Instituto de Ingenieros de Transporte (*Institute of Transportation Engineers*).

KL Divergencia de Kullback–Leibler.

MADDPG Gradiente de política determinista profundo multiagente (*Multi-Agent Deep Deterministic Policy Gradient*).

MADRL Aprendizaje por refuerzo profundo multiagente (*Multi-Agent Deep Reinforcement Learning*).

MAPPO Optimización de política proximal multiagente (*Multi-Agent Proximal Policy Optimization*).

MARL Aprendizaje por refuerzo multiagente (*Multi-Agent Reinforcement Learning*).

MDP Proceso de decisión de Markov (*Markov Decision Process*).

ML Aprendizaje automático (*Machine Learning*).

MLP Perceptrón multicapa (*Multi-Layer Perceptron*).

NFS Sistema de archivos de red (*Network File System*).

NOx Óxidos de nitrógeno.

OD Origen-destino (*Origin-Destination*).

OOD Fuera de distribución (*Out-of-Distribution*).

OSM OpenStreetMap.

PHEMlight Modelo de emisiones para vehículos livianos y pesados (*Passenger car and Heavy duty Emission Model light*).

POMDP Proceso de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*).

POSIX Interfaz portable de sistemas operativos (*Portable Operating System Interface*).

PPO Optimización de política proximal (*Proximal Policy Optimization*).

QMIX Red de mezcla de valores Q (*Q-Mixing Network*).

ReLU Unidad lineal rectificada (*Rectified Linear Unit*).

RL Aprendizaje por refuerzo (*Reinforcement Learning*).

SGD Descenso de gradiente estocástico (*Stochastic Gradient Descent*).

SL Aprendizaje supervisado (*Supervised Learning*).

SLURM Herramienta simple de Linux para gestión de recursos (*Simple Linux Utility for Resource Management*).

SUMO Simulación de movilidad urbana (*Simulation of Urban MObility*).

TAZ Zona de análisis de tránsito (*Traffic Analysis Zone*).

TCP Protocolo de control de transmisión (*Transmission Control Protocol*).

TD Diferencia temporal (*Temporal-Difference*).

TPE Estimador de Parzen estructurado en árbol (*Tree-structured Parzen Estimator*).

TraCI Interfaz de control del tránsito (*Traffic Control Interface*).

TRPO Optimización de política por región de confianza (*Trust Region Policy Optimization*).

UL Aprendizaje no supervisado (*Unsupervised Learning*).

UML Lenguaje unificado de modelado (*Unified Modeling Language*).

UTM Sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*).

VDN Redes de descomposición de valor (*Value-Decomposition Networks*).

VF Función de valor (*Value Function*).

WGS84 Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*).

WoLF-PHC Ascenso de colina en la política con ganar o aprender rápido (*Win or Learn Fast Policy Hill-Climbing*).

XML Lenguaje de marcado extensible (*Extensible Markup Language*).

Glosario

G_t **(retorno)** Suma acumulada, usualmente descontada, de las recompensas futuras a partir del instante t.

$G_{\mathbf{wait}}$ **(coeficiente de Gini de esperas)** Índice adimensional entre 0 y 1 que cuantifica la desigualdad entre las medias temporales de espera por controlador semafórico; no describe la distribución entre vehículos.

K_i **(número de acciones)** Número de fases semafóricas seleccionables por el agente i. Excluye las fases con amarillo y el estado final de todo rojo; las restricciones temporales pueden reducir el subconjunto de acciones habilitado en un instante.

$M_{i_{1:m}}$ **(factor de reponderación HAPPO)** Factor acumulativo de razones de probabilidad que repondera la ventaja conjunta en la actualización secuencial de HAPPO para reflejar los cambios de política de los agentes predecesores.

$Q(s, a)$ **(valor de acción)** Retorno esperado de ejecutar la acción a en el estado s y continuar posteriormente de acuerdo con una política determinada.

$R_t^{\mathbf{target}}$ **(retorno objetivo)** Estimación de retorno utilizada como referencia para ajustar la predicción de valor del crítico en el instante t.

$S[\pi_\theta]$ **(entropía de la política)** Medida de la dispersión de la distribución de acciones de la política; su inclusión en el objetivo favorece la exploración.

$V(s)$ **(valor de estado)** Retorno esperado al comenzar en el estado s y seguir una política determinada, promediando las acciones que dicha política selecciona.

$W_i^{(t)}$ **(espera atribuida a la intersección)** Suma de las esperas atribuidas a los carriles controlados por el agente i en el paso de decisión t, en veh·s. Se utiliza en la observación vecinal y en las diferencias de espera de la recompensa; depende de la memoria temporal finita de SUMO y no equivale a la espera total por viaje de TripInfo.

$W_{\mathbf{net}}$ **(demora acumulada global)** Métrica agregada que suma los segundos de espera de los vehículos detenidos en la red durante la ventana de evaluación.

α **(tasa de aprendizaje)** Paso de actualización que regula la magnitud del ajuste de los parámetros o valores en cada iteración de aprendizaje. En arquitecturas actor–crítico se especializa mediante subíndices, tales como α_θ para la red del actor y α_ϕ (o η) para el crítico.

δ_t **(error de diferencia temporal)** Diferencia entre la recompensa más el valor descontado del estado siguiente y la estimación de valor del estado actual.

ϵ_{clip} **(margen de recorte)** Hiperparámetro de PPO que delimita el intervalo utilizado para recortar la razón entre la probabilidad nueva y la anterior de una acción.

ϵ_{exp} **(exploración)** Probabilidad o coeficiente que controla la selección de acciones exploratorias frente a las acciones favorecidas por la política o la función de valor.

η **(tasa del optimizador)** Tamaño de paso empleado por el optimizador en la actualización de parámetros mediante descenso de gradiente.

γ **(factor de descuento)** Parámetro en el intervalo $[0, 1)$ que pondera la contribución de las recompensas futuras al retorno esperado.

$\hat{A}_t$ **(ventaja estimada)** Estimación del valor de la función de ventaja en el paso temporal t (habitualmente calculada mediante GAE), que cuantifica cuánto supera o desmejora el rendimiento de la acción ejecutada respecto del valor basal esperado del estado.

λ **(suavizado de la ventaja)** Hiperparámetro que pondera errores de diferencia temporal de distintos horizontes al estimar la ventaja y equilibra sesgo y varianza.

$\mathcal{L}(\theta)$ **(función de pérdida)** Función escalar que cuantifica el error del modelo y que se minimiza para ajustar los parámetros θ.

ϕ **(parámetros del crítico)** Vector de parámetros entrenables de la red que estima la función de valor del crítico.

π_θ **(política parametrizada)** Distribución de probabilidad sobre las acciones condicionada por el estado u observación y controlada por el vector de parámetros entrenables θ.

ρ_{coop} **(ponderación cooperativa)** Coeficiente en $[0, 1)$ que pondera la combinación entre la recompensa local de una intersección y la señal promedio de sus vecinos topológicos; los dominios de búsqueda pueden restringirlo a un intervalo menor.

θ **(parámetros de la política)** Vector de parámetros entrenables que determina la distribución de acciones de la política del actor.

a **(aceleración vehicular)** Parámetro de aumento de velocidad del tipo de vehículo en SUMO, expresado en metros por segundo cuadrado. En las ecuaciones de aprendizaje, la letra a puede representar una acción; son usos distintos que se distinguen por el contexto.

a_i, A_t **(acción)** Decisión ejecutada por un agente. En formulaciones temporales, A_t denota la variable aleatoria de acción en el paso t; en entornos multiagente, a_i representa la acción local del agente i y $\mathbf{a} = (a_1, \ldots, a_n)$ la acción conjunta. En el control semafórico corresponde a la fase asignada a la intersección.

c_1 **(coeficiente de valor)** Peso asignado al término de pérdida de la función de valor en el objetivo combinado de PPO.

c_2 **(coeficiente de entropía)** Peso asignado al término de entropía que incentiva la exploración en el objetivo combinado de PPO.

d **(desaceleración vehicular)** Parámetro de frenado del tipo de vehículo en SUMO, expresado como una magnitud positiva en metros por segundo cuadrado.

m_i **(carriles controlados)** Número de carriles entrantes controlados por el agente i. En el entorno utilizado, la observación local incorpora una medida de espera codificada por cada uno de estos carriles.

o_i **(observación local)** Información disponible para el agente i durante la ejecución descentralizada. Se obtiene a partir de los sensores o variables observables de la intersección que controla.

r_t **(recompensa)** Señal escalar recibida en el paso temporal t después de ejecutar una acción. Se utiliza para orientar la actualización de la política o de la función de valor.

s **(estado global)** Representación conjunta de las variables relevantes del entorno en un instante determinado. En el modelo multiagente puede incluir la información de todas las intersecciones y sus corrientes vehiculares.

$v_{\text{máx}}$ **(velocidad máxima vehicular)** Velocidad máxima configurada para un tipo de vehículo, expresada en metros por segundo; el límite de la vía también restringe la velocidad permitida.

Binding Mecanismo que conecta una biblioteca escrita en un lenguaje de programación con otro lenguaje para poder utilizar sus funciones.

Bootstrap Procedimiento de remuestreo con reemplazo utilizado para aproximar la incertidumbre de un estimador a partir de las observaciones disponibles.

Buffer Estructura que almacena temporalmente datos para su posterior procesamiento o remuestreo en algoritmos de aprendizaje.

***Epoch* (época)** Recorrido completo de optimización sobre el conjunto de datos recolectado para una iteración; puede reutilizarse durante varios *epochs*.

Logit Puntuación real sin normalizar que una red de política produce para una acción antes de convertirla en probabilidad.

***Minibatch* (minilote)** Subconjunto reducido de muestras extraído de un lote utilizado para estimar el gradiente y actualizar los parámetros de un modelo durante el entrenamiento.

Replay buffer Estructura que almacena experiencias previas para remuestrearlas durante el entrenamiento de un método basado en valor.

Rollout Secuencia ordenada de transiciones generada mientras una política interactúa con el entorno.

Softmax Función que transforma un conjunto de puntuaciones reales en probabilidades no negativas cuya suma es uno.

checkpoint Archivo que guarda los parámetros del modelo y, si está implementado, el estado del entrenamiento en una iteración intermedia o final. Puede utilizarse para reanudar o evaluar desde ese punto; su existencia no demuestra que el ensayo esté completo ni que sus resultados sean admisibles.

Actor (Aprendizaje por Refuerzo) Componente de la arquitectura del modelo de aprendizaje responsable de seleccionar y ejecutar la acción en el entorno basándose en el estado actual y la política que está aprendiendo.

Agente Entidad computacional que observa el estado de su entorno y ejecuta acciones según una política determinada con el objetivo de maximizar su recompensa a largo plazo.

Ciclo (Ingeniería de Tránsito) Secuencia completa y repetitiva de todas las fases semafóricas programadas en una intersección particular.

Cohorte de evaluación Conjunto de vehículos definido por un criterio común de inclusión, como el ingreso durante un mismo intervalo de simulación, sobre el que se calculan las medidas de desempeño.

Cola vehicular Fila de vehículos que se detienen o avanzan muy lentamente debido a una restricción en la capacidad de la vía, tal como la luz roja de un semáforo.

Crítico (Aprendizaje por Refuerzo) Componente encargado de estimar la función de valor, prediciendo la recompensa futura esperada para evaluar la calidad de las acciones tomadas por el Actor y así guiar su actualización.

Densidad vehicular Número de vehículos presentes en un tramo o longitud específica de un carril en un instante dado.

Distribución loguniforme Distribución uniforme en el logaritmo de una variable positiva. Asigna la misma probabilidad a intervalos con igual razón entre sus extremos y permite explorar distintas escalas de magnitud.

Distribución uniforme continua Distribución que asigna la misma probabilidad a intervalos de igual longitud dentro de un rango. En la búsqueda de hiperparámetros, delimita propuestas para parámetros continuos.

Dominio de búsqueda Conjunto de rangos u opciones permitidos para los hiperparámetros en una búsqueda de configuraciones.

Ensayo de búsqueda de hiperparámetros Evaluación de una configuración concreta de hiperparámetros mediante entrenamiento y registro del objetivo y sus métricas. Puede terminar antes del presupuesto previsto y no equivale a un episodio ni a una iteración de entrenamiento.

Entorno El sistema dinámico con el que interactúa el agente. Responde a las acciones del agente produciendo transiciones de estado y emitiendo señales de recompensa.

Episodio Secuencia de interacción que comienza al reiniciar el entorno y termina cuando se alcanza una condición terminal o el horizonte máximo definido.

Estado Representación cuantitativa o cualitativa de la situación actual del entorno en un instante de tiempo específico. Sirve como base para que el agente seleccione su próxima acción.

Estudio de búsqueda de hiperparámetros Búsqueda que agrupa ensayos y su historial bajo una función objetivo y un espacio de búsqueda. En el protocolo de este proyecto, cada estudio se asocia con un algoritmo, un escenario y una arquitectura.

Fase (Ingeniería de Tránsito) Estado temporal de un controlador semafórico que habilita el derecho de paso a uno o más movimientos o corrientes vehiculares compatibles entre sí para que crucen una intersección de forma segura.

Flujo vehicular Cantidad de vehículos que pasan por una sección transversal específica de un carril o calzada durante un intervalo de tiempo determinado.

Función de Valor Estimación matemática de la cantidad total de recompensa que un agente puede esperar acumular en el futuro, comenzando desde un estado particular y siguiendo una política específica.

Grado de vecindad Cantidad de agentes vecinos directos de un agente. En el entorno utilizado, una conexión vial entre carriles controlados establece una relación de vecindad registrada para ambos agentes, incluso si la vía es de sentido único.

Hiperparámetro Valor de configuración que se fija antes de un entrenamiento, como la tasa de aprendizaje o el tamaño de las capas ocultas. Se distingue de los parámetros de la red, que se ajustan durante el aprendizaje.

Intervalo de confianza Rango que expresa la incertidumbre de una estimación. Su nivel nominal indica la proporción de estudios repetidos en la que el procedimiento cubriría el valor verdadero bajo sus supuestos; no asigna una probabilidad a ese valor una vez calculado el intervalo.

Iteración de entrenamiento Ciclo completo en el que se recolectan datos con una política y se realizan una o más actualizaciones de sus parámetros.

Lote (*batch*) Conjunto de muestras o transiciones procesadas conjuntamente durante una operación de entrenamiento..

Memoria temporal de espera Intervalo finito sobre el cual SUMO acumula la espera consultada mediante TraCI; se distingue de la espera total registrada por viaje en TripInfo.

Máscara de acciones Representación de las acciones habilitadas en una decisión; excluye del proceso de selección las acciones no admisibles. En el control semafórico, la admisibilidad depende de las fases, las transiciones y los tiempos definidos por el control; la máscara no constituye por sí sola una garantía de seguridad vial en campo.

Observación Subconjunto de información del estado completo del entorno que es efectivamente visible o medible por un agente individual, especialmente relevante en escenarios de observabilidad parcial.

Par de políticas Unidad experimental compuesta por dos políticas entrenadas con la misma semilla y evaluadas con el mismo conjunto de semillas de simulación.

Percentil Valor que deja una proporción determinada de las observaciones por debajo o a su altura. El percentil 99 corresponde aproximadamente al 99 % de las observaciones.

Política (Aprendizaje por Refuerzo) Estrategia que define el comportamiento del agente en un momento dado, mapeando los estados observados a probabilidades de selección de las acciones disponibles.

Proceso de Decisión de Markov Marco matemático utilizado para modelar la toma de decisiones en situaciones donde los resultados son en parte aleatorios y en parte bajo el control de un agente tomador de decisiones.

Recompensa Señal escalar devuelta por el entorno que evalúa cuán buena o mala fue la acción ejecutada por el agente en un estado particular. El objetivo del agente es maximizar la suma total de estas señales.

Semilla Número que inicializa un generador de números pseudoaleatorios. Permite repetir su secuencia cuando se conservan el procedimiento y la configuración; usar la misma semilla no garantiza resultados idénticos si se modifican las decisiones del agente o el entorno.

Serialización Conversión de un objeto o mensaje a una representación que puede almacenarse o transmitirse y luego reconstruirse.

Telemetría Conjunto de mediciones que el simulador registra durante la ejecución, como velocidades, posiciones, detenciones y tiempos de espera.

Teletransporte de SUMO Mecanismo de SUMO, denominado originalmente *teleport*, que retira temporalmente un vehículo bloqueado y lo reinserta para evitar que la simulación se detenga.

Transición (Aprendizaje por Refuerzo) Registro de un paso de interacción que contiene el estado u observación actual, la acción ejecutada, la recompensa recibida, el estado siguiente y, cuando corresponde, un indicador de terminación.

Ventaja (Aprendizaje por Refuerzo) Métrica que cuantifica cuánto mejor o peor resulta tomar una acción particular en un estado dado, en comparación con el desempeño promedio esperado según la política actual.

1

Introducción

En las últimas décadas, el crecimiento del parque automotor y la urbanización sostenida han convertido a la congestión vehicular en uno de los principales desafíos para la movilidad urbana. Según los reportes de flota vehicular de Argentina elaborados por la Asociación de Fábricas Argentinas de Componentes (AFAC), y tomando 2012 como línea base, el volumen de vehículos aumentó un 21,7 % en 2017. Esta tendencia continuó durante la última década y alcanzó un incremento acumulado del 44,4 % en 2025 (véase la Figura 1.1) [Aso18, Aso26].

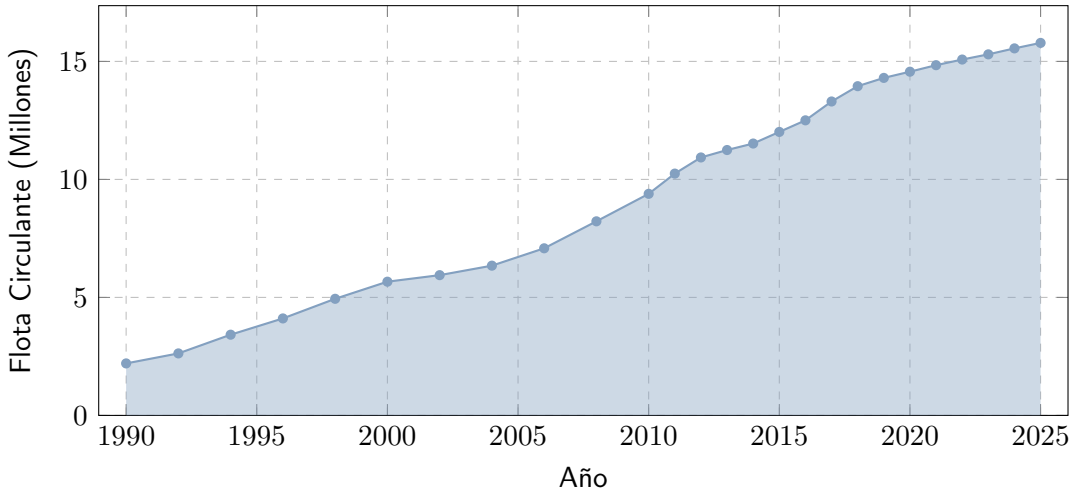


Figura 1.1: Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC.

Como refleja la Figura 1.1, la flota vehicular activa en Argentina experimentó una expansión sostenida entre 1990 y 2025, pasando de aproximadamente 2,2 millones a 15,78 millones de unidades operativas [Aso18, Aso26]. En los reportes de AFAC, la flota circulante comprende automóviles y vehículos comerciales livianos y pesados en condiciones de circular, excluyendo acoplados y remolques; representa una estimación del parque activo bajo esa definición, no el total histórico de unidades patentadas ni la cantidad de viajes realizados. Este incremento acumulado ejerce una presión creciente sobre redes viales consolidadas que poseen una capacidad física fija, agudizando la formación de colas y demoras en las intersecciones urbanas críticas.

La congestión deteriora las condiciones de operación de la red e incrementa las demoras y las detenciones de los vehículos [CyMR18]. Este trabajo aborda decisiones de fases semafóricas en tres escenarios sintéticos y diseña su evaluación también para una red del microcentro de la Ciudad de Mendoza, construida con información parcial de tránsito. La comparación local y vecinal de MAPPO se circunscribe a las redes sintéticas; Mendoza se incorpora al diseño de búsqueda y se comprueba por separado con sus programas semafóricos. La geometría y las condiciones de construcción se especifican en la sección 3.2.

Frente a la congestión, una alternativa que no requiere modificaciones importantes de la infraestructura consiste en ajustar la operación de las intersecciones mediante sistemas semafóricos adaptativos. Los enfoques tradicionales pueden presentar limitaciones para responder a variaciones en el flujo vehicular [PW16]. En este contexto, los avances en inteligencia artificial (*Artificial Intelligence*, IA) y en particular el aprendizaje por refuerzo (*Reinforcement Learning*, RL) permiten modelar los semáforos como agentes que aprenden políticas de control a partir de la interacción con el entorno. Las acciones corresponden a cambios de fase y la recompensa puede definirse, por ejemplo, como el negativo del tiempo de espera acumulado. Diversos estudios han mostrado la viabilidad de este enfoque, desde algoritmos tabulares aplicados a una sola intersección [Wie00] hasta modelos de redes coordinadas basados en redes neuronales profundas y control multiagente [W$^+$19].

Sin embargo, la aplicación simultánea de este enfoque en múltiples intersecciones introduce desafíos adicionales. Un esquema centralizado puede resultar poco escalable, pues el espacio de acciones conjunto crece multiplicativamente con el número de intersecciones [ACS24]. En una formulación de aprendizaje por refuerzo multiagente (*Multi-Agent Reinforcement Learning*, MARL) con aprendizaje independiente (*Independent Learning*, IL), cada semáforo optimiza su propia intersección sin considerar explícitamente el efecto de sus decisiones sobre el resto de la red [AS22]. Además, como todos los agentes aprenden y modifican sus políticas al mismo tiempo, cada uno observa un entorno no estacionario. Esto dificulta la convergencia de los algoritmos de agente único [FNF$^+$17].

Por esta razón, este trabajo estudia el control multiagente con un objetivo de coordinación vial. Cada agente recibe una recompensa individual. Las búsquedas informadas

emplean una señal local, mientras que una línea experimental separada compara MAPPO local con una variante que agrega observaciones y recompensa vecinales. En MAPPO y HAPPO, un crítico centralizado incorpora información conjunta durante el entrenamiento. La observación vecinal, la recompensa mixta y la centralización del crítico son mecanismos distintos cuyo uso depende del protocolo.

El objetivo general es desarrollar y evaluar procedimientos de control semafórico mediante aprendizaje por refuerzo multiagente. Se consideran MAPPO y HAPPO con entrenamiento centralizado y ejecución descentralizada, e IPPO e IDQN con aprendizaje independiente. La búsqueda organiza una fase amplia y otra refinada para estos cuatro algoritmos en las tres redes sintéticas y en Mendoza, con dominios propios de cada escenario. En las tres redes sintéticas se compara además una configuración de cooperación vecinal de MAPPO con su variante local mediante pares con semillas compartidas. La comprobación operacional de Mendoza usa los programas semafóricos incluidos en la red, no políticas aprendidas.

La propuesta incorpora una codificación de la espera acumulada por carril mediante una transformación logarítmico-lineal en I niveles. La resolución no uniforme distingue con mayor detalle valores bajos de esa medida y acota los valores representados a partir de un umbral. La elección de cómo representar el tránsito influye en el problema de aprendizaje y cuenta con antecedentes en control semafórico [GR18]. En este trabajo, la codificación se utiliza en todos los agentes.

También se estudia una adaptación de HAPPO para intersecciones con distintas geometrías, fases disponibles y demandas. A diferencia de la actualización de actores utilizada en MAPPO, esta adaptación los actualiza secuencialmente y repondera las ventajas individuales según los cambios de política precedentes [KCW+22]. El crítico centralizado incorpora información conjunta durante el entrenamiento. La comparación con MAPPO, IPPO y IDQN examina el desempeño de procedimientos completos en esos escenarios, sin atribuir de antemano una ventaja a HAPPO ni aislar el efecto de su actualización secuencial.

1.1. Motivación

La motivación de este trabajo se centra en estudiar cómo coordinar agentes que aprenden en una misma red vial. El control adaptativo basado en aprendizaje por refuerzo permite ajustar decisiones a las condiciones observadas [MMLK25]. En una red de múltiples intersecciones, interesa examinar cómo esa adaptación puede incorporar los efectos de las decisiones locales sobre los cruces vecinos.

Las diferencias de demanda, geometría y fases disponibles entre intersecciones motivan, además, el estudio de políticas que puedan adaptarse a esas condiciones. La comparación de MAPPO y HAPPO con los esquemas independientes IPPO e IDQN permite examinar distintas formas de aprendizaje y coordinación bajo una misma formulación del problema.

1.2. Organización del documento

Para dar respuesta a los objetivos planteados y estructurar el desarrollo de la investigación, el presente documento se organiza en cinco capítulos y un apéndice técnico.

Este capítulo contextualiza la congestión vehicular urbana y presenta el alcance y los objetivos del trabajo. El capítulo 2 desarrolla los fundamentos de ingeniería de tránsito, procesos de decisión de Markov, aprendizaje profundo y aprendizaje por refuerzo multiagente necesarios para seguir el diseño experimental.

El capítulo 3 describe los escenarios, la representación de observaciones, acciones y recompensas, y los protocolos de búsqueda y evaluación. Comprende las tres redes sintéticas y la red urbana de Mendoza, construida con conteos parciales; la comprobación de esta última con los programas semafóricos de la red se distingue de la búsqueda de políticas. El capítulo 4 organiza la evidencia de búsqueda, la comparación entre las variantes local y vecinal de MAPPO y la comprobación operacional de Mendoza, que no evalúa políticas aprendidas.

Finalmente, el capítulo 5 sintetiza las conclusiones principales a la luz de los objetivos propuestos, delimita el alcance y las limitaciones del estudio, y proyecta líneas de trabajo futuro. Como complemento técnico, el apéndice A describe el diseño de software del paquete `marlTLC` mediante diagramas de clases en lenguaje unificado de modelado (*Unified Modeling Language*, UML) y reúne los dominios, las semillas y las especificaciones necesarias para reproducir los experimentos.

2

Marco teórico

Este capítulo presenta los fundamentos teóricos del trabajo y vincula la ingeniería de tránsito con el aprendizaje por refuerzo para la toma de decisiones secuenciales en redes viales. Estos conceptos permiten modelar una red semafórica como un entorno estocástico y formular políticas de control adaptativo con agentes homogéneos y heterogéneos.

Primero se introducen los conceptos de ingeniería de tránsito y simulación (sección 2.1). Luego se desarrollan los fundamentos del aprendizaje por refuerzo y su integración con redes neuronales (sección 2.2), seguidos de su extensión al caso multiagente y de los algoritmos MAPPO y HAPPO (sección 2.3). Finalmente, se revisan los antecedentes recientes de control semafórico multiagente según sus esquemas de coordinación, representación del estado y tratamiento de la heterogeneidad (sección 2.3.7).

2.1. Ingeniería de tránsito y simulación

La ingeniería de tránsito es una rama de la ingeniería de transporte, tradicionalmente vinculada con la ingeniería civil. Se ocupa de la planificación, el diseño geométrico y la operación segura y eficiente del tránsito en calles, carreteras y redes viales, de acuerdo con manuales especializados como el *Traffic Engineering Handbook* [PW16] y con la literatura clásica del área [CyMR18]. En este trabajo, sus conceptos se abordan desde la perspectiva del modelado computacional y la gestión adaptativa. Permiten describir el funcionamiento físico de la red, caracterizar la oferta y la demanda de circulación e identificar las variables sobre las que puede intervenir el sistema de control.

Históricamente, la práctica de la ingeniería de tránsito puso un fuerte énfasis en aumentar la capacidad física vial mediante la construcción de nuevas obras o el ensanche de

calzadas [PW16]. Sin embargo, el crecimiento sostenido del parque automotor evidenció que la ampliación de la infraestructura no constituye una solución definitiva al problema de la congestión. En áreas urbanas consolidadas, además, modificar el trazado vial resulta costoso y, en algunos casos, difícil de realizar [CyMR18]. Por ello, la práctica contemporánea complementa la infraestructura física con estrategias de control dinámico del tránsito, priorizando la optimización de los recursos existentes mediante dispositivos de control semafórico.

La operación de una red vial puede analizarse a partir de la relación entre la demanda vehicular y la capacidad disponible. La demanda vehicular (D) corresponde a la cantidad de vehículos que requieren desplazarse por un determinado sistema vial en un lapso dado, mientras que la oferta vial o capacidad (c) representa la cantidad máxima de vehículos que pueden circular por dicho espacio físico bajo determinadas condiciones de operación [CyMR18]. Cuando la demanda se aproxima a la capacidad, el tránsito se vuelve inestable; cuando la supera ($D > c$), se presentan condiciones de sobresaturación o flujo forzado, caracterizadas por detenciones frecuentes y grandes demoras. En este sentido, la congestión no se entiende solo como una percepción general de mal funcionamiento, sino como un fenómeno con manifestaciones físicas medibles: demoras, colas y detenciones [FA11].

Estas magnitudes cumplen una función doble en el modelo computacional: pueden formar parte de las observaciones y de la señal de recompensa de los agentes, y también se utilizan para evaluar el desempeño del agente. Su definición permite vincular la descripción física del tránsito con la formulación posterior del problema de aprendizaje.

2.1.1. Terminología de tránsito

Antes de formular el problema de control, es necesario distinguir algunos términos que se utilizarán a lo largo del capítulo. En particular, se diferencian los movimientos que realizan los vehículos, las fases que habilitan esos movimientos y los intervalos durante los cuales se mantienen las indicaciones semafóricas [Fed08].

En este contexto, se emplearán las siguientes nociones:

- **Intersección (*intersection*), acceso (*approach*) y carril (*lane*):** una intersección es el área física y operacional en la que confluyen dos o más corrientes de tránsito. Cada una de las calzadas de aproximación que ingresan a ella constituye un acceso [PW16], organizándose internamente en uno o más carriles destinados a canalizar los flujos vehiculares [CyMR18].

- **Movimiento vehicular y movimientos en conflicto:** un movimiento vehicular representa la trayectoria específica que describe un vehículo al cruzar la intersección hacia una salida determinada, por ejemplo, un giro o un movimiento directo [W$^+$19]. Dos movimientos pueden generar puntos de conflicto cuando sus trayectorias se intersectan (véase la Figura 2.2). Cuando la geometría y la señalización no permiten

proteger su operación simultánea, esos movimientos deben programarse en intervalos distintos por razones de seguridad [FA11].

- **Fase (*phase*), ciclo (*cycle*) e intervalo (*interval*):** una fase es una configuración de señales que habilita simultáneamente un conjunto de movimientos compatibles [PW16]. El ciclo corresponde a la secuencia completa de fases necesaria para atender los movimientos de la intersección. Un intervalo es el período durante el cual las indicaciones semafóricas permanecen constantes [CyMR18].

La Figura 2.1 ilustra estos conceptos en una intersección de cuatro accesos. Los números del 1 al 8 identifican movimientos vehiculares individuales y las etiquetas $P2$, $P4$, $P6$ y $P8$ identifican cruces peatonales del esquema. Esta numeración sigue de manera esquemática una convención habitual de los manuales de temporización semafórica; no representa una configuración experimental ni una asignación obligatoria de fases para el caso de estudio. Las flechas, los accesos y las etiquetas de calles permiten distinguir trayectorias y sentidos, mientras que los cruces peatonales muestran otros usuarios que también deben considerarse al ordenar los movimientos. En este trabajo, la Figura se utiliza para separar la noción geométrica de movimiento de la regla semafórica que habilita un conjunto de movimientos compatibles [Fed08]. Por ejemplo, dos movimientos que no comparten un punto de conflicto pueden incluirse en una misma fase, mientras que los que sí lo comparten deben atenderse en fases compatibles o sucesivas.

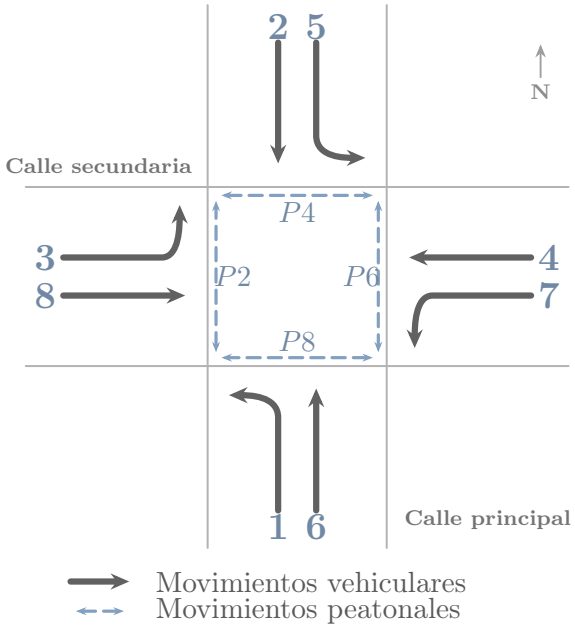


Figura 2.1: Convención esquemática de movimientos vehiculares, fases y cruces peatonales concurrentes en una intersección de cuatro accesos, adaptada conceptualmente del manual de temporización semafórica de Administración Federal de Carreteras (*Federal Highway Administration*, FHWA) [Fed08].

La Figura 2.2 presenta tres formas esquemáticas de relación entre trayectorias. Hay

cruce cuando dos trayectorias se intersectan en un punto; *convergencia* cuando trayectorias distintas se unen en una misma corriente; y *divergencia* cuando una corriente se separa en trayectorias diferentes. Los puntos coloreados marcan el lugar de interacción que se analiza en cada caso. Cada relación puede originar un punto de conflicto según la geometría y la simultaneidad de los movimientos: por eso, la incompatibilidad no depende solo de que las líneas se relacionen, sino de que su operación conjunta comprometa la seguridad. En el modelo semafórico, estos casos se utilizan para determinar qué movimientos pueden habilitarse en una misma fase y cuáles deben retenerse; la Figura muestra relaciones geométricas, no una programación temporal concreta.

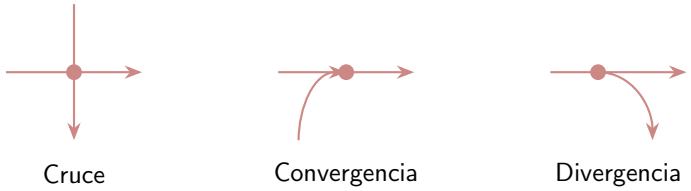


Figura 2.2: Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.

En arterias urbanas, el tránsito se desarrolla bajo condiciones de flujo interrumpido: los vehículos no avanzan únicamente por su interacción con otros vehículos, sino también por la presencia de intersecciones, señales y dispositivos de control de tránsito [PW16]. Las intersecciones son puntos críticos de la red porque allí confluyen distintos accesos y movimientos vehiculares de cruce, convergencia y divergencia que pueden resultar incompatibles entre sí, como resume la Figura 2.2. Por ello, su operación requiere ordenar estos movimientos y asignar la prioridad de paso de manera segura y eficiente.

El semáforo es uno de los dispositivos de control de tránsito utilizados para regular la circulación en intersecciones. Su función es asignar la prioridad de paso en forma alternada entre corrientes vehiculares incompatibles, evitando que distintos movimientos intenten ocupar simultáneamente el mismo espacio de cruce [FA11]. Desde el punto de vista del modelado algorítmico, esta capacidad de modificar la prioridad de paso convierte al semáforo en el actuador mediante el cual un sistema de control puede intervenir sobre la operación de la intersección y afectar, directa o indirectamente, las demoras y colas observadas en la red.

2.1.2. Fundamentos de la dinámica vehicular

Para modelar y optimizar la operación de una red vial, es necesario caracterizar físicamente el movimiento de los vehículos. La ingeniería de tránsito estudia este fenómeno a partir de las características fundamentales de la corriente de tránsito, que permiten describir las condiciones de operación de una vía o intersección mediante magnitudes observables como el flujo, la velocidad y la densidad [PW16]. Cal y Mayor distingue estas magnitudes entre variables principales, que describen el comportamiento agregado del flu-

jo vehicular, y variables asociadas, que expresan relaciones temporales y espaciales entre vehículos consecutivos [CyMR18].

A nivel macroscópico, la corriente de tránsito se representa mediante variables agregadas como la tasa de flujo, la densidad y los promedios de velocidad. A nivel microscópico, se analizan magnitudes propias de vehículos individuales e interacciones entre pares consecutivos, como la velocidad puntual, el intervalo y el espaciamiento [CyMR18]. Esta separación entre escalas es fundamental para el modelado computacional: las variables macroscópicas resumen el estado operacional de los accesos de la red, mientras que las variables microscópicas describen el comportamiento de cada vehículo dentro del simulador [FA11]. La correspondencia y las relaciones de conversión entre las variables de ambas escalas se resumen en la Tabla 2.1.

En el nivel macroscópico, las variables principales de la corriente de tránsito son [CyMR18]:

- **Flujo o tasa de flujo (q):** representa la cantidad de vehículos que atraviesan una sección transversal de la vía por unidad de tiempo. Si el período de observación es menor a una hora, se proyecta como una tasa horaria equivalente en vehículos por hora, calculada como $q = N/T$, donde N es el número de vehículos observados y T es la duración del intervalo de medición expresado en horas.

- **Densidad o concentración (k):** indica el número de vehículos que ocupan simultáneamente una longitud específica de vía, expresada en vehículos por kilómetro, calculada como $k = N/d$, donde d representa la longitud del tramo de vía analizado. En la práctica, se suele estimar indirectamente a través del porcentaje de ocupación temporal registrado por detectores inductivos [PW16].

- **Velocidad media espacial (v_s):** es la velocidad media de los vehículos que ocupan un tramo de vía en un instante determinado. Corresponde al promedio armónico de las velocidades puntuales de los vehículos individuales y representa la magnitud de velocidad consistente con la dinámica del flujo espacial.

- **Velocidad media temporal (v_t):** representa el promedio aritmético de las velocidades de los vehículos que cruzan una sección transversal fija de la vía durante un período de tiempo determinado.

En el nivel microscópico, las variables asociadas son [CyMR18]:

- **Velocidad puntual (v_i):** es la velocidad instantánea a la que un vehículo individual (i) atraviesa una sección transversal específica de la vía.

- **Intervalo temporal (h):** es el tiempo transcurrido entre el paso de dos vehículos consecutivos por una sección fija de la calzada, medido respecto a puntos homólogos,

por ejemplo, los paragolpes delanteros. El intervalo promedio en segundos (h_{prom}) se relaciona con la tasa de flujo como $h_{\text{prom}} = 3600/q$.

- **Espaciamiento simple (s):** es la distancia física entre puntos homólogos de dos vehículos consecutivos en un instante de tiempo. El espaciamiento promedio en metros (s_{prom}) se relaciona de forma inversa con la densidad de la corriente de tránsito como $s_{\text{prom}} = 1000/k$.

La Figura 2.3 complementa estas definiciones con las magnitudes que se miden en el espacio y en el tiempo. El largo del vehículo l y la separación libre e son distancias, expresadas en metros; el espaciamiento s es la distancia entre puntos homólogos de vehículos consecutivos, también en metros. El avance temporal β es el tiempo asociado al recorrido del largo del vehículo a la velocidad v, la brecha temporal g es el tiempo entre el paso del vehículo precedente y el siguiente respecto de puntos de referencia distintos, y el intervalo temporal h es el tiempo entre pasos de puntos homólogos; β, g y h se expresan en segundos. Las cotas inferiores representan relaciones espaciales y las superiores relaciones temporales; no son dos vehículos distintos ni dos mediciones independientes. En este trabajo, estas magnitudes sirven para vincular la descripción individual de los vehículos con las variables agregadas del flujo.

Tabla 2.1: Comparación y correspondencia de variables de tránsito en escalas macroscópica y microscópica.

Escala	**Variable Principal**	**Relación / Conversión**
Macroscópica	Flujo (q, vehículos por hora)	$q = 3600/h_{\text{prom}}$
	Densidad (k, vehículos por kilómetro)	$k = 1000/s_{\text{prom}}$
	Velocidad media espacial (v_s, kilómetros por hora)	$q = k \cdot v_s$
	Velocidad media temporal (v_t, kilómetros por hora)	$v_t = v_s + \frac{\sigma_s^2}{v_s}$ (Relación de Wardrop)
Microscópica	Velocidad puntual (v_i, kilómetros por hora)	Variable individual de cada vehículo
	Intervalo temporal (h, segundos)	Tiempo entre pasos de vehículos sucesivos
	Espaciamiento simple (s, metros)	Distancia espacial entre paragolpes homólogos

La Figura 2.3 presenta de manera conjunta estas distancias y tiempos entre vehículos, y permite interpretar cómo las magnitudes microscópicas describen la separación y el avance de una corriente.

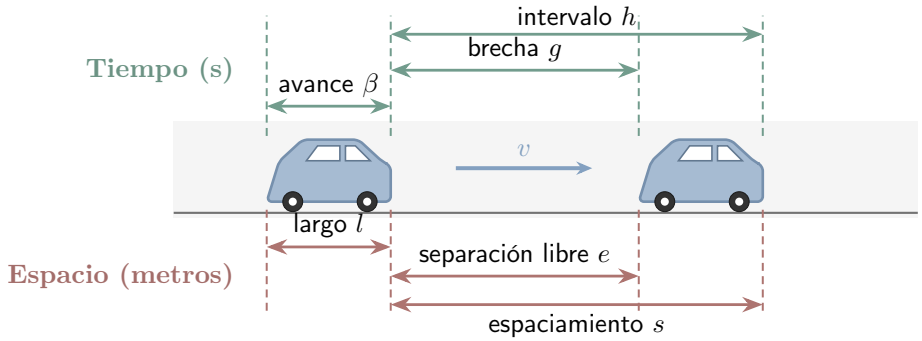


Figura 2.3: Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s).

Estas variables macroscópicas se relacionan de manera directa a través de la ecuación fundamental del flujo de tránsito:

$$q = k \cdot v_s \tag{2.1}$$

donde q es la tasa de flujo, k es la densidad y v_s es la velocidad media espacial. La ecuación (2.1) utiliza la velocidad media espacial, ya que relaciona el flujo y la cantidad de vehículos presentes en un tramo. La velocidad media temporal (v_t) es la media aritmética de las velocidades observadas en un punto durante un intervalo, mientras que v_s, asociada al promedio de las velocidades de una muestra espacial tomada sobre un tramo, se estima en la práctica como media armónica de las velocidades puntuales medidas en una sección durante un intervalo. Cuando existe variabilidad entre las velocidades, ambas magnitudes pueden diferir. La relación entre ellas se expresa mediante la relación de Wardrop [FA11]:

$$v_t = v_s + \frac{\sigma_s^2}{v_s} \tag{2.2}$$

donde σ_s^2 representa la varianza de la distribución de las velocidades de los vehículos en el espacio. La ecuación (2.2) muestra que, dado que la varianza es no negativa ($\sigma_s^2 \geq 0$), se cumple $v_t \geq v_s$, con igualdad únicamente cuando todos los vehículos circulan a la misma velocidad. En una intersección semaforizada, una indicación roja puede reducir localmente la velocidad media y aumentar la acumulación de vehículos en los accesos. Estos efectos se manifiestan en la formación de colas y en el incremento de las demoras [CyMR18].

Métricas de ineficiencia operativa

La interrupción del flujo causada por semáforos o saturación genera efectos adversos que los sistemas de control buscan mitigar. Siguiendo la terminología de la ingeniería de tránsito, las principales manifestaciones medibles de esta ineficiencia son las demoras, las colas y las detenciones [FA11]:

- **Demoras:** representan el tiempo adicional que un vehículo debe invertir para atravesar una intersección respecto del tiempo que le tomaría circular a velocidad de flujo libre. En intersecciones semaforizadas, una parte de esa demora aparece por la espera normal durante la luz roja. Otras demoras se producen cuando las llegadas de vehículos varían, cuando la demanda se acerca o supera la capacidad de la intersección, o cuando quedan colas sin disiparse al finalizar el verde, de modo que algunos vehículos deben esperar más de un ciclo para cruzar.

- **Colas:** corresponden al número de vehículos acumulados detrás de una línea de detención a la espera de traspasarla. Esta medida espacial es especialmente relevante porque, cuando la cola supera la longitud del arco de aproximación, puede bloquear la intersección precedente y propagar la congestión al resto de la red.

- **Detenciones:** cuantifican la cantidad de veces que un vehículo debe reducir su velocidad hasta cero durante el recorrido. Una alta frecuencia de detenciones deteriora la continuidad operativa del flujo y suele asociarse con mayores procesos de aceleración y desaceleración.

Estas magnitudes permiten describir el desempeño operacional de una intersección y de la red. En la formulación computacional, también pueden utilizarse para construir observaciones, señales de recompensa o criterios de evaluación del agente.

Descripción probabilística del flujo y agrupamientos vehiculares

En el modelado de tránsito, las llegadas vehiculares no siempre pueden representarse adecuadamente como una secuencia determinista y constante. Para flujos vehiculares bajos y medios, y cuando las llegadas no están fuertemente condicionadas por semáforos previos, es habitual aproximar el número de vehículos que llega a una sección durante un intervalo de tiempo mediante un proceso de Poisson [CyMR18].

Esta aproximación supone independencia estadística entre llegadas y permite describir la variabilidad observada en la demanda de acceso a una intersección [CyMR18]. La probabilidad discreta $P(X)$ de que lleguen exactamente X vehículos en un intervalo de tiempo se define como:

$$P(X) = \frac{m^X \cdot e^{-m}}{X!} \tag{2.3}$$

donde $X \in \mathbb{N}_0$ es la variable aleatoria discreta que representa el número de arribos vehiculares, $m > 0$ representa el valor esperado o media de llegadas en dicho intervalo, y e es la base del logaritmo natural. La ecuación (2.3) formaliza esta probabilidad discreta. Bajo el mismo supuesto, los intervalos de tiempo continuos (h) medidos entre vehículos sucesivos se describen mediante una distribución de probabilidad exponencial negativa [CyMR18]:

$$P(h \geq t) = e^{-\frac{q}{3600} \cdot t} \tag{2.4}$$

donde h es el intervalo temporal en segundos, q es el flujo en vehículos por hora, y t es el umbral de tiempo analizado. La ecuación (2.4) expresa la probabilidad de que el intervalo supere dicho umbral.

Esta representación probabilística supone independencia entre arribos sucesivos y resulta adecuada para flujos bajos o moderados en condiciones de flujo libre, sin influencia de semáforos cercanos en intersecciones precedentes [CyMR18]. El supuesto pierde validez en condiciones de congestión o en redes urbanas semaforizadas. En estos casos, los semáforos interrumpen el flujo agrupando los vehículos, lo que introduce dependencia espacio-temporal entre arribos consecutivos. La Figura 2.4 compara conceptualmente los intervalos temporales resultantes bajo ambas condiciones. En ella, $h_{\text{mín}}$ representa un umbral mínimo conceptual para el intervalo mostrado en la curva con agrupamientos vehiculares; no es una estimación obtenida de datos de este trabajo. Las curvas y ese umbral tienen, por lo tanto, función ilustrativa y no constituyen una distribución empírica particular.

En este trabajo, la limitación de esos modelos analíticos motiva el uso de simulación microscópica para analizar la operación semafórica bajo condiciones variables. En el simulador *Simulation of Urban Mobility* (SUMO), la formación, dispersión e interacción de los agrupamientos vehiculares emergen de la dinámica simulada de los vehículos y de sus cambios de carril. Los procesos aleatorios del simulador se inicializan con una semilla (*seed*), un número que permite repetir su secuencia pseudoaleatoria bajo la misma configuración y versión. Cambiar la semilla permite estudiar variaciones de la simulación. Repetirla no hace iguales los resultados si también cambia el agente o sus decisiones.

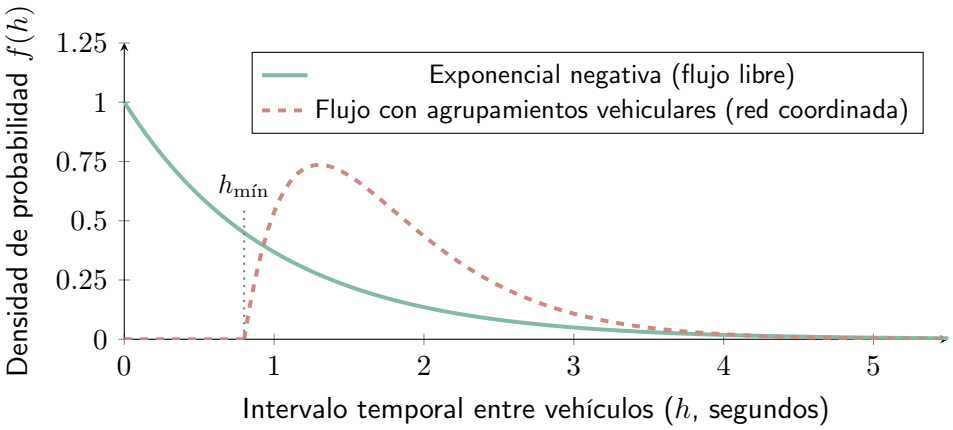


Figura 2.4: Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de agrupamientos vehiculares. Las curvas representan situaciones conceptuales y no una distribución empírica particular.

La Figura 2.4 se interpreta comparando la forma de ambas curvas. El eje horizontal representa el intervalo h entre vehículos y el eje vertical la densidad de probabilidad $f(h)$, es decir, la concentración relativa de intervalos alrededor de cada valor. En el modelo

exponencial, la densidad disminuye desde el origen y asigna mayor frecuencia relativa a los intervalos breves. En la representación con agrupamientos vehiculares, la densidad comienza después de $h_{\text{mín}}$ y alcanza un máximo alejado del origen, lo que ilustra que el arribo de un vehículo queda condicionado por la separación y la coordinación con los que lo preceden. La comparación permite reconocer por qué el supuesto de arribos independientes puede resultar insuficiente en una red semaforizada.

2.1.3. Sistemas de control de tránsito

Para ordenar los movimientos en conflicto y regular el paso en las intersecciones urbanas se utilizan sistemas de control de tránsito [PW16]. El semáforo asigna el derecho de paso de manera alternada mediante indicaciones luminosas rojas, amarillas y verdes [CyMR18].

En el modelo computacional, el semáforo actúa como el mecanismo mediante el cual el controlador interviene sobre la red. La selección de fases, la duración del verde efectivo y la coordinación entre intersecciones modifican la prioridad de paso y afectan la evolución de las colas y las demoras.

Sobre esta base terminológica, la programación temporal de un semáforo se describe mediante la duración de las fases, la longitud de ciclo y la definición de los intervalos de transición y despeje. Estos parámetros determinan cuánto tiempo recibe prioridad cada conjunto de movimientos compatibles y condicionan directamente la capacidad de descarga, las demoras y la seguridad operacional de la intersección [PW16].

En relación con los giros izquierdos, los manuales de temporización distinguen entre operación permisiva, protegida y protegida-permisiva. En una fase permisiva, el giro se ejecuta aprovechando brechas en la corriente opuesta; en una fase protegida, el giro recibe una indicación exclusiva y los movimientos conflictivos permanecen detenidos; en una fase protegida-permisiva, ambas condiciones se combinan durante distintos intervalos del ciclo [Fed08]. En este análisis, la distinción resulta útil porque muestra que una fase no es solo un color del semáforo, sino una regla operacional que habilita ciertos movimientos, impone cesión de paso en otros y restringe los movimientos en conflicto.

Para garantizar la seguridad vial, las transiciones entre fases no pueden ejecutarse de manera instantánea, sino que deben respetar intervalos de cambio y despeje ($t_{\text{amarillo}} + t_{\text{rojo_todo}}$) [PW16]. En esta investigación, esos tiempos se consideran restricciones operativas de seguridad previamente definidas en la configuración del simulador y del sistema semafórico. En consecuencia, su determinación no forma parte del proceso de decisión del agente, cuyo control se limita a la selección o extensión de fases dentro de un esquema temporal compatible con dichas restricciones.

Modelado de la capacidad y el verde efectivo

Desde el punto de vista operativo, la capacidad de descarga de un acceso semaforizado depende del tiempo efectivo de servicio que recibe cada fase y de la tasa con la que la

fila puede disiparse cuando el movimiento tiene prioridad [FA11]. En la Figura 2.5, $q_{\text{máx}}$ representa la tasa de descarga de saturación, g el intervalo de verde efectivo, y l_1 y l_2 las pérdidas inicial y final de servicio, respectivamente; el tiempo se expresa en segundos y la tasa de flujo en vehículos por hora. La barra inferior distingue los intervalos de rojo, verde y amarillo, mientras que la curva superior muestra cómo la descarga alcanza y abandona el régimen de saturación. La descarga observada no depende solo de la duración nominal del verde, sino también de pérdidas de arranque, transiciones y otras restricciones temporales del control. La Figura es conceptual: estos parámetros no se estiman en este trabajo. En su lugar, los efectos se representan mediante la dinámica microscópica del simulador y la configuración temporal del sistema semafórico, ya que esas interacciones determinan las colas, demoras y detenciones que el controlador busca mejorar.

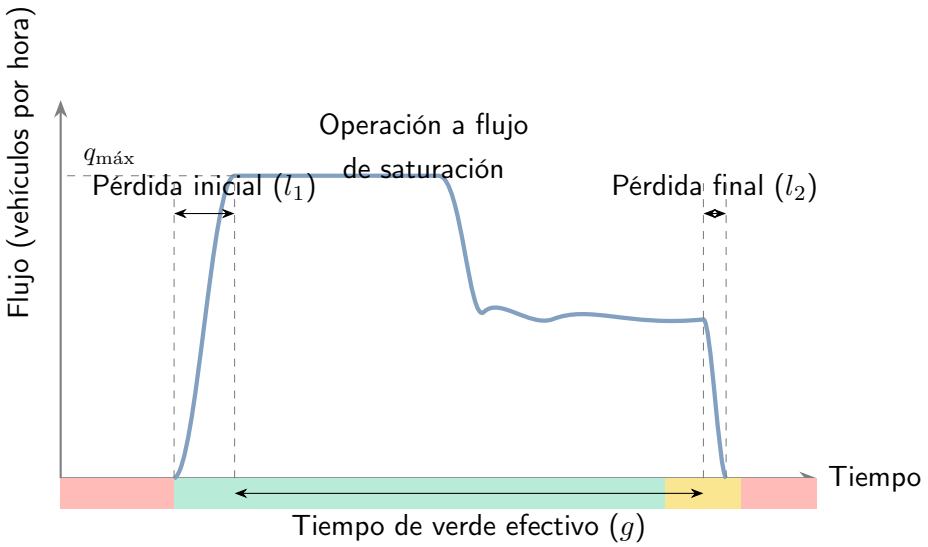


Figura 2.5: Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.

La Figura 2.5 debe leerse como una relación temporal: las pérdidas l_1 y l_2 reducen el intervalo en que la fase puede descargar vehículos a la tasa $q_{\text{máx}}$. Por tanto, el verde efectivo g no equivale necesariamente al verde nominal programado en el controlador.

Clasificación de los controladores de semáforos

De acuerdo con su mecanismo operativo y su grado de respuesta a la demanda vehicular, los controladores de semáforos se clasifican clásicamente en control de tiempo fijo y control accionado por el tránsito [PW16]. En el primero, los planes de señales se repiten de manera preprogramada a partir de datos históricos y no cambian en respuesta a la demanda instantánea. En el segundo, la intersección utiliza detectores en los accesos para variar dinámicamente el inicio o la duración de las fases según la demanda observada en tiempo real.

Incluso en los esquemas accionados, la operación permanece sujeta a parámetros límite

de seguridad y funcionamiento, como el verde mínimo ($g_{\text{mín}}$) y el verde máximo ($g_{\text{máx}}$) [PW16]. Estos parámetros restringen cuánto puede sostenerse o finalizarse una fase y, en este trabajo, forman parte de las condiciones operativas dentro de las cuales el controlador toma decisiones.

2.1.4. Simulación de tránsito

Debido a la densidad y complejidad de las redes urbanas, la sincronización y optimización de los tiempos semafóricos requiere representar la evolución del tránsito bajo condiciones variables. Los modelos analíticos y agregados resultan útiles bajo determinados supuestos, pero pueden ser limitados para describir la formación de colas y las interacciones locales durante la operación dinámica de una red [PW16]. De acuerdo con la taxonomía clásica [FA11], los modelos de tránsito pueden organizarse en tres niveles de resolución. Los modelos macroscópicos describen el tránsito de manera agregada mediante variables como flujo, densidad y velocidad; los mesoscópicos representan entidades intermedias, como agrupamientos vehiculares; y los microscópicos modelan individualmente los vehículos, sus rutas y su evolución dentro de la red. Para esta investigación resulta especialmente relevante este último nivel, pues permite examinar cómo las decisiones semafóricas inciden en la formación de colas, las detenciones y las interacciones locales entre vehículos.

La simulación de tránsito constituye una representación abstracta del fenómeno real. Su objetivo no es reproducir visualmente una escena particular, sino modelar los procesos que determinan la evolución de la circulación mediante reglas de comportamiento y variables observables. La calidad de los resultados depende de la representación de la red, la demanda, la flota, los modelos de comportamiento y las reglas de control. Por ello, la simulación debe entenderse como un entorno experimental sujeto a supuestos y parámetros, cuyos resultados requieren una evaluación operacional acorde con el objetivo del estudio. La correspondencia entre el fenómeno y su representación se resume en la Figura 2.6: el tránsito real constituye la referencia y el simulador representa sus vehículos, red y reglas dinámicas para permitir experimentos controlados.



Figura 2.6: Correspondencia entre la circulación real y su representación microscópica en SUMO. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

La Figura 2.6 muestra, de forma sintética, la correspondencia entre una escena vial y su representación microscópica en SUMO. Su función es ilustrar cómo los vehículos y la infraestructura se transforman en entidades computacionales; no constituye una validación del simulador ni una representación del escenario utilizado en este trabajo.

SUMO pertenece a la categoría microscópica descrita por Fernández [FA11]. Es una *suite* libre y de código abierto iniciada en el Instituto de Sistemas de Transporte del Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*, DLR), que permite modelar sistemas de tránsito intermodales, incluidos vehículos, transporte público y peatones. También ofrece herramientas para importar redes, calcular rutas, visualizar y evaluar simulaciones, además de interfaces para controlar su ejecución [DLR26].

En SUMO, la evolución vehicular se actualiza en pasos temporales discretos sobre una representación espacial continua. El entorno permite modelar calles con múltiples carriles, cambios de carril, distintos tipos de vehículos, rutas individuales, semáforos y salidas de simulación a nivel de vehículo, carril, arco o detector [DLR26].

La interfaz gráfica del programa `sumo-gui` permite inspeccionar visualmente la red, los vehículos y algunos valores de la simulación durante la ejecución. Esta visualización cumple una función de apoyo para la inspección del escenario y la depuración del modelo; las métricas utilizadas para la evaluación se obtienen de las salidas de simulación y de las interfaces de control [DLR26]. La interfaz y su función de inspección se muestran en la Figura 2.7; la interfaz gráfica de usuario (*Graphical User Interface*, GUI) no constituye la fuente de las métricas ni reemplaza su cálculo operacional.

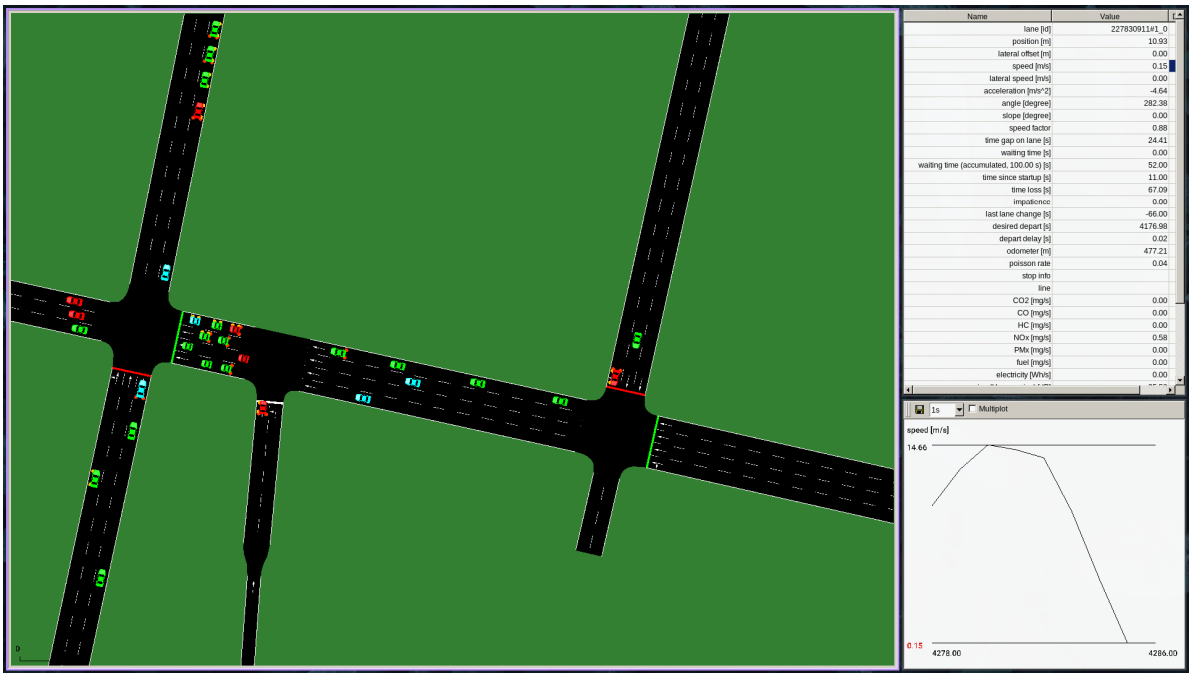


Figura 2.7: Interfaz del programa `sumo-gui` durante una simulación microscópica. El panel izquierdo presenta una vista general de la red; el derecho muestra información sobre un vehículo particular; el recuadro inferior grafica su velocidad y el superior muestra los valores de sus variables relacionadas. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

La Figura 2.7 reúne cuatro zonas de inspección. A la izquierda se observa la red completa, útil para localizar la parte del escenario que se está examinando. A la derecha se muestra una vista ampliada de la calzada y de un vehículo seleccionado, mientras que el gráfico superpuesto en la parte inferior representa la evolución temporal de su velocidad, expresada en metros por segundo. La franja inferior contiene el registro de mensajes y advertencias producidos durante la ejecución, y la barra superior agrupa los controles de avance, pausa, tiempo y escala de visualización. En conjunto, estos elementos permiten verificar visualmente la presencia de vehículos, su movimiento y posibles inconsistencias de configuración. En este trabajo la interfaz se usa con fines de inspección y depuración; la evaluación cuantitativa se realiza con las salidas de simulación y las interfaces de control [DLR26].

Esta granularidad vuelve a la simulación microscópica especialmente adecuada para estudiar control semafórico mediante aprendizaje por refuerzo. Las decisiones sobre fases y tiempos de servicio pueden evaluarse mediante sus consecuencias sobre la dinámica simulada: colas, tiempos de espera, detenciones, ocupación de carriles, completitud de viajes y propagación de congestión. En la formulación computacional, las variables de tránsito constituyen observaciones, las fases admisibles constituyen acciones y la evolución simulada constituye la transición del entorno. La definición operacional de estas variables y su procedimiento de cálculo se presentan en la metodología.

Representación microscópica de intersecciones en SUMO

La naturaleza microscópica de la simulación no se reduce a representar visualmente vehículos. Cada unidad modifica su velocidad y posición de acuerdo con modelos de seguimiento vehicular y cambio de carril, los parámetros de su tipo de vehículo y las restricciones de la red. De este modo, variables como la aceleración, la separación entre vehículos, la velocidad deseada y la disponibilidad de carriles intervienen en la formación y dispersión de colas. Los valores concretos utilizados para la flota y para estos parámetros pertenecen a la configuración experimental y se detallan en la metodología [DLR26].

En este contexto, el simulador de tránsito constituye el entorno operativo sobre el cual se formaliza el problema de control semafórico. La Figura 2.8 esquematiza la representación microscópica de una intersección con circulación por la derecha [DLR26], en la que se distinguen carriles, líneas de detención y conexiones:

- **Carriles entrantes ($L_{\mathbf{in}}$) y salientes ($L_{\mathbf{out}}$):** los carriles entrantes conducen el flujo vehicular hacia la intersección y los salientes reciben la descarga tras cruzar el nodo, indicados mediante flechas que señalan el sentido de circulación.
- **Línea de detención (*stop line*):** líneas blancas continuas próximas al final de cada carril entrante, delante de las cuales se detienen los vehículos cuando la fase no habilita su movimiento.
- **Conexiones (*links*):** relaciones que conectan un carril entrante con uno saliente a través del área de cruce. En SUMO, cada enlace controlado se asocia con un índice y con un estado de señal (G para verde prioritario, g para verde con cesión de paso, y para amarillo y r para rojo). En la fase conceptual ilustrada, las flechas verdes corresponden a los movimientos directos opuestos habilitados del eje horizontal, mientras que las flechas rojas señalan las conexiones retenidas.

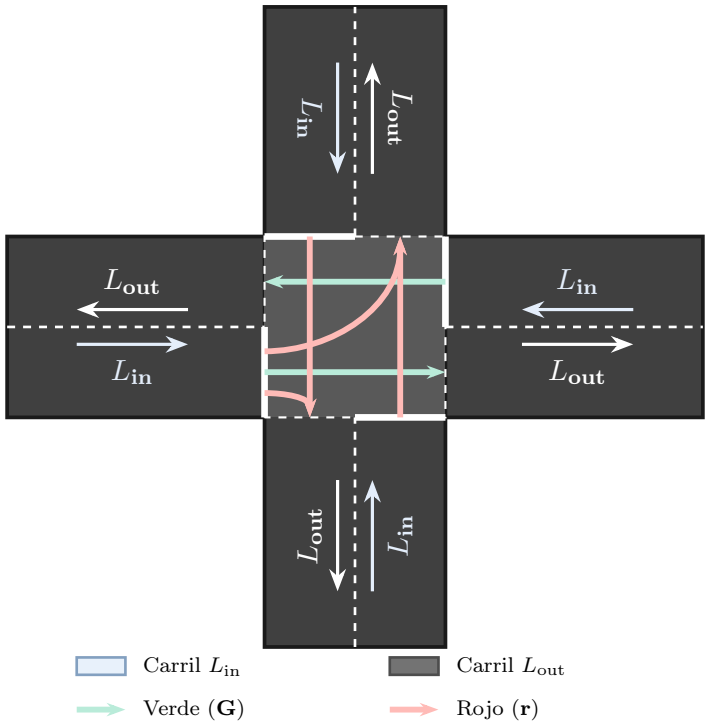


Figura 2.8: Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos.

Por consiguiente, una fase no modifica la geometría ni el sentido de los carriles, sino que determina qué conexiones pueden utilizarse durante cada intervalo de control. Esta representación resulta directamente compatible con la formulación algorítmica del problema, donde una acción válida debe seleccionar una fase compatible con la geometría de la intersección y respetar las transiciones y restricciones de seguridad definidas por el entorno.

Componentes y configuración de SUMO

La *suite* SUMO reúne aplicaciones para construir redes, generar demandas, ejecutar simulaciones, visualizar sus resultados y controlar la ejecución mediante interfaces externas [DLR26]:

- **netconvert:** aplicación que importa redes desde distintas fuentes y compila su representación en el formato nativo de SUMO. También permite reconstruir o ajustar elementos de la red, como las conexiones y los programas semafóricos.
- **netedit:** editor gráfico para crear y modificar de forma interactiva nodos, calles, carriles, conexiones, restricciones de giro y elementos de control semafórico.
- **osmWebWizard:** herramienta que automatiza la generación de un escenario a partir de un área seleccionada de OpenStreetMap (OSM). Se considera una alter-

nativa de importación y no reemplaza el procesamiento específico realizado con `netconvert`.

- **interfaz de control del tránsito (*Traffic Control Interface*, TraCI):** interfaz que permite consultar el estado de una simulación en ejecución y modificarlo durante el avance paso a paso, incluyendo variables de vehículos, carriles, detectores y semáforos [DLR26].

- **libsumo:** implementación embebida que ofrece la interfaz de programación de SUMO sin la comunicación mediante conexiones de red (*sockets*) de la modalidad TraCI estándar. Esta alternativa resulta útil para integrar el simulador con controladores externos y reducir la sobrecarga de comunicación.

Desde la perspectiva del modelado computacional, una simulación de SUMO se construye a partir de archivos lenguaje de marcado extensible (*Extensible Markup Language*, XML) de red, demanda, elementos adicionales y configuración [DLR26]. Los archivos TAZ pueden utilizarse durante la estructuración espacial y la generación de demanda, mientras que las salidas de la simulación se solicitan mediante opciones de configuración específicas. Sus funciones generales se resumen en la Tabla 2.2:

Tabla 2.2: Archivos de entrada y configuración empleados en una simulación de SUMO.

Archivo	**Extensión**	**Función en el Modelo Computacional**
Red Vial	`.net.xml`	Topología física, carriles, conexiones y lógicas semafóricas.
Rutas y Demanda	`.rou.xml`	Tipos de vehículos, viajes, flujos e itinerarios.
Zonas TAZ	`.taz.xml`	Zonas para estructurar espacialmente la demanda.
Elementos Adicionales	`.add.xml`	Detectores, tipos de vehículo y programas auxiliares.
Configuración	`.sumocfg`	Archivos de entrada, opciones de ejecución y salidas.

Métricas de evaluación operacional

La simulación produce medidas para evaluar el control semafórico. En este trabajo se consideran las siguientes categorías de indicadores:

- **Demora y tiempo de espera:** cuantifican el tiempo adicional o el tiempo detenido que experimentan los vehículos durante su recorrido. Pueden analizarse de forma agregada en la red y desagregarse por intersección, acceso o agente, siempre que se mantenga una definición operacional consistente [CyMR18, DLR26].

- **Completitud y flujo servido:** la cantidad de vehículos que completa su itinerario y su relación con la demanda generada permiten distinguir una reducción genuina de la demora de una disminución del flujo atendido.
- **Equidad espacial:** la distribución de las demoras entre intersecciones permite identificar si una política mejora el desempeño global a costa de concentrar la espera en accesos o agentes particulares.
- **Indicadores de validez de la simulación:** el mecanismo de teletransporte de SUMO, denominado originalmente *teleport*, retira temporalmente vehículos bloqueados y los reinserta para evitar que la simulación se detenga. Sus activaciones, los vehículos pendientes y otras incidencias permiten detectar bloqueos o condiciones que comprometen la comparabilidad de una ejecución.

Las definiciones operacionales, fórmulas, unidades, ventanas de evaluación y procedimientos de agregación empleados en este trabajo se presentan en la metodología. Las recompensas y las pérdidas de optimización corresponden al entrenamiento de los algoritmos de aprendizaje y se desarrollan en las secciones dedicadas a RL y MARL.

En este marco, SUMO permite observar el estado de la red y modificar las fases semafóricas durante la ejecución mediante interfaces de control. Esta capacidad convierte el control semafórico en un problema de decisión secuencial, lo que motiva la introducción del aprendizaje por refuerzo en la siguiente sección [DLR26].

2.2. Aprendizaje por refuerzo (RL)

El aprendizaje automático (*Machine Learning*, ML) suele dividirse en tres paradigmas. En el aprendizaje supervisado (*Supervised Learning*, SL), el modelo se entrena con ejemplos etiquetados que indican la salida esperada. En el aprendizaje no supervisado (*Unsupervised Learning*, UL), se busca descubrir estructura en datos no etiquetados, por ejemplo mediante agrupamiento o reducción de dimensionalidad. El aprendizaje por refuerzo (RL) aborda un problema distinto: una entidad aprende mediante interacción con un entorno qué acciones elegir para maximizar una recompensa acumulada [SB18].

Esta diferencia es central. En RL no existe, en general, un supervisor que indique cuál era la acción correcta en cada situación. El agente aprende por experiencia, ensayando acciones, observando sus consecuencias y ajustando su comportamiento a partir de las recompensas obtenidas. Además, las consecuencias relevantes de una acción no siempre se manifiestan de forma inmediata: una decisión puede producir una recompensa baja en el corto plazo, pero conducir a estados favorables en el futuro. Por este motivo, el aprendizaje por refuerzo se ocupa explícitamente del problema de la recompensa diferida y de la asignación de crédito a decisiones pasadas [SB18].

La formulación básica distingue entre el agente (*agent*), que aprende y toma decisiones, y el entorno (*environment*), que representa todo aquello externo al agente con lo cual

este interactúa. En cada paso, el agente observa una situación, selecciona una acción y recibe una nueva observación junto con una recompensa: este ciclo constituye la interacción agente–entorno. El comportamiento está determinado por una política (*policy*), denotada por π, que especifica cómo elegir acciones a partir de los estados u observaciones disponibles. La señal de recompensa (*reward signal*) define el objetivo inmediato, mientras que la función de valor estima la conveniencia de estados o acciones en términos de recompensa futura. Sutton y Barto también distinguen el modelo del entorno, entendido como una descripción de sus transiciones y recompensas; cuando no se conoce o no se utiliza explícitamente, se habla de métodos libres de modelo (*model-free*).

En este trabajo, se utiliza controlador para el sistema general que regula las señales y agente para la entidad que aprende y selecciona acciones. Cada agente regula una intersección y la red vial simulada en SUMO constituye su entorno. La selección de una fase constituye la acción, mientras que la evolución posterior del tránsito produce nuevas observaciones y una señal de recompensa que orienta el aprendizaje [DLR26].

Para fijar ideas, considérese un par de intersecciones vecinas. El agente observa localmente las colas y demoras de su intersección, selecciona una fase y modifica la corriente de vehículos que llegará a la siguiente. Esa consecuencia aguas abajo puede cambiar sus arribos y colas, y se refleja en una recompensa asociada al desempeño de la red. El retorno permite acumular esos efectos a lo largo de varias decisiones, en lugar de evaluar una fase solo por su consecuencia inmediata.

Un aspecto característico de RL es el dilema entre exploración y explotación. El agente debe explotar acciones que, según su experiencia, producen buenos resultados, pero también debe explorar alternativas menos conocidas que podrían conducir a políticas superiores. Si explora demasiado, desperdicia oportunidades de obtener recompensa; si explota demasiado pronto, puede converger a comportamientos subóptimos. Este equilibrio es especialmente relevante en problemas de control, donde las acciones modifican la experiencia futura disponible para el aprendizaje [SB18].

2.2.1. Procesos de decisión de Markov (MDP)

El marco matemático estándar para estudiar problemas de aprendizaje por refuerzo de un solo agente es el proceso de decisión de Markov (*Markov Decision Process*, MDP). En un MDP finito, el agente y el entorno interactúan en pasos temporales discretos $t = 0, 1, 2, \ldots$: en cada instante, el agente recibe $S_t \in \mathcal{S}$, selecciona $A_t \in \mathcal{A}(S_t)$ y el entorno emite $R_{t+1} \in \mathcal{R}$ y transita a S_{t+1} [LML+25]. La notación se ilustra en la Figura 2.9.

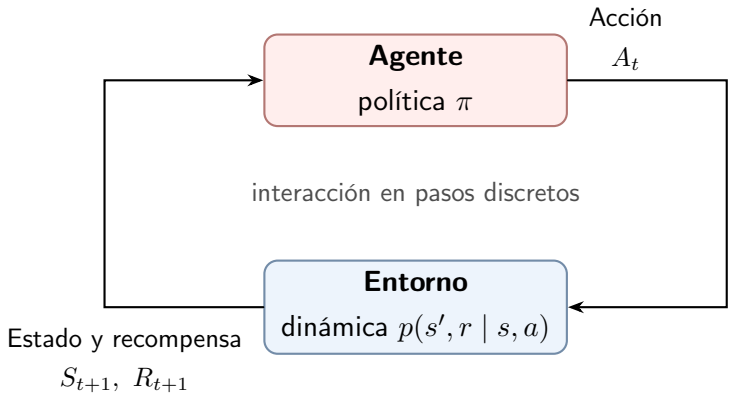


Figura 2.9: Interacción agente-entorno en un proceso de decisión de Markov [SB18].

De forma compacta, un MDP finito queda caracterizado por la tupla $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, donde $\mathcal{S}$ es el espacio de estados, $\mathcal{A}$ el espacio de acciones, P la dinámica de transición probabilística, R la función de recompensa y $\gamma \in [0, 1)$ el factor de descuento.

La dinámica del entorno queda definida por la función $p(s', r \mid s, a)$, que especifica la probabilidad condicional conjunta de observar el próximo estado s' y la recompensa r al ejecutar la acción a en el estado s:

$$p(s', r \mid s, a) \doteq \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\} \tag{2.5}$$

En esta formulación, $S_t \in \mathcal{S}$ y $A_t \in \mathcal{A}$ denotan el estado y la acción en el instante t, mientras que $S_{t+1} \in \mathcal{S}$ y $R_{t+1} \in \mathbb{R}$ representan el estado sucesor y la recompensa escalar devueltos por el entorno. Si bien conceptualmente el estado comprende toda la información necesaria para determinar la dinámica, en el control semafórico el agente suele disponer de observaciones locales de tránsito que constituyen una representación parcial de dicho estado.

El proceso satisface la propiedad de Markov si la distribución condicional del próximo estado y recompensa depende exclusivamente del estado y la acción presentes, resultando condicionalmente independiente del historial de transiciones previas [SB18].

Una suposición habitual del MDP es que el agente observa un estado suficientemente informativo del entorno. Sin embargo, en muchas aplicaciones reales el agente solo recibe observaciones parciales o ruidosas. En estos casos, el marco se generaliza a un proceso de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*, POMDP), donde el historial de observaciones puede ser necesario para inferir el estado subyacente [ACS24].

Para formalizar la meta de maximizar la recompensa a largo plazo se define el retorno descontado G_t, que incorpora un factor $\gamma \in [0, 1)$ para ponderar la importancia de las

recompensas futuras [SB18]:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.6}$$

En esta formulación, el retorno G_t acumula las recompensas futuras ponderadas por el factor de descuento $\gamma \in [0, 1)$. Un valor de γ cercano a cero privilegia la optimización a muy corto plazo, mientras que valores próximos a uno confieren mayor peso a las consecuencias remotas [ACS24]. En el control semafórico, esta acumulación temporal resulta indispensable para que el agente aprenda a prevenir condiciones de bloqueo o saturación en ciclos futuros, en lugar de limitarse a disipar la cola inmediata a expensas de la red.

2.2.2. Métodos basados en valor y basados en política

Los algoritmos de RL pueden agruparse, a grandes rasgos, en métodos basados en valor y métodos basados en política. Esta distinción no agota la taxonomía del campo, pero permite introducir las familias de algoritmos más relevantes para este trabajo.

Esta clasificación distingue dos formas de obtener una regla de decisión. Los métodos basados en política aprenden directamente una función que relaciona cada estado con una acción o con una distribución de probabilidad sobre las acciones. En cambio, los métodos basados en valor aprenden indirectamente la regla de decisión: estiman la conveniencia de los estados o de los pares estado–acción y, a partir de esas estimaciones, determinan qué alternativa conviene ejecutar [SB18]. En el control semafórico, las alternativas corresponden a las distintas fases admisibles de una intersección.

La Figura 2.10 representa esta segunda alternativa. A partir del estado observado, la función de valor estima la conveniencia esperada de las acciones disponibles y la decisión se obtiene comparando esas estimaciones. El esquema corresponde directamente al caso de una función de valor de acción $Q(s, a)$; si se utiliza una función de valor de estado $V(s)$, la elección de la acción requiere valorar los estados sucesores que produciría cada alternativa. Por ello, $V(s)$ y $Q(s, a)$ cumplen funciones relacionadas, pero no intercambiables en la evaluación de los estados y en la selección de acciones.

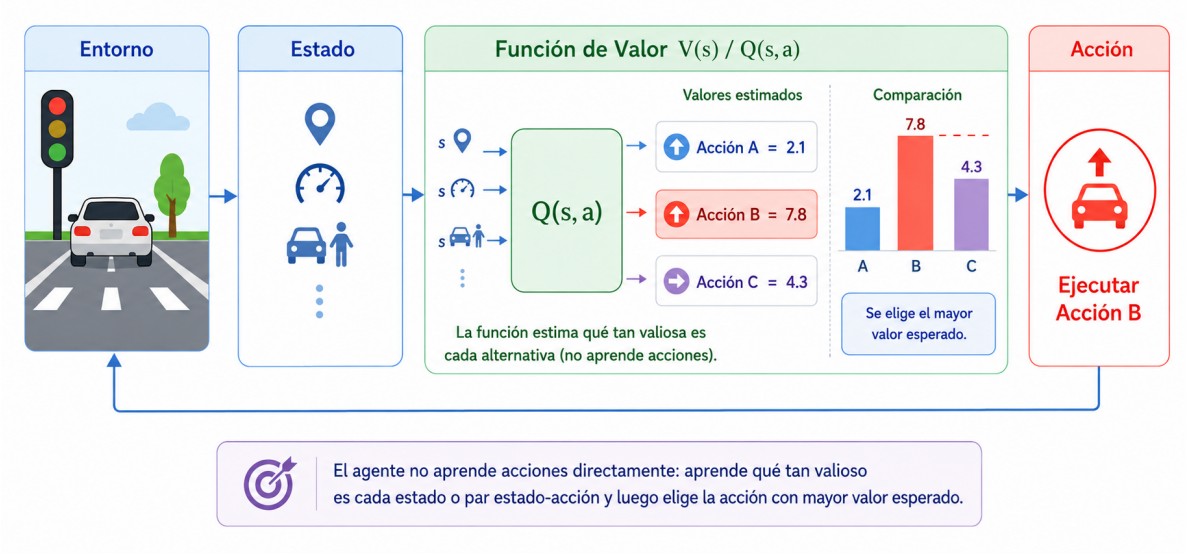


Figura 2.10: Esquema de un método basado en valor. A partir del estado observado s, la función de valor estima el retorno esperado de los pares estado–acción $Q(s, a)$. Las estimaciones para las acciones disponibles se comparan y se selecciona la alternativa con mayor valor esperado para ejecutarla en el entorno. La exploración puede incorporarse mediante estrategias como la selección ε-greedy [SB18].

En los métodos basados en valor, el objetivo es determinar la conveniencia de encontrarse en un estado o de ejecutar una acción concreta. Formalmente, esto se expresa mediante la función de valor de estado $v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]$ y la función de valor de acción $q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$, las cuales satisfacen las relaciones recursivas de las ecuaciones de Bellman [SB18]. En la notación empleada en la Figura 2.10, estas funciones se escriben habitualmente como $V(s)$ y $Q(s, a)$ cuando no es necesario hacer explícita la política respecto de la cual se calcula el valor.

Cuando el propósito es identificar la política óptima en lugar de evaluar una fija, se emplean las ecuaciones de optimalidad de Bellman:

$$q_*(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a\right]. \tag{2.7}$$

En la ecuación (2.7), la esperanza $\mathbb{E}$ promedia las transiciones estocásticas del entorno y la maximización sobre las fases admisibles a' formaliza el principio de optimalidad. En el ámbito semafórico, esta relación expresa el compromiso entre la recompensa inmediata obtenida por la fase activa y la calidad del estado vehicular resultante para las decisiones subsiguientes [SB18].

En ausencia de un modelo analítico de transición, los métodos de TD aproximan estas funciones a partir de la experiencia mediante *bootstrapping*: utilizan una estimación del valor del estado siguiente para actualizar el valor actual. Un ejemplo es Q-learning, cuya regla de actualización converge hacia la función óptima $q_*(s, a)$ en el caso tabular, bajo el supuesto de que todos los pares estado–acción continúen explorándose y que la tasa de

aprendizaje decrezca adecuadamente a lo largo del tiempo [SB18]:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \tag{2.8}$$

donde $\alpha \in (0, 1]$ es la tasa de aprendizaje (*learning rate*) que pondera la magnitud del ajuste, R_{t+1} es la recompensa inmediata observada, y el término $\max_a Q(S_{t+1}, a)$ aproxima el retorno óptimo alcanzable desde el estado sucesor S_{t+1}.

Q-learning es un algoritmo *off-policy*: la política que genera los datos de interacción puede diferir de la política objetivo cuya función de valor se optimiza. Por el contrario, los métodos de gradiente de política operan generalmente de forma *on-policy*, utilizando transiciones producidas por la política en curso de optimización. Esta diferencia resulta decisiva al extender los algoritmos al entorno multiagente, donde las políticas de las intersecciones vecinas evolucionan simultáneamente.

En los métodos basados en política, la regla de decisión se parametriza directamente como $\pi_\theta(a \mid s) = \Pr\{A_t = a \mid S_t = s, \theta\}$, donde $\theta \in \mathbb{R}^d$ representa el vector de parámetros ajustables de la política. La política puede ser determinista, si asigna siempre la misma acción a un estado, o estocástica, si produce una distribución de probabilidad sobre las acciones posibles [SB18]. La Figura 2.11 ilustra este último caso: para el estado observado, la política asigna probabilidades a cada fase admisible y la selección se efectúa mediante muestreo o según la alternativa más probable, de acuerdo con la regla de ejecución adoptada.

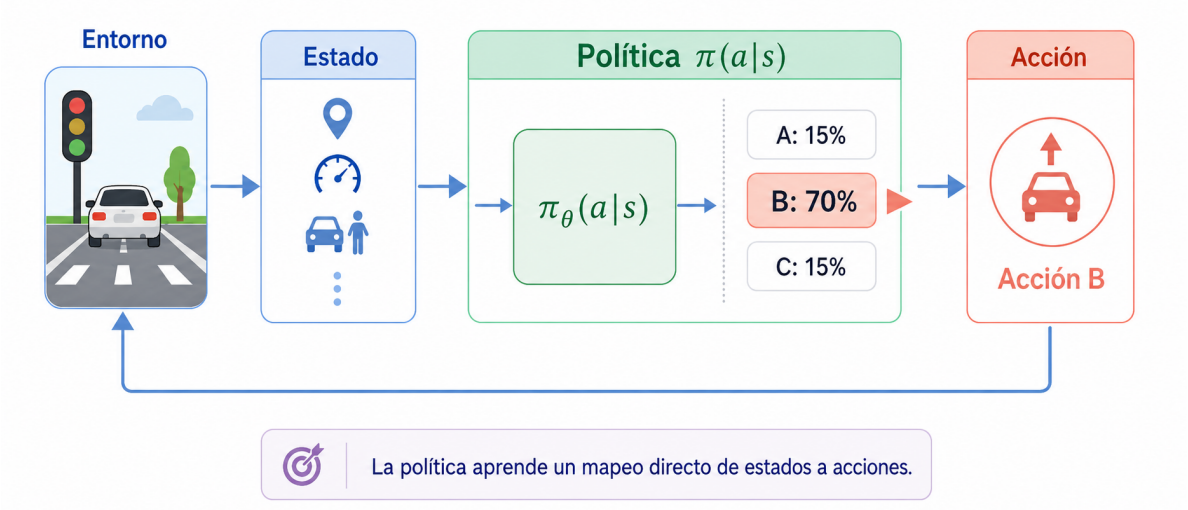


Figura 2.11: Esquema de un método basado en política. A partir del estado observado s, la política parametrizada $\pi_\theta(a \mid s)$ genera directamente una distribución de probabilidad sobre las acciones disponibles. La regla de ejecución puede seleccionar la alternativa de mayor probabilidad o muestrear una acción a partir de dicha distribución [SB18].

La optimización persigue maximizar el rendimiento esperado $J(\theta)$ mediante ascenso

de gradiente:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t) \tag{2.9}$$

donde $\nabla J(\theta_t)$ es el gradiente del rendimiento respecto de los parámetros y $\alpha > 0$ determina la magnitud del paso de actualización. El Teorema del Gradiente de Política (*Policy Gradient Theorem*) proporciona el sustento teórico para estimar analíticamente $\nabla J(\theta_t)$ a partir de trayectorias muestreadas, ajustando la política para elevar la probabilidad de aquellas fases que conducen a menores demoras en la red [SB18].

Los métodos actor-crítico combinan una política parametrizada con una estimación de valor y actualizan ambos componentes de manera concurrente [KT03, SB18]. El actor representa la política $\pi_\theta(a \mid s)$ y determina la distribución utilizada para seleccionar la acción. El crítico no controla el entorno: estima, por ejemplo, el valor de estado $V_\phi(s)$ y produce una señal que indica si el resultado observado fue mejor o peor que lo esperado bajo la política vigente. Los parámetros θ y ϕ pueden optimizarse de forma independiente o compartir una representación común de las observaciones de tránsito.

En una formulación actor-crítico de un paso, después de ejecutar A_t en S_t, el entorno devuelve la recompensa R_{t+1}, el estado S_{t+1} y el indicador terminal d_{t+1}. El crítico calcula el error de diferencia temporal mediante:

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t), \tag{2.10}$$

En la ecuación (2.10), el factor $(1 - d_{t+1})$ anula la contribución del estado siguiente ante transiciones terminales. Cuando V_ϕ aproxima adecuadamente el valor real V^π, δ_t actúa como un estimador de un paso de la ventaja: un valor positivo refleja que la fase ejecutada reportó un desempeño superior al esperado en ese estado de tránsito, mientras que un valor negativo indica lo contrario. El crítico ajusta sus parámetros ϕ minimizando el error temporal, mientras que el actor utiliza esta misma señal δ_t para modular la actualización de la política:

$$\theta \leftarrow \theta + \alpha_\theta \, \delta_t \, \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{2.11}$$

En la regla (2.11), $\alpha_\theta > 0$ es la tasa de aprendizaje del actor y $\nabla_\theta \log \pi_\theta(A_t \mid S_t)$ indica cómo cambiar sus parámetros para aumentar la probabilidad de la acción elegida. Un error δ_t positivo favorece esa acción, mientras que uno negativo reduce su probabilidad. Los métodos posteriores emplean estimadores de ventaja de varios pasos, como estimación generalizada de ventaja (*Generalized Advantage Estimation*, GAE), y procesan conjuntos de transiciones llamados lote (*batch*). Esta regla permite comprender cómo el crítico orienta la actualización del actor.

La Figura 2.12 sintetiza el flujo operativo entre actor y entorno, así como el circuito de evaluación mediante el cual el crítico genera la señal de retroalimentación compartida.

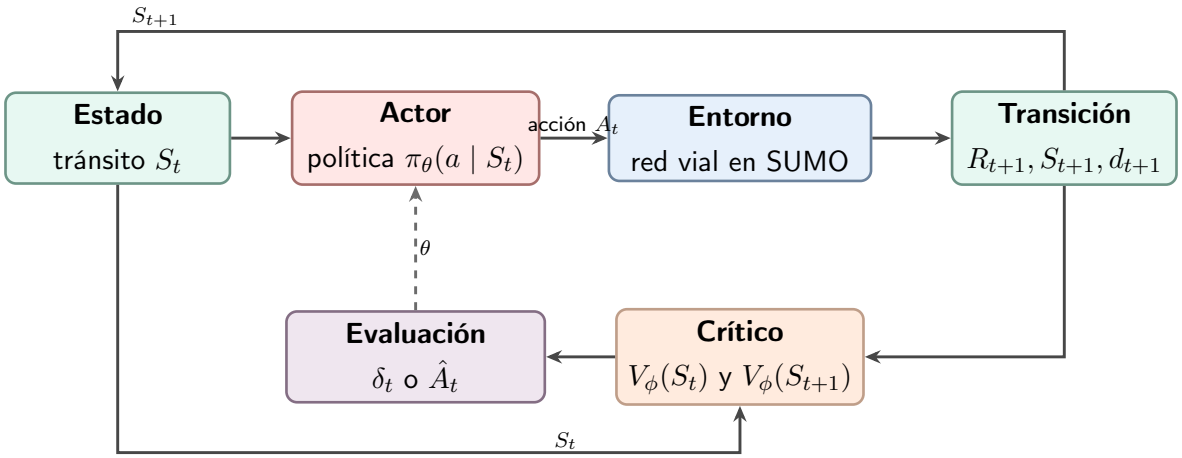


Figura 2.12: Arquitectura actor-crítico aplicada al control semafórico en la formulación de un MDP. El actor selecciona una acción y el crítico evalúa la transición para construir la señal que actualiza ambos componentes. Elaboración propia a partir de [KT03, SB18].

En síntesis, el estado resume las condiciones de tránsito disponibles para el agente, la acción corresponde a la selección de una fase, la recompensa refleja la consecuencia operativa inmediata y el valor estima su impacto a largo plazo. Cuando el espacio de estados es continuo o de gran escala, como ocurre en las redes viales urbanas, las funciones de valor y las políticas requieren aproximadores de alta capacidad representacional. A continuación se introducen las redes neuronales artificiales como el modelo adoptado para este fin, preservando intactos los fundamentos de RL expuestos hasta aquí.

2.2.3. Aprendizaje profundo y redes neuronales

Para estructurar este desarrollo, se expone en primer término la unidad neuronal básica y su composición en redes multicapa; luego se aborda su optimización mediante retropropagación y descenso de gradiente; finalmente, se analiza su integración en RL a través de DQN y PPO. Cabe destacar que la red neuronal actúa como un aproximador de funciones de alta capacidad, mientras que la dirección del aprendizaje proviene exclusivamente de las señales de recompensa generadas por la interacción con el entorno [SB18].

El aprendizaje profundo (*Deep Learning*) se enfoca en el entrenamiento de arquitecturas con múltiples capas de parámetros ajustables capaces de extraer representaciones jerárquicas y relaciones no lineales directamente de los datos [BB24, GBC16]. En este trabajo, esta capacidad se aprovecha para aproximar funciones de valor y políticas de control semafórico donde una formulación tabular resulta impracticable debido a la continuidad y dimensionalidad del estado del tránsito.

Aunque la denominación "neuronal" proviene de una analogía simplificada con modelos biológicos iniciales, una red artificial constituye un modelo matemático determinista cuyos parámetros se optimizan a partir de datos y gradientes de pérdida [GBC16].

La unidad neuronal y redes *feedforward*

En problemas de control con espacios de estados continuos o de gran escala, los métodos tabulares resultan impracticables [SB18]. La aproximación de funciones resuelve esta limitación mediante representaciones parametrizadas, lo que permite generalizar lo aprendido a situaciones similares no observadas. Para este fin se emplean redes neuronales artificiales [BB24].

La unidad neuronal artificial constituye el bloque computacional básico de estas arquitecturas [BB24, GBC16]. Esta procesa un vector de entradas ponderándolo mediante un vector de pesos, añade un término de sesgo y aplica una función de activación no lineal para producir una respuesta escalar. En el ámbito semafórico, las entradas corresponden típicamente a variables observables del tránsito (como tiempos de espera acumulados o longitudes de cola por carril), mientras que los pesos modulan la contribución relativa de cada acceso hacia la decisión final.

La Figura 2.13 ilustra este procesamiento: la transformación afín $z = \mathbf{w}^\top \mathbf{x} + b$ integra las entradas $\mathbf{x} = (x_1, \ldots, x_n)$ con los pesos $\mathbf{w}$ y el sesgo b, antes de que la activación $y = g(z)$ genere la salida. Sin la introducción de estas no linealidades $g(\cdot)$, una red multicapa se reduciría a una transformación afín simple, imposibilitando la captura de patrones de congestión complejos.

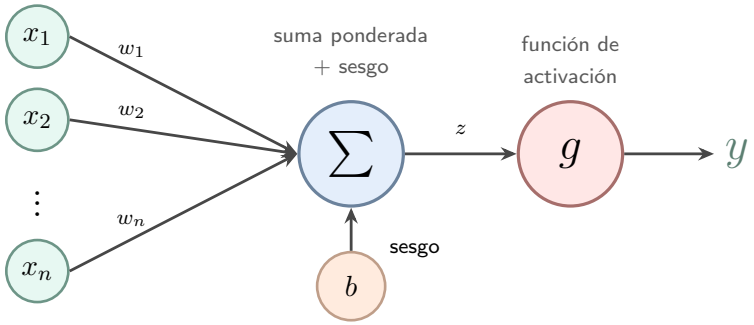


Figura 2.13: Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación.

Como ejemplo conceptual, la red puede recibir las métricas de demora de una intersección y emitir estimaciones de calidad o probabilidades sobre las fases admisibles, orientando la asignación del verde hacia las aproximaciones más críticas.

En una red multicapa, la operación para una unidad u dentro de la capa k se formaliza mediante la ecuación (2.12) [BB24, GBC16]:

$$a_{k,u} = \mathbf{w}_{k,u}^\top \mathbf{x}_{k-1} + b_{k,u}, \qquad y_{k,u} = g_k(a_{k,u}) \tag{2.12}$$

donde $\mathbf{x}_{k-1}$ es el vector de activaciones proveniente de la capa anterior, $\mathbf{w}_{k,u}$ es el vector de pesos, $b_{k,u}$ es el sesgo y g_k es la función de activación. Entre las funciones más empleadas en capas intermedias destacan:

- **unidad lineal rectificada (*Rectified Linear Unit*, ReLU):** $g(z) = \max\{0, z\}$, que propaga linealmente las activaciones positivas y bloquea las negativas, con pendiente uno para valores positivos durante la optimización.
- **Leaky ReLU:** $g(z) = \max\{0, z\} + \alpha \min\{0, z\}$, con $\alpha \ll 1$, que conserva una pendiente pequeña en el semieje negativo para mitigar la inactivación de unidades.
- **Tanh:** $g(z) = \tanh(z)$, que ofrece una respuesta no lineal diferenciable acotada en $(-1, 1)$.

Las redes *feedforward* encadenan estas transformaciones en capas sucesivas sin conexiones recurrentes sin retroalimentación temporal. Para una arquitectura con L capas parametrizadas, la propagación completa se describe como:

$$\mathbf{h}_k = g_k\left(\mathbf{W}_k^\top \mathbf{h}_{k-1} + \mathbf{b}_k\right), \qquad k = 1, \ldots, L, \qquad \mathbf{h}_0 = \mathbf{x}, \qquad \mathbf{y} = \mathbf{h}_L \tag{2.13}$$

donde $\mathbf{W}_k$ y $\mathbf{b}_k$ denotan la matriz de pesos y el vector de sesgos de la capa k, respectivamente, y g_k se aplica componente a componente. En el agente semafórico, el vector de entrada $\mathbf{h}_0 = \mathbf{x}$ contiene las esperas acumuladas por carril y la salida $\mathbf{y} = \mathbf{h}_L$ produce las valoraciones Q o probabilidades de selección de fase.

La Figura 2.14 esquematiza esta composición con dos capas ocultas conectadas en cascada hacia la salida.

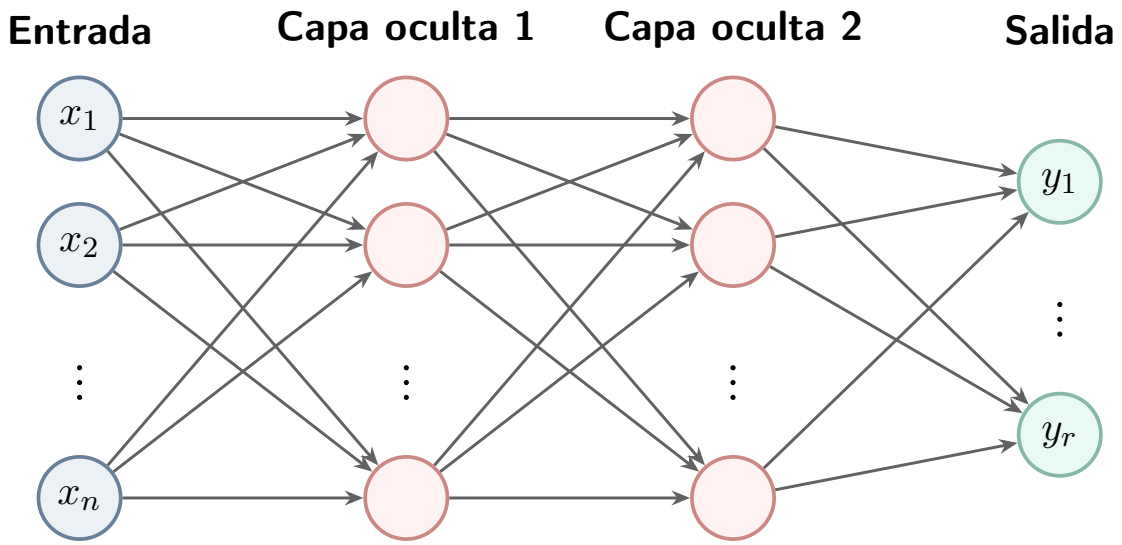


Figura 2.14: Arquitectura conceptual de una red neuronal *feedforward* con dos capas ocultas.

El Teorema de Aproximación Universal es un resultado de existencia: bajo condiciones apropiadas, una red de dos capas parametrizadas, con una capa oculta suficientemente ancha, puede aproximar una función continua sobre un dominio compacto con una precisión arbitraria. El teorema no proporciona por sí mismo un procedimiento para encontrar

los parámetros, no garantiza un entrenamiento eficiente, una buena generalización ni un buen desempeño práctico sobre datos no observados. La profundidad puede ofrecer representaciones jerárquicas y una mayor eficiencia expresiva para ciertas familias de funciones, pero agregar capas o neuronas no garantiza por sí solo un menor error de prueba [BB24].

El proceso de optimización

Durante el entrenamiento, la red procesa las observaciones mediante propagación hacia adelante y contrasta su respuesta frente a un objetivo mediante una función de pérdida escalar diferenciable $\mathcal{L}(\theta)$. El algoritmo de retropropagación (*backpropagation*) calcula analíticamente las derivadas parciales de dicha pérdida respecto de cada peso y sesgo aplicando de forma sistemática la regla de la cadena [BB24].

Los pesos y sesgos son parámetros que se ajustan durante el aprendizaje; los hiperparámetros, como la tasa de aprendizaje y el tamaño de las capas ocultas, son valores de configuración que se fijan antes de entrenar y pueden seleccionarse mediante una búsqueda.

Con estos gradientes, los parámetros se actualizan iterativamente en sentido opuesto a la pendiente del error:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t), \tag{2.14}$$

donde θ reúne los parámetros entrenables de la red, $\nabla_\theta \mathcal{L}(\theta_t)$ es el gradiente de la función de pérdida y $\eta > 0$ es la tasa de aprendizaje. Calcular el gradiente con todos los datos puede resultar costoso. Por ello, se utilizan subconjuntos de muestras, denominados *minibatches*, para realizar cada actualización. Adam adapta el tamaño del paso para cada parámetro a partir de promedios móviles del gradiente y de su cuadrado [BB24].

A diferencia del aprendizaje supervisado convencional —donde la pérdida compara la salida con una etiqueta externa fija—, en RL el objetivo se deriva de las recompensas y transiciones producidas por la interacción con el simulador, y cambia a medida que la política mejora su comportamiento.

2.2.4. Aprendizaje por refuerzo profundo (DRL)

El DRL combina algoritmos de RL con aproximadores neuronales. En el control semafórico, las observaciones del tránsito, como colas, ocupación o velocidades, pueden alimentar la red; sus salidas pueden representar valores de acción o parámetros de una política; y las recompensas asociadas con demoras, detenciones y colas orientan la función de pérdida del algoritmo de aprendizaje. Esta integración resulta útil cuando el espacio de estados es continuo o de alta dimensionalidad, aunque exige mecanismos de estabilización y una evaluación cuidadosa de la eficiencia de las muestras [ACS24].

Factores de inestabilidad del aprendizaje

La integración de aproximadores no lineales con aprendizaje por refuerzo puede inducir inestabilidad numérica o divergencia en las estimaciones si no se controlan las condiciones

de actualización. Sutton y Barto [SB18] describen este fenómeno mediante la denominada tríada mortal (*deadly triad*), originada por la concurrencia de tres elementos:

- **Aproximación de funciones:** el uso de un modelo parametrizado, como una red neuronal, para estimar valores en lugar de almacenarlos individualmente en una tabla.
- ***Bootstrapping*:** la actualización de estimaciones de valor apoyándose en otras estimaciones aprendidas previamente (métodos TD), sin esperar a disponer del retorno completo.
- **Entrenamiento *off-policy*:** el aprendizaje sustentado en una distribución de datos generada por una política de comportamiento distinta de la política objetivo.

Aunque prescindir del *bootstrapping* o de la aproximación de funciones comprometería la eficiencia muestral o la escalabilidad ante redes viales complejas, los algoritmos profundos modernos introducen mecanismos arquitectónicos específicos para amortiguar esta inestabilidad [SB18].

Deep Q-Network (DQN): aproximación neuronal de valores de acción

En los métodos basados en valor, DQN sustituye la tabla de estados y acciones por una red neuronal $Q(s, a; \theta)$ parametrizada por θ [MKS+15]. Ante una observación de tránsito, la red genera una estimación del valor Q para cada fase admisible. La función de acción-valor óptima $q_*(s, a)$ representa el máximo retorno esperado al seleccionar la mejor secuencia de decisiones futuras, objetivo que DQN persigue aproximar a partir de las recompensas del entorno.

Para mitigar la inestabilidad de la tríada mortal —en particular el problema del valor objetivo cambiante, donde actualizar el valor de un estado altera impredeciblemente las estimaciones de estados adyacentes—, DQN incorpora dos componentes estructurales [MKS+15]:

- **Redes objetivo (*Target Networks*):** una red secundaria idéntica cuyos parámetros θ^- se congelan temporalmente. Esta red se utiliza exclusivamente para calcular el término de *bootstrapping* del valor objetivo TD, desacoplando la estimación en curso del objetivo de aprendizaje y amortiguando oscilaciones.
- **Memoria de repetición de experiencias (*replay buffer*):** almacena transiciones históricas $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ en un *buffer* $\mathcal{D}$. La optimización extrae *mini-batches* aleatorios de esta memoria, lo cual mitiga la correlación temporal propia del flujo continuo de vehículos en el simulador y aproxima los datos a la hipótesis de muestras independientes.

Durante la interacción con SUMO, la red en línea evalúa la observación de tránsito y una regla ϵ_{exp}-greedy selecciona la fase a ejecutar; la transición resultante se almacena en $\mathcal{D}$. Durante el entrenamiento, la red en línea actualiza sus pesos θ a partir de los *minibatches* muestreados del *buffer* contra el valor objetivo fijado por la red objetivo, cuyos parámetros θ^- se sincronizan solo de forma periódica (Figura 2.15).

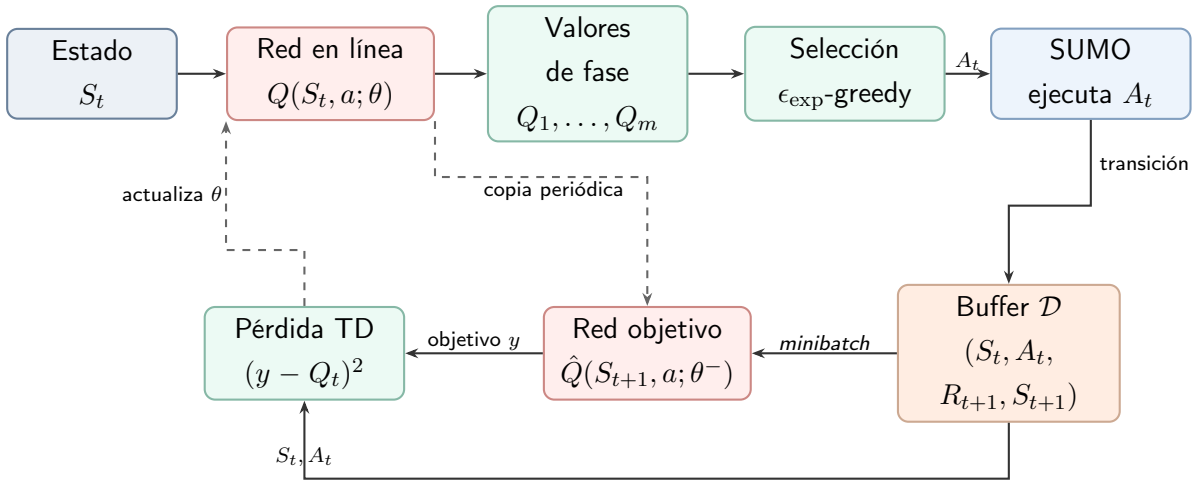


Figura 2.15: Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de *bootstrapping* del valor objetivo TD. Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS+15].

La función de pérdida minimizada por DQN, representada en el bloque inferior de la Figura 2.15, es la de la ecuación (2.15):

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1}) \sim \mathcal{D}} \left[\left(\underbrace{r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)}_{\text{Valor objetivo de TD}} - \underbrace{Q(s_j, a_j; \theta)}_{\text{Predicción actual}} \right)^2 \right] \tag{2.15}$$

En la ecuación (2.15), la esperanza se evalúa empíricamente promediando sobre un *minibatch* de transiciones $(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1})$ extraídas de $\mathcal{D}$. La función de pérdida minimiza la discrepancia cuadrática entre la estimación actual $Q(s_j, a_j; \theta)$ y el valor objetivo temporal compuesto por la recompensa inmediata observada y la mayor estimación del valor futuro sobre las fases admisibles, calculada por la red objetivo $\hat{Q}(s_{j+1}, a'; \theta^-)$. El factor $(1 - d_{j+1})$ anula la contribución del estado siguiente cuando la transición es terminal $(d_{j+1} = 1)$, mientras que a' recorre las fases semafóricas admisibles.

Los cuatro bloques numerados como algoritmos en este capítulo son pseudocódigos: descripciones ordenadas e independientes de un lenguaje de programación, no fragmentos ejecutables. En ellos, una transición registra lo ocurrido en un paso; un episodio agrupa pasos desde el reinicio hasta la terminación; y un *rollout* es la secuencia de transiciones

recolectada con una política. Una iteración de entrenamiento comprende la recolección de datos y su optimización, mientras que una *epoch* es una reutilización completa de esos datos dentro de la misma iteración. Cada actualización puede calcularse sobre un *minibatch*, es decir, un subconjunto de las muestras. Las líneas "Entrada" y "Salida" indican, respectivamente, qué información necesita el procedimiento y qué componentes entrenados produce; las líneas indentadas pertenecen al ciclo inmediatamente superior. En lo sucesivo se utiliza lote para *batch* y se conservan los términos *minibatch*, *buffer* y *epoch*, definidos aquí y en el glosario.

El procedimiento de DQN combina la recolección de experiencias, la selección de fases con política ϵ_{exp}-greedy y la optimización de la red neuronal a partir de *minibatches* muestreados del *replay buffer*. El algoritmo 1 especifica este procedimiento:

Algoritmo 1 Deep Q-Network con repetición de experiencias

Entrada: entorno, fases admisibles, capacidad N, horizonte T en pasos de decisión, γ y ϵ_{exp}
Salida: parámetros entrenados θ de la red de acción-valor

Inicializar el *buffer* de experiencias $\mathcal{D}$ con capacidad N
Inicializar la red de acción-valor $Q(s, a; \theta)$ con parámetros aleatorios
Inicializar la red objetivo $\hat{Q}(s, a; \theta^-)$ con $\theta^- \leftarrow \theta$
para episodio $= 1, 2, \ldots$ **hacer**
    Inicializar el estado s_t
    para $t = 1, 2, \ldots, T$ **hacer**
        Seleccionar a_t mediante una política ϵ_{exp}-greedy basada en $Q(s_t, a; \theta)$
        Ejecutar a_t y observar la recompensa r_{t+1}, el estado s_{t+1} y si el episodio terminó
        Almacenar $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ en $\mathcal{D}$
        si $\mathcal{D}$ contiene al menos un *minibatch* completo **entonces**
            Extraer un *minibatch* de transiciones de $\mathcal{D}$
            Calcular $y_j = r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)$
            Actualizar θ minimizando $(y_j - Q(s_j, a_j; \theta))^2$
        fin si
        Actualizar periódicamente la red objetivo: $\theta^- \leftarrow \theta$
        $s_t \leftarrow s_{t+1}$
        si $d_{t+1} = 1$ **entonces**
            Terminar el ciclo interno y comenzar un nuevo episodio
        fin si
    fin para
fin para

En esta formulación, el agente desacopla la recolección de experiencias en el simulador de la optimización paramétrica de la red. La memoria $\mathcal{D}$ reduce la correlación temporal

de las observaciones de tráfico, mientras que la red objetivo estabiliza el valor objetivo de Bellman mediante la sincronización periódica de θ^-. La especificación detallada de los hiperparámetros operacionales (capacidad N, horizonte T, tasa de aprendizaje y cronograma de exploración) se desarrolla en el capítulo 3.

Aproximación de políticas, función de ventaja y *Policy Gradient*

Como alternativa a derivar la política de forma indirecta a partir de una función de valor (como en DQN), DRL permite parametrizar directamente la política del agente mediante una red neuronal $\pi(a|s;\theta)$ [ACS24]. Bajo la formulación MDP, en un problema de acciones discretas la red recibe el estado s, calcula un vector de puntajes continuos $z \in \mathbb{R}^m$ y los proyecta a una distribución de probabilidad mediante la función *softmax* [SB18, BB24]:

$$\pi_\theta(a_i \mid s) = \frac{e^{z_i}}{\sum_{j=1}^{m} e^{z_j}} \tag{2.16}$$

la cual garantiza valores estrictamente positivos que suman la unidad. En este ejemplo, las acciones corresponden a las fases semafóricas admisibles: las variables del estado de tránsito ingresan a la red y sus salidas representan probabilidades sobre dicho conjunto de acciones. Las políticas basadas en observaciones locales se formalizan posteriormente mediante Dec-POMDP. La acción puede muestrearse de esa distribución durante el entrenamiento, preservando la exploración estocástica.

La Figura 2.16 muestra este recorrido para un agente semafórico. A diferencia de DQN, las salidas no son estimaciones de valor: son probabilidades que suman uno. La señal de ventaja no entra a la red para seleccionar la acción; se utiliza durante el entrenamiento para modificar los parámetros en la dirección indicada por el gradiente de la log-probabilidad de la acción observada.

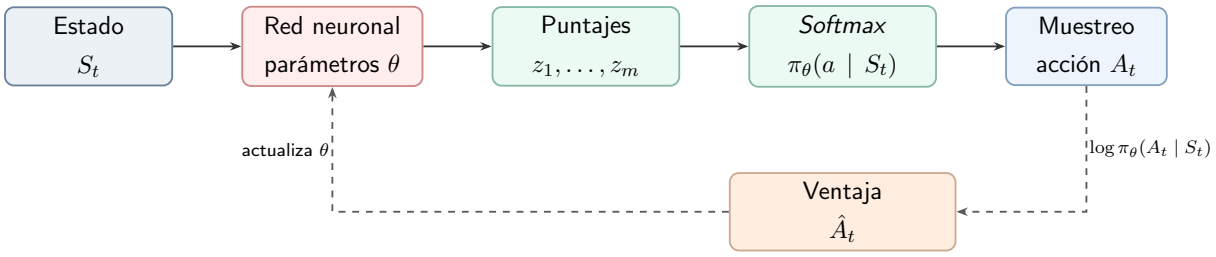


Figura 2.16: Red de política para un espacio discreto de acciones en la formulación de un MDP. El estado determina una distribución sobre las acciones disponibles; durante el aprendizaje, la ventaja pondera el gradiente de la log-probabilidad de la acción seleccionada [SB18, KT03].

La optimización de políticas se fundamenta en aislar la contribución específica de una acción respecto del rendimiento promedio del estado. Formalmente, la función de ventaja (*Advantage Function*) se define como la diferencia entre el valor de la acción y el valor

medio del estado [SB18]:

$$A^{\pi}(s,a) \doteq Q^{\pi}(s,a) - V^{\pi}(s) \tag{2.17}$$

donde $Q^{\pi}(s,a)$ es el retorno esperado al seleccionar la acción a en el estado s bajo la política π, y $V^{\pi}(s) = \mathbb{E}_{a\sim\pi}[Q^{\pi}(s,a)]$ representa el valor esperado medio en dicho estado. Un valor $A^{\pi}(s,a) > 0$ señala que la fase semafórica ejecutada supera el desempeño promedio de la intersección ante esas condiciones de cola, mientras que $A^{\pi}(s,a) < 0$ indica una decisión comparativamente deficiente.

Para estimar la ventaja combinando errores temporales de distintos horizontes, Schulman et al. [SML+16] desarrollaron el estimador GAE:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} \doteq \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^{V} \tag{2.18}$$

donde $\delta_t^V = R_{t+1} + \gamma(1-d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t)$ es el error temporal del crítico parametrizado por ϕ, d_{t+1} es el indicador terminal, $\gamma \in [0,1)$ es el factor de descuento y $\lambda \in [0,1]$ es el parámetro de suavizado exponencial. El hiperparámetro λ interpola estimaciones de ventaja de distintos horizontes. Para $\lambda = 0$, se obtiene el residuo TD de un paso δ_t^V, que suele reducir la varianza a costa de depender de la precisión del crítico y, por tanto, puede introducir sesgo. Para $\lambda = 1$, con el retorno completo, se recupera el estimador Monte Carlo descontado menos el valor del estado; este suele presentar mayor varianza. En trayectorias truncadas, el *bootstrap* y la aproximación del crítico pueden modificar este compromiso.

El Teorema del Gradiente de Política (*Policy Gradient Theorem*) [SB18] fundamenta analíticamente la optimización directa de los parámetros de la política maximizando el retorno esperado $J(\theta)$. Empleando la función de ventaja como línea de base para mitigar la varianza, el gradiente se aproxima mediante la esperanza muestral sobre un lote de transiciones recolectadas:

$$\nabla_\theta J(\theta) \approx \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \hat{A}_t \right] \tag{2.19}$$

donde $\pi_\theta(A_t \mid S_t)$ denota la probabilidad asignada a la fase A_t bajo los parámetros θ, y $\hat{A}_t$ es la ventaja estimada mediante GAE. Como esquematiza la Figura 2.16, la dirección del vector gradiente escala proporcionalmente a $\hat{A}_t$, incrementando la probabilidad de ocurrencia de aquellas fases que superan el desempeño esperado y desincentivando aquellas que provocan demoras excesivas.

Proximal Policy Optimization (PPO) y control de las actualizaciones

Una actualización demasiado grande de los parámetros puede reducir el rendimiento de la política. Los métodos que se describen a continuación buscan limitar ese cambio para conservar un aprendizaje estable.

Para acotar el tamaño de paso en el espacio de distribuciones de probabilidad, Kakade y Langford [KL02] y Schulman et al. formularon el algoritmo de optimización de política por región de confianza (*Trust Region Policy Optimization*, TRPO) [SLA+15] mediante métodos de región de confianza. En el análisis teórico de TRPO, bajo los supuestos del modelo y una actualización que respete la región de confianza basada en la divergencia de Kullback–Leibler (KL, una medida probabilística que cuantifica la discrepancia estadística entre la nueva política y la previa [GBC16, SLA+15]), la maximización exacta de la cota sustituta produce una política con retorno esperado no decreciente:

$$J(\pi_{k+1}) \geq J(\pi_k) \tag{2.20}$$

El algoritmo práctico de TRPO aproxima este procedimiento mediante ventajas estimadas, una divergencia KL promedio y optimización numérica. Sin embargo, resolver la optimización cuadrática con restricciones de TRPO en redes neuronales profundas resulta costoso en términos de cómputo. Como alternativa de primer orden, PPO [SWD+17] reemplaza la restricción formal por un objetivo sustituto recortado (CLIP) que busca limitar actualizaciones excesivas, sin transferir automáticamente la garantía teórica de TRPO.

Definiendo la razón de probabilidad entre la política actual y la previa:

$$r_t(\theta) \doteq \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{2.21}$$

donde $\pi_{\theta_{\text{old}}}$ representa la política que recolectó el lote de transiciones, PPO maximiza el objetivo recortado:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\text{mín} \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}})\hat{A}_t \right) \right] \tag{2.22}$$

En la ecuación (2.22), $\epsilon_{\text{clip}} > 0$ fija el margen de recorte. La función $\text{clip}(x, l, u)$ conserva x si está entre l y u, devuelve l si queda por debajo y u si lo supera. El operador mín elige el menor de los dos términos. Así, una ventaja positiva deja de incentivar aumentos de la razón por encima de $1+\epsilon_{\text{clip}}$, y una ventaja negativa deja de incentivar reducciones por debajo de $1-\epsilon_{\text{clip}}$. Este mecanismo limita el incentivo del objetivo; no impone una restricción estricta al cambio de la política.

En la arquitectura actor-crítico práctica, este objetivo de política se integra de forma conjunta con la regresión del crítico y un término de regularización entrópica:

$$J^{\text{PPO}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right] + c_2 \hat{\mathbb{E}}_t \left[S[\pi_\theta](s_t) \right] \tag{2.23}$$

donde ϕ son los parámetros del crítico, R_t^{target} es el retorno objetivo estimado (calculado como $\hat{A}_t + V_\phi(s_t)$), $S[\pi_\theta](s_t) \doteq -\sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t)$ es la entropía de Shannon de la política en s_t, y $c_1, c_2 > 0$ son coeficientes de ponderación [SWD+17]. La inclusión del

término entrópico favorece la exploración activa al penalizar distribuciones excesivamente concentradas, favoreciendo que se consideren distintas fases durante el aprendizaje.

PPO opera de forma predominantemente *on-policy*: recolecta un lote de trayectorias con la política vigente $\pi_{\theta_{\text{old}}}$, ejecuta un número acotado de *epochs* de optimización sobre *minibatches*, y luego descarta el lote. De esta forma, las siguientes actualizaciones utilizan experiencia obtenida con una política más reciente. El algoritmo 2 describe este ciclo de entrenamiento:

Algoritmo 2 Proximal Policy Optimization (PPO)

Entrada: entorno, horizonte de recolección, *epochs* K, γ, λ_{GAE} y ϵ_{clip}
Salida: parámetros entrenados θ del actor y ϕ del crítico

Inicializar los parámetros del actor θ y del crítico ϕ
Inicializar la política de recolección con $\theta_{\text{old}} \leftarrow \theta$
para iteración $= 1, 2, \ldots$ **hacer**
    Recolectar trayectorias con la política $\pi_{\theta_{\text{old}}}$
    Calcular los retornos objetivo y las ventajas estimadas $\hat{A}_t$
    para *epoch* $= 1$ hasta K **hacer**
        Muestrear *minibatches* del conjunto de trayectorias recolectado
        Calcular $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
        Actualizar θ maximizando $L^{\text{CLIP}}(\theta)$
        Actualizar ϕ minimizando el error cuadrático del valor
        Incorporar, si corresponde, un término de entropía para favorecer la exploración
    fin para
    $\theta_{\text{old}} \leftarrow \theta$
fin para

En PPO, θ_{old} representa la política recolectora y permanece fijo al calcular las razones de probabilidad. El lote se reutiliza durante un número acotado de *epochs*; el recorte reduce el incentivo para que las razones superen el intervalo $[1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}]$, pero no impone una restricción estricta ni garantiza una mejora monótona [SWD+17]. Después de la optimización, θ_{old} se actualiza con θ y se recolectan nuevas experiencias. Durante la ejecución en la red semafórica, el crítico se desacopla y el agente utiliza únicamente su actor local π_θ.

2.3. Aprendizaje por refuerzo multiagente (MARL)

El MARL generaliza el marco de RL hacia sistemas donde múltiples agentes autónomos interactúan y aprenden en un entorno compartido [ACS24]. En el control de redes semafóricas, la hipótesis de agente aislado deja de sostenerse: la fase habilitada en un

cruce regula el flujo vehicular emitido hacia los accesos de las intersecciones adyacentes, acoplando espacialmente las dinámicas de congestión de toda la red. Cada agente influye directamente en las tasas de arribo y longitudes de cola de sus vecinos, lo que demanda esquemas de coordinación capaces de optimizar el flujo global.

Liang et al. [LML+25] categorizan los problemas multiagente según la estructura de sus recompensas en completamente competitivos, mixtos o completamente cooperativos. Este trabajo persigue un objetivo de coordinación vial, pero sus implementaciones entregan recompensas individuales. Las búsquedas informadas emplean recompensa local; la comparación vecinal de MAPPO agrega un término de vecinos en una de sus variantes (sección 3.3.2). La finalidad cooperativa no hace idénticos los retornos individuales ni convierte automáticamente la implementación en un juego de recompensa escalar compartida.

2.3.1. Fundamentos teóricos: juegos estocásticos, equilibrio de Nash y Dec-POMDP

La extensión matemática del MDP ante múltiples actores decisores se formaliza mediante los Juegos Estocásticos o Juegos Markovianos (*Stochastic Games*), introducidos originalmente por Shapley [Sha53] y adaptados al aprendizaje por refuerzo contemporáneo por Albrecht et al. [ACS24].

Un Juego Estocástico de n agentes se define mediante la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^{n}, P, \{R_i\}_{i=1}^{n}, \gamma \rangle \tag{2.24}$$

donde $\mathcal{N} = \{1, \ldots, n\}$ es el conjunto de agentes (intersecciones), $\mathcal{S}$ representa el espacio de estados globales de la red vial, $\mathcal{A}_i$ es el conjunto discreto de fases semafóricas disponibles para el agente i, y $\mathbf{a} = (a_1, \ldots, a_n) \in \boldsymbol{\mathcal{A}} \equiv \mathcal{A}_1 \times \cdots \times \mathcal{A}_n$ denota la acción conjunta. La función de transición $P(s' \mid s, \mathbf{a})$ modela la probabilidad de evolucionar hacia el estado global s' dada la acción conjunta ejecutada en el simulador SUMO, y $R_i(s, \mathbf{a})$ es la función de recompensa asignada al agente i.

En el análisis de estabilidad de estos sistemas, el equilibrio de Nash describe una configuración donde ningún agente puede incrementar unilateralmente su retorno esperado desviándose de su política actual [ACS24]. Formalmente, una política conjunta $\boldsymbol{\pi}^* = (\pi_1^*, \ldots, \pi_n^*)$ constituye un equilibrio de Nash si:

$$J(\pi_i, \boldsymbol{\pi}_{-i}^*) \leq J(\pi_i^*, \boldsymbol{\pi}_{-i}^*), \quad \forall \pi_i \in \Pi_i, \quad \forall i \in \mathcal{N} \tag{2.25}$$

donde $J(\cdot)$ denota el retorno esperado del agente i bajo el perfil de políticas indicado, $\boldsymbol{\pi}_{-i}^*$ representa el perfil de políticas conjuntas de todos los agentes excepto el agente i, y Π_i es el espacio de políticas admisibles. Aunque en sistemas estrictamente cooperativos este equilibrio caracteriza puntos de estabilidad mutua, no garantiza por sí mismo la optimalidad global de Pareto.

En redes urbanas reales, los agentes carecen de información instantánea y global sobre todo el trazado vial. Cada intersección opera sujeta a observabilidad parcial, percibiendo únicamente el estado de sus detectores locales. Una formulación teórica para observabilidad parcial y retorno compartido es el proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*, Dec-POMDP) [AOS20], definido por la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^{n}, P, R, \{\Omega_i\}_{i=1}^{n}, O, \gamma \rangle \tag{2.26}$$

donde Ω_i es el conjunto de observaciones locales accesibles al agente i, y $O(o_1, \ldots, o_n \mid s', \mathbf{a}) = \Pr\{\mathbf{o} = (o_1, \ldots, o_n) \mid s', \mathbf{a}\}$ es la función de emisión estocástica de observaciones [AOS20]. El estado global s es una variable del modelo que no tiene por qué ser accesible a cada agente; en general, una política descentralizada puede depender del historial local de observaciones y acciones τ_i y expresarse como $\pi_i(a_i \mid \tau_i)$. En la tupla, R es una recompensa escalar compartida. La implementación experimental entrega señales individuales R_i y se describe como un juego parcialmente observable, sin identificarla automáticamente con ese caso cooperativo de recompensa compartida (sección 3.3.2). Cada agente adopta una política reactiva basada en la observación actual $\pi_i(a_i \mid o_i)$, sin identificar o_i con s (Figura 2.17).

La Figura 2.17 esquematiza este acoplamiento: cada agente procesa su observación local o_i y emite una fase a_i. La concurrencia de estas decisiones conforma la acción conjunta $\mathbf{a}$, la cual perturba el estado global del simulador de tráfico y devuelve a cada agente su nueva observación y su retroalimentación individual r_i.

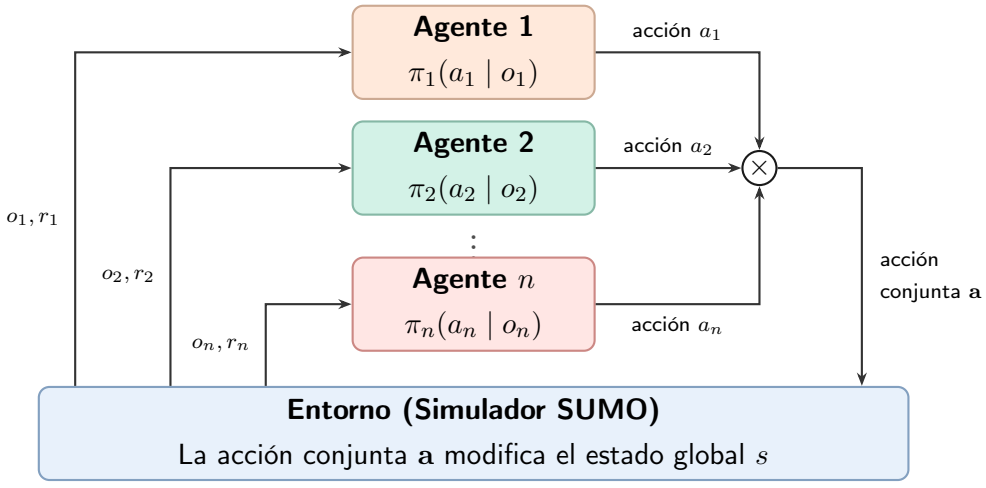


Figura 2.17: Interacción agente-entorno en un esquema MARL cooperativo. Cada semáforo emite una acción local (a_i) formando una acción conjunta ($\mathbf{a}$) que modifica el estado global del entorno de tráfico.

2.3.2. Desafíos del aprendizaje multiagente

La transición desde el aprendizaje individual hacia el dominio multiagente introduce desafíos teóricos y computacionales específicos [ACS24]:

- **Explosión combinatoria del espacio de acciones conjuntas:** si cada agente i dispone de un conjunto de acciones locales $\mathcal{A}_i$, la cardinalidad del espacio de acciones conjuntas es:

$$|\boldsymbol{\mathcal{A}}| = \prod_{i=1}^{n} |\mathcal{A}_i| \,. \tag{2.27}$$

  Cuando todos los agentes poseen A acciones, esta cardinalidad es A^n y, por tanto, crece exponencialmente con el número de agentes. Para una red de escala urbana, evaluar o enumerar exhaustivamente $\boldsymbol{\mathcal{A}}$ deja de ser una alternativa viable, lo que motiva el diseño de enfoques escalables, como políticas de decisión localmente descentralizadas.

- **No estacionariedad y el problema del valor objetivo cambiante:** desde la perspectiva local del agente i, la función de transición ambiental percibida:

$$\mathcal{P}_i(s' \mid s, a_i) = \sum_{\mathbf{a}_{-i}} P(s' \mid s, a_i, \mathbf{a}_{-i}) \boldsymbol{\pi}_{-i}(\mathbf{a}_{-i} \mid s) \tag{2.28}$$

  varía de forma continua a medida que los restantes agentes adaptan sus respectivas políticas $\boldsymbol{\pi}_{-i}$. Como formalizaron Bowling y Veloso [BV02], este cambio produce el problema del valor objetivo cambiante (*moving target*): una política óptima frente al comportamiento vigente de los vecinos puede dejar de ser adecuada en cuanto estos ajustan sus parámetros. Aunque el sistema global puede seguir describiéndose mediante un juego markoviano, la dinámica observada por un agente local se vuelve no estacionaria cuando las políticas de los demás agentes cambian y no forman parte de su información. En consecuencia, las condiciones de convergencia de los métodos de agente único basados en MDPs estacionarios dejan de ser suficientes [FNF+17, LML+25].

  En juegos competitivos o de suma cero como Piedra, Papel o Tijera, el aprendizaje independiente con tasas fijas genera trayectorias oscilatorias alrededor del equilibrio. Algoritmos especializados como ascenso de colina en la política con ganar o aprender rápido (*Win or Learn Fast Policy Hill-Climbing*, WoLF-PHC) modulan adaptativamente su tasa de aprendizaje, amortiguando las espirales hacia el equilibrio de Nash ($\pi^*(a) = 1/3$), como ilustra la Figura 2.18. En el control semafórico, la no estacionariedad se manifiesta cuando un cruce extiende su verde para permitir el paso de un grupo de vehículos y modifica las llegadas a los cruces siguientes; si los agentes aprenden sin coordinación, corren el riesgo de generar variaciones de las

colas que dificulten el servicio en otros cruces.

- **Observabilidad parcial:** bajo el formalismo Dec-POMDP, cada semáforo basa su inferencia exclusivamente en detectores locales, desconociendo el estado global de la red vial y las decisiones simultáneas de cruces lejanos, lo cual dificulta la estimación del valor de sus decisiones.

- **Asignación de crédito multiagente (*Credit Assignment*):** cuando múltiples agentes actúan en paralelo, determinar la contribución marginal de cada decisión local sobre el desempeño del sistema resulta complejo [ACS24]. Una acción local desfavorable puede quedar enmascarada si el resto de los semáforos operó de forma eficiente, o viceversa. El diseño de recompensas individuales compuestas (sección 3.3.2) incorpora señales de la intersección y sus vecinos, pero no identifica la contribución causal de cada acción.

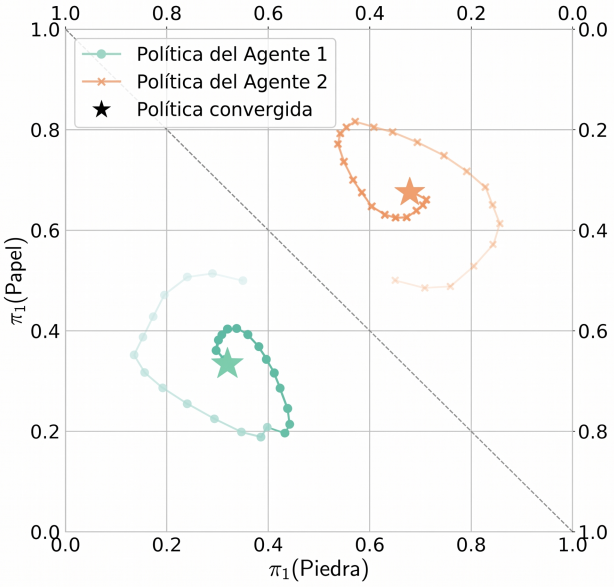


Figura 2.18: Evolución de las políticas de dos agentes con el algoritmo WoLF-PHC en el juego de Piedra, Papel o Tijera. La diagonal discontinua divide el gráfico en dos símplex de probabilidad, uno para cada agente. Cada punto en el símplex representa una distribución sobre las tres acciones disponibles; las trayectorias muestran el avance de las políticas durante el entrenamiento y los marcadores centrales destacan la política convergida en el equilibrio de Nash ($\pi^*(a) = 1/3$ para cada acción). Adaptado de [BV02, ACS24].

2.3.3. Paradigmas de entrenamiento y ejecución

Para mitigar la no estacionariedad respetando la observabilidad parcial de los agentes durante la ejecución, la literatura distingue dos paradigmas fundamentales según la información disponible durante la optimización [ZLYZ23]:

- **Aprendizaje independiente (Independent Learning, IL):** cada agente aprende de forma totalmente descentralizada tratando a los demás agentes como componentes estocásticos del entorno. Aunque computacionalmente simple y altamente escalable, es susceptible a la inestabilidad por no estacionariedad.

- **Entrenamiento centralizado con ejecución descentralizada (Centralized Training with Decentralized Execution, CTDE):** durante la etapa de entrenamiento en el simulador, se aprovecha información global privilegiada (estados conjuntos o concatenaciones de observaciones) para estimar valores y orientar el aprendizaje de las políticas locales; durante la etapa de ejecución, el componente centralizado se desacopla y cada agente opera con su política local. La Figura 2.19 ilustra esta separación entre etapas.

2.3.4. aprendizaje por refuerzo profundo multiagente (*Multi-Agent Deep Reinforcement Learning*, MADRL) y líneas de base

El uso de redes neuronales para aproximar políticas y funciones de valor de varios agentes forma parte del MADRL [LWT+17]. Para evaluar empíricamente el valor agregado del entrenamiento centralizado (MAPPO y HAPPO), esta investigación establece dos líneas de base de aprendizaje independiente (IL):

- **IDQN (Línea de base basada en valor):** extensión de DQN donde cada intersección mantiene su propia red $Q_i(o_i, a_i)$ [TMK+17]. Al operar *off-policy* con un *replay buffer*, almacena transiciones generadas bajo políticas anteriores, por lo que las experiencias pueden reflejar comportamientos distintos de los actuales [FNF+17].

- **IPPO (Línea de base basada en política):** instancia un optimizador PPO independiente por agente [dWGM+20]. Dado su régimen *on-policy*, descarta el lote tras cada iteración, mitigando la acumulación de datos obsoletos. Aunque no elimina el acoplamiento no estacionario, su mecanismo de recorte permite utilizarlo como referencia frente a los métodos coordinados.

La Figura 2.19 resume esta partición operativa entre ambas etapas.

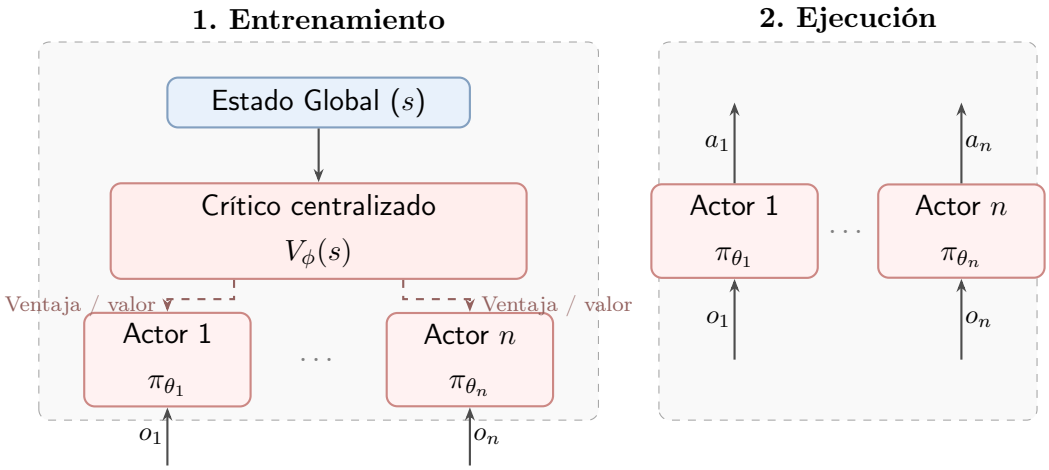


Figura 2.19: Paradigma de entrenamiento centralizado con ejecución descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales.

2.3.5. Multi-Agent Proximal Policy Optimization (MAPPO)

Para extender las ventajas de estabilidad de PPO al contexto multiagente bajo el paradigma CTDE, Yu et al. [YVV+22] estudiaron una configuración de Multi-Agent Proximal Policy Optimization (MAPPO) basada en PPO y un crítico centralizado. En lugar de rediseñar el estimador base, MAPPO preserva la simplicidad computacional de PPO mediante la factorización explícita entre políticas descentralizadas y evaluación centralizada:

- **Actores descentralizados ($\pi_{\theta_i}(a_i \mid o_i)$):** cada agente i procesa exclusivamente su vector de observación local o_i para seleccionar su fase semafórica a_i. En intersecciones con geometrías idénticas puede compartirse parámetros entre actores, aunque esto no constituye una restricción conceptual.
- **Crítico centralizado ($V_\phi(s)$):** durante el entrenamiento, el crítico puede utilizar información global que no está disponible para los actores. Una representación global adecuada puede facilitar la estimación del valor. Una estimación más precisa también puede reducir la variabilidad de las actualizaciones, aunque esto depende de la configuración [YVV+22].

El objetivo de optimización de los actores promedia los términos individuales de recorte PPO sobre los n agentes:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^{n} \min \left(r_t^i(\theta) \hat{A}_t(s_t, \mathbf{a}_t),\ \text{clip}(r_t^i(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}}) \hat{A}_t(s_t, \mathbf{a}_t) \right) \right] \tag{2.29}$$

donde $r_t^i(\theta) = \frac{\pi_{\theta_i}(a_t^i|o_t^i)}{\pi_{\theta_{i,\text{old}}}(a_t^i|o_t^i)}$ denota la razón de probabilidad local de cada agente y ϵ_{clip} es el margen de recorte. Paralelamente, el crítico centralizado se actualiza minimizando

el error cuadrático frente al retorno objetivo R_t^{target}, estimado empíricamente sobre el lote de transiciones mediante la suma de la ventaja GAE y el valor de estado previo $\hat{A}_t(s_t, \mathbf{a}_t) + V_\phi(s_t)$ [YVV+22, SML+16]:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]. \tag{2.30}$$

Factores críticos de diseño y optimización en MAPPO

Yu et al. [YVV+22] analizaron empíricamente diversos factores de implementación e hiperparámetros que afectan el rendimiento de MAPPO:

1. **Normalización de retornos:** en los experimentos de Yu et al., normalizar los objetivos de valor puede favorecer el ajuste del crítico cuando cambia la escala de las recompensas.

2. **Estructuración del estado global del crítico:** en ese estudio, la entrada s se evaluó con distintas representaciones: concatenación directa de observaciones (CL : $s = (o_1, \ldots, o_n)$), estado global extraído directamente del simulador (EP), o estados aumentados específicos por agente ($AS_i = EP \cup o_i$), combinados opcionalmente con filtrado de redundancias (FP).

3. **Reutilización de datos:** la cantidad de *epochs* (K) y la división en *minibatches* regulan la reutilización del lote. Yu et al. observaron que una reutilización excesiva puede deteriorar el rendimiento, especialmente en tareas difíciles.

El algoritmo 3 especifica el ciclo computacional de MAPPO bajo el paradigma CTDE.

Algoritmo 3 Algoritmo Multi-Agent Proximal Policy Optimization (MAPPO)

Entrada: entorno multiagente, n agentes, horizonte de recolección, *epochs* K, γ, λ_{GAE} y ϵ_{clip}

Salida: parámetros entrenados de los actores θ y del crítico centralizado ϕ

Inicializar los parámetros de la política (actor) θ y del valor (crítico) ϕ
Inicializar el *buffer* de datos $\mathcal{D} \leftarrow \emptyset$
para iteración $= 1, 2, \ldots$ **hacer**
    Recolectar trayectorias *on-policy* para todos los agentes utilizando la política conjunta $\boldsymbol{\pi}_\theta$
    Calcular las ventajas conjuntas $\hat{A}_t(s_t, \mathbf{a}_t)$ mediante GAE a partir del crítico centralizado $V_\phi(s_t)$
    Calcular los retornos objetivos R_t^{target} normalizados para el crítico
    Almacenar las transiciones $(s_t, \mathbf{o}_t, \mathbf{a}_t, \hat{A}_t, R_t^{\text{target}})$ en $\mathcal{D}$
    para *epoch* $= 1$ hasta K **hacer**
        Muestrear *minibatches* de datos desde $\mathcal{D}$
        Actualizar la política maximizando el objetivo de recorte PPO por agente: $L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^{n} \text{mín} \left(r_t^i(\theta)\hat{A}_t,\ \text{clip}(r_t^i(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}})\hat{A}_t \right) \right]$
        Actualizar el crítico ϕ minimizando el error cuadrático medio del valor: $\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$
    fin para
    Vaciar el *buffer* de datos $\mathcal{D} \leftarrow \emptyset$
fin para

Durante el entrenamiento, el crítico centralizado $V_\phi(s)$ procesa el estado global de la red vial para calcular ventajas conjuntas mediante GAE, guiando la optimización paralela de los actores locales π_{θ_i} con el objetivo recortado. Al finalizar cada iteración, el *buffer* $\mathcal{D}$ se descarta por completo para preservar el régimen *on-policy* del método. En la etapa de ejecución, el crítico se desacopla y cada agente actúa con autonomía computacional a partir de su vector local o_t^i. Los hiperparámetros de entrenamiento y la arquitectura de las redes se detallan en el capítulo 3.

La formulación anterior presenta la variante teórica con una ventaja conjunta. En la implementación de este trabajo, el crítico centralizado produce una salida por agente y los objetivos de GAE se calculan por agente a partir de la recompensa individual compuesta descrita en la sección 3.3.2; por tanto, la ecuación resume el paradigma CTDE y no debe leerse como una identidad exacta del código ejecutado.

2.3.6. Heterogeneous-Agent Proximal Policy Optimization (HAPPO)

En sistemas de control complejos, los agentes pueden presentar asimetrías físicas y operacionales: diferencias en la configuración de actuadores, distintos espacios de acciones o demandas regionales contrapuestas. En entornos homogéneos, compartir parámetros puede mejorar la eficiencia de aprendizaje; en entornos heterogéneos, esa práctica puede limitar la especialización. Además, las actualizaciones independientes no poseen una garantía general de convergencia debido al aprendizaje simultáneo no coordinado [KCW+22].

Kuba et al. y Zhong et al. propusieron algoritmos de aprendizaje por refuerzo multiagente para políticas heterogéneas, sin exigir que todos los agentes compartan parámetros. Esta línea se denomina aprendizaje por refuerzo con agentes heterogéneos (Heterogeneous-Agent Reinforcement Learning, HARL). Cada agente i puede mantener su propia arquitectura y parámetros θ^i, adaptados a su espacio de acciones local $\mathcal{A}_i$. En esta línea, optimización de política por región de confianza con agentes heterogéneos (*Heterogeneous-Agent Trust Region Policy Optimization*, HATRPO) y HAPPO emplean una actualización secuencial de los agentes. HAPPO sustituye la restricción KL de la formulación de región de confianza por un objetivo recortado. Zhong et al. lo encuadran en el marco de aprendizaje espejo con agentes heterogéneos (*Heterogeneous-Agent Mirror Learning*, HAML), que establece resultados de mejora monótona y convergencia bajo sus condiciones, entre ellas una recompensa compartida [KCW+22, ZKF+24].

Compartición de parámetros y políticas heterogéneas

Una distinción central entre MAPPO y HAPPO radica en la homogeneidad de las políticas y en el orden de actualización:

- **Compartición de parámetros:** en MAPPO se utiliza habitualmente cuando los agentes poseen espacios de observación y acción compatibles. Todos los agentes pueden compartir los mismos pesos $\theta_1 = \cdots = \theta_n = \theta$, aunque la formulación de MAPPO también puede implementarse sin esta restricción.

- **Políticas heterogéneas y actualización secuencial:** HAPPO permite que cada agente mantenga una política independiente $\pi_{\theta^i}(a_i \mid o_i)$ y actualiza los actores siguiendo una permutación. El lote de trayectorias de la iteración actual se reutiliza de forma limitada; esto no equivale a un *buffer* histórico de *experience sharing* como el empleado por algoritmos específicos de compartición de experiencia.

Lema de descomposición de la ventaja multiagente

La actualización simultánea de todos los agentes puede generar inestabilidad porque el cambio concurrente de las políticas altera tanto las distribuciones de acción como la distribución conjunta de las trayectorias. HAPPO aborda este problema mediante una

actualización secuencial: en cada iteración se optimiza un agente por vez, condicionando cada paso a los cambios ya efectuados por los precedentes. Para sustentar este esquema, la ventaja conjunta de la red debe poder descomponerse en contribuciones marginales ordenadas [KCW+22].

Esta descomposición se formaliza en el lema de descomposición de la ventaja multiagente propuesto por Kuba et al. [KCW+22]. En cualquier juego markoviano cooperativo, para una política conjunta $\boldsymbol{\pi}$ y un orden arbitrario de los agentes $i_{1:n} = (i_1, \ldots, i_n)$, la ventaja conjunta de la red se descompone exactamente como la suma de las ventajas marginales de cada agente, condicionadas por las decisiones de los que lo preceden en la secuencia:

$$A^{\boldsymbol{\pi}}(s, \mathbf{a}) = \sum_{m=1}^{n} A^{\boldsymbol{\pi}}_{i_m}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \tag{2.31}$$

donde la ventaja marginal del agente i_m se define como:

$$A^{\boldsymbol{\pi}}_{i_m}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \doteq Q^{\boldsymbol{\pi}}_{i_{1:m}}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) - Q^{\boldsymbol{\pi}}_{i_{1:m-1}}(s, \mathbf{a}_{i_{1:m-1}}). \tag{2.32}$$

En la ecuación (2.32), $Q^{\boldsymbol{\pi}}_{i_{1:m}}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m})$ evalúa el retorno esperado fijando las acciones de los primeros m agentes de la secuencia, mientras que las decisiones de los agentes posteriores ($\mathbf{a}_{i_{m+1:n}}$) se promedian según sus probabilidades bajo la política vigente.

La deducción matemática original de este lema asume un juego markoviano estrictamente cooperativo con un retorno escalar compartido $R(s, \mathbf{a})$ para toda la red. En sistemas espaciales distribuidos de gran escala, como el control semafórico urbano, asignar una recompensa global idéntica a todas las intersecciones dificulta la asignación de crédito, extremo que Chu et al. respaldan con evidencia empírica en redes extensas [CWCL19], y encarece la estimación de las ventajas [ACS24]. En este trabajo se emplean señales locales y, en protocolos específicos, de vecindario inmediato. Esta decisión de implementación no conserva por sí sola los supuestos del lema original.

Bajo retornos desacoplados, la identidad aditiva de la ecuación (2.31) no se traduce en una equivalencia algebraica directa entre un único valor global y los retornos individuales; no obstante, el principio de optimización secuencial por bloques (tipo Gauss-Seidel) conserva su utilidad práctica para desacoplar el aprendizaje concurrente [ACS24, WZK+23].

La relevancia teórica de este lema radica en que es una identidad analítica exacta para cualquier juego cooperativo sin imponer restricciones de aditividad o de monotonía sobre la función de valor en la red de mezclado (a diferencia de las condiciones de factorización IGM de arquitecturas como redes de descomposición de valor (*Value-Decomposition Networks*, VDN) [SLG+18] o red de mezcla de valores Q (*Q-Mixing Network*, QMIX) [RSDW+18]).

Esquema de actualización secuencial y factor de razón compuesto M

Para explotar analíticamente esta descomposición teniendo en cuenta los cambios de los agentes ya actualizados, HAPPO actualiza las políticas de los agentes de manera

secuencial dentro de cada iteración k. En cada ciclo se sortea una permutación de los agentes $i_{1:n} \in \mathrm{Sym}(n)$ (donde $\mathrm{Sym}(n)$ representa el conjunto de todas las permutaciones posibles de los n agentes) para que ningún agente ocupe siempre la misma posición en la secuencia [KCW+22, WZK+23].

Cuando corresponde el turno del agente i_m, los agentes previos en la secuencia ($i_{1:m-1}$) ya han actualizado sus parámetros a $\theta_{k+1}^{i_j}$, mientras que los agentes posteriores ($i_{m+1:n}$) conservan aún sus parámetros previos $\theta_k^{i_j}$. Para guiar al agente i_m teniendo en cuenta la alteración de la distribución conjunta producida por los agentes ya actualizados, Kuba et al. [KCW+22] demostraron que no se requiere entrenar un crítico separado por cada subgrupo; la ventaja marginal puede computarse reponderando la ventaja conjunta $A^{\boldsymbol{\pi}_k}(s, \mathbf{a})$ mediante la productoria acumulada de las razones de probabilidad de los agentes ya actualizados:

$$M_{i_{1:m}}(s_t, \mathbf{a}_t) \doteq \left(\prod_{j=1}^{m-1} \frac{\pi_{\theta_{k+1}^{i_j}}^{i_j}(a_t^{i_j} \mid o_t^{i_j})}{\pi_{\theta_k^{i_j}}^{i_j}(a_t^{i_j} \mid o_t^{i_j})} \right) A^{\boldsymbol{\pi}_k}(s_t, \mathbf{a}_t) \tag{2.33}$$

donde $\pi_{\theta_{k+1}^{i_j}}^{i_j}$ es la política recién optimizada del agente i_j y $\pi_{\theta_k^{i_j}}^{i_j}$ es su versión previa. Para el primer agente ($m = 1$), la productoria es vacía y $M_{i_1}(s_t, \mathbf{a}_t) \equiv A^{\boldsymbol{\pi}_k}(s_t, \mathbf{a}_t)$. En la formulación algorítmica práctica, el término acumulado $M_{i_{1:m}}$ se mantiene en una variable de memoria M_{i_m} por cada agente, la cual se actualiza recursivamente conforme avanzan las optimizaciones precedentes. El factor $M_{i_{1:m}}$ actúa como un estimador de importancia que ajusta la ventaja conjunta al nuevo perfil de comportamiento sin requerir simulaciones adicionales en el entorno [KCW+22].

Así, la política del agente i_m se optimiza maximizando el objetivo sustituto recortado ponderado por $M_{i_{1:m}}$:

$$\begin{aligned} L^{\mathrm{HAPPO}}(\theta^{i_m}) = \hat{\mathbb{E}}_t \Big[\min \Big(& r_t^{i_m}(\theta^{i_m}) M_{i_{1:m}}(s_t, \mathbf{a}_t), \\ & \mathrm{clip}\big(r_t^{i_m}(\theta^{i_m}), 1 - \epsilon_{\mathrm{clip}}, 1 + \epsilon_{\mathrm{clip}}\big) M_{i_{1:m}}(s_t, \mathbf{a}_t) \Big) \Big] \end{aligned} \tag{2.34}$$

donde $r_t^{i_m}(\theta^{i_m}) = \frac{\pi_{\theta^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ representa la razón de probabilidad individual del agente.

Concluida la optimización de θ^{i_m}, se computa la razón post-optimización:

$$\rho_{\mathrm{post}}^{i_m} \doteq \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m} \mid o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} \mid o_t^{i_m})} \tag{2.35}$$

y se propaga recursivamente hacia los agentes pendientes mediante:

$$M_{i_{1:m+1}}(s_t, \mathbf{a}_t) \leftarrow \rho_{\mathrm{post}}^{i_m} \cdot M_{i_{1:m}}(s_t, \mathbf{a}_t). \tag{2.36}$$

Garantía de mejora monótona

El principio de mejora monótona en aprendizaje por refuerzo formaliza que la secuencia de retornos esperados de la política conjunta no disminuye entre ciclos sucesivos de entrenamiento, comportándose como una función monótonamente no decreciente respecto de la iteración k:

$$J(\boldsymbol{\pi}_{k+1}) \geq J(\boldsymbol{\pi}_k). \tag{2.37}$$

Para políticas heterogéneas, Kuba et al. [KCW+22] demostraron analíticamente esta propiedad para el algoritmo HATRPO bajo los supuestos del juego markoviano cooperativo con recompensa compartida y una actualización restringida por la divergencia KL. Resolver las restricciones de HATRPO en redes neuronales profundas requiere un costo de cómputo elevado. Por ello, HAPPO sustituye la cota formal de divergencia KL por el objetivo sustituto recortado CLIP de primer orden de la ecuación (2.34).

La garantía de HATRPO no se transfiere automáticamente al sustituir su restricción por un recorte. El análisis de HAPPO dentro de HAML establece garantías bajo las condiciones de ese marco [KCW+22, ZKF+24]. Su aplicación a un controlador semafórico requiere comprobar esas condiciones: las recompensas específicas de cada agente no definen por sí solas el retorno compartido del planteo cooperativo. La metodología delimita por ello el alcance de las garantías para la adaptación utilizada y evalúa empíricamente su desempeño. La Figura 2.20 resume este esquema de propagación secuencial.

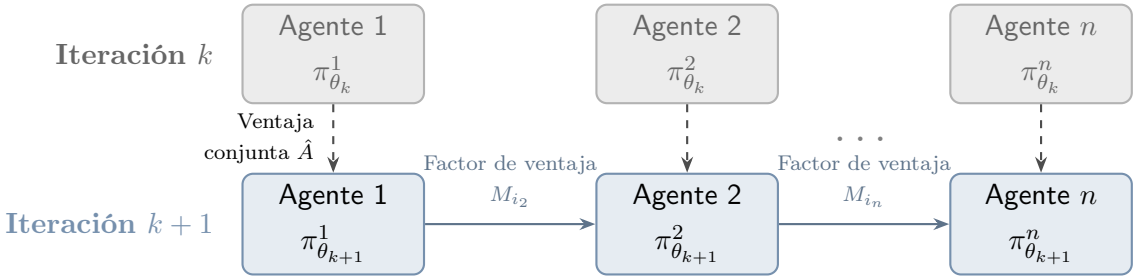


Figura 2.20: Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m}, que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia.

El algoritmo 4 presenta la secuencia completa de optimización de HAPPO.

Algoritmo 4 Algoritmo Heterogeneous-Agent Proximal Policy Optimization (HAPPO)

Entrada: entorno multiagente, conjunto $\mathcal{N}$ de n agentes, horizonte de recolección, γ, λ_{GAE} y ϵ_{clip}

Salida: políticas heterogéneas entrenadas $\{\theta^i\}_{i=1}^n$ y crítico centralizado ϕ

Inicializar los parámetros de las políticas individuales $\{\theta^i, \forall i \in \mathcal{N}\}$ y del crítico centralizado $V_\phi(s)$
para iteración $= 1, 2, \ldots$ **hacer**
    Recolectar un conjunto de trayectorias $\mathcal{D}$ utilizando la política conjunta $\boldsymbol{\pi}_\theta$
    Calcular el estimador de ventaja conjunta $\hat{A}(s, \mathbf{a})$ con GAE a partir del crítico $V_\phi(s)$
    Calcular los retornos objetivo R_t^{target} utilizados para ajustar el crítico
    Sortear una permutación de los agentes $i_{1:n} \in \text{Sym}(n)$
    Inicializar los factores $M_{i_m}(s, \mathbf{a}) \leftarrow \hat{A}(s, \mathbf{a})$ para todo $m \in \{1, \ldots, n\}$
    para $m = 1 \ldots n$ **hacer**
        Obtener el agente actual i_m y su factor modificador $M_{i_m}(s, \mathbf{a})$
        Actualizar θ^{i_m} a $\theta_{k+1}^{i_m}$ maximizando el objetivo recortado de la ecuación (2.34)
        Recalcular la razón post-optimización: $\rho_{\text{post}}^{i_m} \leftarrow \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$
        para $j = m + 1 \ldots n$ **hacer**
            Actualizar el factor modificador para agentes posteriores: $M_{i_j}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_j}(s_t, \mathbf{a}_t)$
        fin para
    fin para
    Actualizar el crítico centralizado ϕ mediante descenso de gradiente: $\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$
fin para

El procedimiento de HAPPO organiza secuencialmente la actualización de los actores locales mediante la permutación aleatoria $i_{1:n}$ y la propagación recursiva de los factores modificadores M_{i_m}. Tras optimizar sucesivamente a los n agentes sobre el lote de transiciones $\mathcal{D}$, el crítico centralizado V_ϕ ajusta sus parámetros mediante descenso de gradiente sobre los retornos acumulados. Durante la etapa de ejecución descentralizada, cada política local selecciona a_i a partir de o_i, sin requerir evaluar el crítico centralizado ni comunicación entre agentes para seleccionar su acción [KCW+22].

En la adaptación para control semafórico de este trabajo, la secuencia conserva esta estructura de permutación y actualización sucesiva, pero inicializa los factores M_{i_m} con las ventajas individuales $\hat{A}_i(s, \mathbf{a})$ calculadas mediante el crítico centralizado multi-salida a partir de las señales de recompensa compuestas (sección 3.3.2), propagándolos mediante la razón $\rho_{\text{post}}^{i_m}$ tras optimizar a cada actor. Esta adaptación utiliza ventajas individuales,

mientras que la demostración teórica parte de una recompensa global compartida. Su efecto sobre el aprendizaje debe comprobarse mediante los experimentos; esta adaptación no hereda por sí sola la garantía teórica de mejora.

Para finalizar el estudio comparativo de los algoritmos derivados de PPO en escenarios MARL, la Tabla 2.3 sintetiza sus principales propiedades operativas, arquitectónicas y teóricas.

Tabla 2.3: Comparación sintética de familias algorítmicas PPO en entornos multiagente cooperativos.

Eje de análisis	**PPO de agente único**	**IPPO independiente**	**MAPPO / CTDE**	**HAPPO / HARL**
Paradigma	agente único	IL (descentralizado)	CTDE (centralizado/desc.)	CTDE; familia HARL
Actores	único $\pi_\theta(a \mid s)$	n políticas locales	actores locales; puede compartir parámetros	n políticas heterogéneas
Arquitectura del crítico	único $V_\phi(s)$	n críticos locales $V_{\phi_i}(o_i)$	un crítico centralizado $V_\phi(s)$	un crítico centralizado $V_\phi(s)$
Optimización	PPO estándar	PPO local simultáneo	PPO local con crítico global	actualización secuencial $i_{1:n}$
Señal de ventaja	ventaja local $\hat{A}(s, a)$	ventaja local $\hat{A}_i(o_i, a_i)$	ventaja conjunta $\hat{A}(s, \mathbf{a})$	factor compuesto $M_{i_{1:m}}(s, \mathbf{a})$
Mejora monótona	No estricta para PPO	Sin garantía general	Sin garantía estricta	Bajo las condiciones de HAML
Tipo de agente	No aplica	Depende de la implementación	Pensado para agentes homogéneos; la compartición de parámetros es opcional	Pensado para agentes heterogéneos

2.3.7. Antecedentes recientes del control semafórico multiagente

Para orientar la comparación, los antecedentes seleccionados se organizan según tres ejes: coordinación y entrenamiento, representación de las observaciones, y tratamiento de la heterogeneidad o la equidad entre intersecciones. Se priorizan investigaciones directamente relacionadas con la formulación de este trabajo.

El control coordinado de múltiples intersecciones mediante aprendizaje por refuerzo constituye un área de investigación activa. Entre los enfoques iniciales se encuentran aquellos que asignaban un agente a cada semáforo y utilizaban información local sobre colas, fases o vehículos detenidos. Sin embargo, la optimización aislada de cada intersección no contempla que una acción modifica el flujo recibido por los cruces vecinos. Por este motivo, la literatura evolucionó hacia formulaciones cooperativas que incorporan mecanismos de coordinación espacial, representaciones estructuradas del estado y esquemas de entrenamiento centralizado [ACS24, LML+25].

Coordinación y entrenamiento

En la consolidación del control semafórico coordinado mediante aprendizaje profundo, varios trabajos abordaron de manera temprana la asignación de crédito y la no estacionariedad en redes viales:

- **Control actor-crítico con descuento espacial (MA2C):** Chu et al. [CWCL19] propusieron una arquitectura de ventaja actor-crítico multiagente donde cada intersección incorpora a su función de valor y recompensa la información de sus vecinos inmediatos atenuada por un factor de descuento espacial. Este trabajo fundamentó conceptualmente y evidenció de manera empírica que una recompensa de equipo global en redes extensas induce severos problemas de asignación de crédito, mientras que las recompensas regionalmente estructuradas permiten coordinar corredores sin requerir estados globales inmanejables.

- **Coordinación basada en presión vehicular (PressLight):** Wei et al. [WCZ+19] fundamentaron su función de recompensa en el principio de *Max-Pressure* de la ingeniería de transporte, que en la teoría de control de redes garantiza estabilidad de cola bajo supuestos específicos de un esquema de servicio discreto. PressLight incorpora ese principio mediante una recompensa igual al negativo de la presión de la intersección, calculada a partir de las densidades vehiculares de entrada y salida. Esa incorporación no hereda la garantía formal de estabilidad y coordina de manera implícita las intersecciones adyacentes sin necesidad de comunicación explícita.

- **Atención en grafos para cooperación a nivel de red (CoLight):** Wei et al. [WXZ+19] introdujeron redes de atención en grafos para permitir que los agentes semafóricos intercambien dinámicamente representaciones de sus estados con cruces vecinos, adaptando la influencia de cada intersección adyacente según las condiciones variables de congestión.

- **Límites del aprendizaje independiente en redes viales:** Ault y Sharon [AS22] analizaron las limitaciones de los enfoques independientes frente a formulaciones cooperativas en redes de múltiples intersecciones, evidenciando un deterioro pro-

gresivo en el desempeño y en la fluidez del tránsito a medida que la demanda se incrementa debido a la falta de coordinación y a la no estacionariedad del entorno.

A partir de estas bases fundacionales, diversas investigaciones recientes han explorado mecanismos alternativos de optimización y distribución de cómputo. Wu et al. [WWS+22] estudian el control de redes de gran escala mediante agentes distribuidos de aprendizaje por refuerzo profundo y una arquitectura de cómputo distribuido. Proponen dos variantes, Nash-A2C y Nash-A3C, que incorporan un equilibrio de Nash en el aprendizaje por refuerzo y se despliegan en una arquitectura distribuida para simulación y control. En sus experimentos reportan reducciones del tiempo de congestión y de la demora total de red frente a controladores de referencia. Este antecedente es pertinente como referencia de escalabilidad, pero no es equivalente a la formulación de este trabajo: aquí la ejecución también es local por intersección, mientras que la coordinación se estudia mediante la señal individual compuesta local-vecinal y el entrenamiento centralizado, sin introducir una capa de cómputo fog ni un equilibrio de Nash.

En una formulación cooperativa basada en DQN, Haddad et al. [HHA22] incorporan a la función de aprendizaje de cada intersección información sobre los estados, acciones y recompensas de sus vecinos. El estudio informa mejoras en tiempo de espera, longitud de cola y emisiones de dióxido de carbono frente a controladores de referencia. Esta evidencia respalda la necesidad de considerar el efecto de las decisiones vecinas, pero su mecanismo de coordinación es una transferencia explícita de información hacia la pérdida de DQN. En el protocolo de búsqueda con dominios reducidos de este estudio, IPPO e IDQN implementan el paradigma de aprendizaje independiente y reciben una señal de recompensa estrictamente local ($R_i = L_i$, $\rho_{\text{coop}} = 0$); MAPPO y HAPPO usan también recompensa local en la búsqueda informada, con crítico centralizado durante el entrenamiento. La mezcla vecinal se evalúa en una línea separada de MAPPO y no se atribuye a todos los algoritmos. Estas alternativas no son equivalentes al mecanismo propuesto por Haddad et al., sino procedimientos de coordinación distintos que se comparan bajo una misma representación y escenario.

Fang et al. [FYX+23] proponen un método de coordinación de red que formula el control semafórico como un problema multiobjetivo: combina eficiencia, seguridad y coordinación espacial entre intersecciones mediante un esquema de entrenamiento centralizado con ejecución descentralizada. Además de reportar resultados en redes sintéticas y arterias reales, el trabajo destaca que las dependencias espacio-temporales entre agentes deben representarse explícitamente. Su propuesta ofrece un contraste con la señal individual compuesta y la representación interpretable utilizadas en este trabajo, que no incorporan una optimización multiobjetivo ni una representación latente basada en atención.

Representación del estado

En cuanto a la representación agregada y el aprendizaje basado en valor, Kolat et al. [KKBA23] proponen un enfoque DQN multiagente con parámetros compartidos. Los agentes observan cantidades escaladas de vehículos detenidos y reciben una recompensa que penaliza el desequilibrio entre los accesos. Los resultados muestran mejoras en el tiempo de viaje y el consumo de combustible respecto de un controlador de ciclo fijo. Sirve como referencia para contrastar una representación agregada del estado y una coordinación inducida por la recompensa con la evaluación de IDQN y de métodos basados en políticas realizada en esta investigación.

Dentro de la línea centrada en la representación del estado, Yang [Yan23] propone HG-M2I, una arquitectura que combina información espacial y temporal mediante un grafo jerárquico. El modelo integra estados de los agentes y de sus vecinos en representaciones latentes utilizadas por las políticas. Posteriormente, Chen et al. [CYL+24] presentan CoevoMARL, que utiliza atención en grafos y una unidad recurrente para aprender cómo varía en el tiempo la dependencia entre intersecciones. Mientras HG-M2I organiza la información según una jerarquía espacial, CoevoMARL adapta las relaciones entre agentes a la evolución del tránsito. Ambos trabajos muestran la importancia de construir observaciones informativas, aunque recurren a representaciones latentes aprendidas, a diferencia de la transformación explícita e interpretable que se evalúa aquí.

En el caso de una representación basada en celdas, Deng et al. [DLSZ26] introducen una partición de longitud variable. Los accesos se dividen mediante una función que combina componentes logarítmicos y lineales, y cada celda almacena la cantidad de vehículos, la velocidad media y la ocupación espacial. Esta partición asigna mayor detalle a las zonas próximas a la intersección y comprime las regiones más alejadas. Entre los trabajos seleccionados, es el antecedente más cercano al uso de una transformación logarítmica en la representación semafórica; sin embargo, el logaritmo determina la longitud espacial de las celdas y no codifica el tiempo de espera acumulado por carril.

Heterogeneidad y equidad

Ren et al. [RDZ+24] abordan la coordinación bajo demandas heterogéneas mediante un esquema de dos capas. Cada agente combina su ventaja local con la de las intersecciones vecinas a través de un factor de cooperación adaptable, que luego se ajusta según el rendimiento global de la red. El método considera la demora y las emisiones vehiculares, y coordina los objetivos locales y globales mediante ese factor adaptable. Este antecedente permite contrastar la coordinación adaptativa de las señales de aprendizaje con la heterogeneidad de políticas considerada por HAPPO, aunque no emplea su procedimiento de actualización secuencial.

En la coordinación mediante entrenamiento centralizado, Liu et al. [LZF+25] formulan el control semafórico como un juego parcialmente observable con restricciones y proponen

VF-MAPPO. En este método, la equidad se expresa mediante una cota sobre el tiempo máximo de espera de los vehículos, mientras que un crítico centralizado con atención espaciotemporal estima tanto la eficiencia de la red como el cumplimiento de la restricción. Esta formulación resulta especialmente pertinente para esta investigación porque vincula con una variable temporal de operación el paradigma de entrenamiento centralizado con ejecución descentralizada. No obstante, el tiempo de espera se incorpora como restricción de optimización y no mediante una codificación no lineal de la observación.

En el tratamiento de agentes heterogéneos, otro enfoque es HAPS-PPO, presentado por Lu et al. [LFYZ25]. El método completa con relleno las observaciones de distinta dimensión y agrupa a los agentes según sus espacios de acción, asignando cabezales de política específicos a cada grupo. Esto permite coordinar intersecciones con geometrías y conjuntos de fases diferentes.

Posicionamiento del proyecto

La selección analizada sugiere que el desempeño del control semafórico multiagente depende de tres decisiones relacionadas: el algoritmo utilizado para aprender y coordinar las políticas, la representación de las condiciones de operación y el tratamiento de intersecciones con demandas, geometrías o espacios de acción diferentes. Los enfoques basados en DQN proporcionan una referencia para el aprendizaje por valor. Los modelos de grafos construyen representaciones latentes de las relaciones espaciales, mientras que las variantes de MAPPO incorporan información global durante el entrenamiento. Los esquemas de cooperación adaptable o con múltiples cabezales atienden distintas formas de heterogeneidad. La codificación por carril y la discretización de variables de tránsito también cuentan con antecedentes en control semafórico [GR18]; aquí se emplean para representar la espera acumulada por carril mediante la transformación especificada en la metodología.

Entre los antecedentes considerados, HeteroLight aborda la heterogeneidad mediante una arquitectura de parámetros compartidos con módulos que extraen características generales y específicas de cada intersección [ZLFS24], mientras que MATLight se apoya en un marco de aprendizaje especular para agentes heterogéneos (*heterogeneous-agent mirror learning*) y emplea un Transformer para procesar secuencias de estados [ZWL24]. Son mecanismos distintos de la adaptación HAPPO implementada aquí: no se los presenta como equivalentes ni como evidencia de que HAPPO sea superior. Entre las fuentes consultadas no se localizó una evaluación conjunta de IDQN, IPPO, MAPPO y HAPPO bajo una misma formulación, ni una aplicación directa del HAPPO original al control de fases semafóricas urbanas. Esta observación delimita la búsqueda realizada y no permite concluir que tales estudios no existan.

En particular, este trabajo explora una codificación híbrida logarítmica-lineal del tiempo de espera acumulado por carril. Al asignar valores continuos a un conjunto finito de

intervalos, la cuantización agrupa esperas dentro de una misma categoría y limita los valores representables; la saturación también reúne los valores que alcanzan la cota. En aprendizaje por refuerzo, una abstracción de estados puede simplificar el problema y conservar comportamiento cercano al óptimo cuando los estados agrupados son suficientemente similares, bajo las condiciones teóricas correspondientes [AHL16]. Ese resultado no constituye una garantía para esta codificación: aquí se transforman observaciones locales en un problema parcialmente observable y no se demuestra que los valores agrupados sean equivalentes para la política óptima. La resolución no uniforme del codificador asigna más niveles a esperas bajas, lo que permite distinguirlas con mayor detalle; es una hipótesis de diseño, pues la cuantización también puede descartar distinciones relevantes, en particular cerca de los umbrales entre intervalos.

Su efecto se examina como parte del sistema completo, sin un experimento que compare el mismo sistema con y sin esa transformación. Esta propuesta se diferencia de las observaciones agregadas utilizadas por los enfoques DQN, de las proyecciones latentes basadas en grafos y de la partición espacial de Deng et al. [DLSZ26], que divide los accesos y no codifica la espera acumulada por carril.

A su vez, el diseño experimental contempla la búsqueda de configuraciones de IDQN, IPPO, MAPPO y HAPPO sobre una misma representación local de observación. En MAPPO se utiliza un crítico centralizado compartido de múltiples salidas, condicionado por la concatenación de observaciones locales, con valores y ventajas calculados por agente. En HAPPO, las ventajas individuales estimadas mediante GAE se reponderan en las actualizaciones secuenciales de los actores; esta adaptación no aplica directamente el lema original de descomposición de ventajas. Los críticos estiman valores durante el entrenamiento en ambos procedimientos, pero no se atribuye al crítico la diferencia entre ellos ni se postula una actualización secuencial del crítico.

Dado que la comparación se basa en procedimientos completos, no es posible aislar el efecto directo de un solo elemento, como la actualización secuencial o el crítico centralizado. Determinar dicho impacto requeriría una retirada de componentes, evaluando el sistema sin esa pieza específica mientras los demás factores permanecen constantes. De este modo, la solución puede contrastarse con los antecedentes recientes en términos de representación, paradigma de entrenamiento y capacidad para tratar agentes heterogéneos. El capítulo 3 describe cómo se concreta esta propuesta en el diseño experimental.

3

Metodología

El problema consiste en aprender políticas para controlar los semáforos de una red de tráfico simulada. Cada agente recibe observaciones del tránsito en su intersección y elige una fase permitida. Se examina la espera de cada agente y, a nivel de red, la espera de los vehículos, la atención a la demanda y la distribución de la espera entre agentes.

Se comparan implementaciones de MAPPO y una adaptación de HAPPO, con IPPO e IDQN como referencias. La actualización secuencial de los actores de HAPPO motiva su comparación con MAPPO en escenarios heterogéneos, como $4 \times 4_H$ y la red parcial de Mendoza.

El diseño contempla dos líneas de estudio. La primera compara los cuatro algoritmos en cada escenario: busca hiperparámetros, selecciona las configuraciones admisibles y entrena desde cero las políticas que luego evalúa. La segunda estudia, con MAPPO, el efecto conjunto de incorporar información vecinal a la observación y a la recompensa.

El capítulo describe los escenarios y el sistema de control. Luego presenta la búsqueda y selección de configuraciones, la comparación de variantes y los procedimientos de evaluación y análisis.

3.1. Diseño experimental

Las dos fases de búsqueda abarcan los cuatro escenarios: la fase 1 explora dominios amplios y la fase 2 los refina a partir de los resultados del mismo escenario. La búsqueda aporta configuraciones candidatas; la selección determina cuáles son admisibles para el entrenamiento final. Los modelos obtenidos se evalúan por el desempeño de cada agente y por el comportamiento global de la red.

En las redes sintéticas, entre ambas fases de búsqueda se desarrolla la segunda línea, que mantiene fija la configuración de aprendizaje en cada campaña. La prueba A–E examina cinco formulaciones locales y vecinales de observación y recompensa con MAPPO; la comparación A/C contrasta las formulaciones local (A) y vecinal (C) en las tres redes sintéticas. Sus protocolos se detallan en las secciones 3.5.1 y 3.5, respectivamente.

La medida principal es la espera media observada hasta el cierre entre los vehículos que ingresan durante la ventana de evaluación, incluidos los viajes inconclusos. Se interpreta junto con la cantidad de vehículos insertados y la proporción que llega a destino. Así se examina si una espera menor coincide con una reducción del tránsito atendido; se ha señalado que el tiempo medio de viaje puede ocultar congestión cuando los vehículos nuevos no consiguen ingresar a la red [JLW+24]. Las métricas y las poblaciones que utilizan se precisan en la subsección 3.6.1.

La comprobación operacional de Mendoza forma parte de la preparación del escenario y examina el funcionamiento de la red y la demanda con programas semafóricos de referencia, distinguiéndose de la búsqueda de políticas aprendidas. La Figura 3.1 muestra cómo los escenarios preparados alimentan las dos líneas.

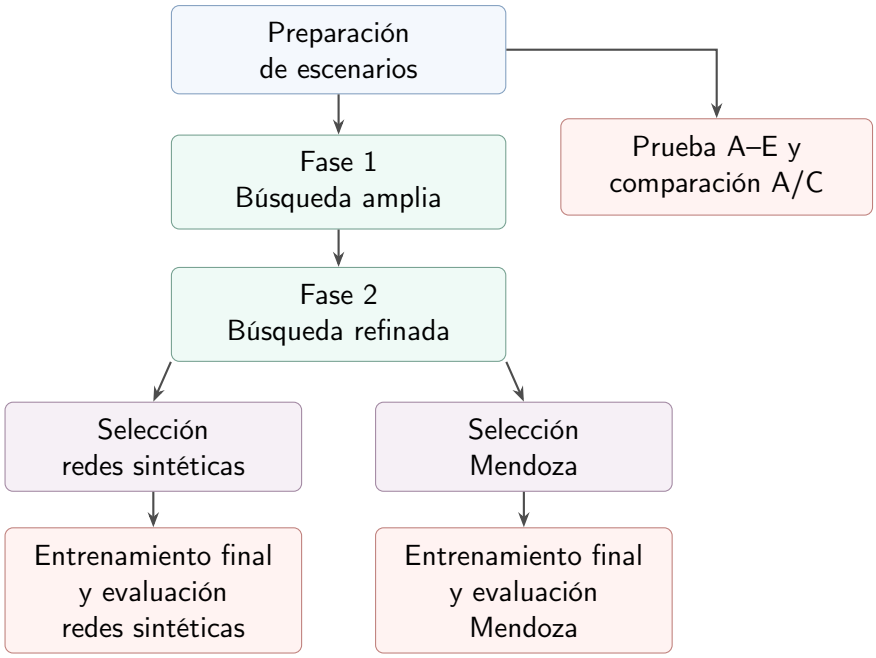


Figura 3.1: Flujo de las etapas experimentales. Los escenarios preparados alimentan la búsqueda de hiperparámetros y los estudios de variantes. Las fases 1 y 2 abarcan los cuatro escenarios; A–E y A/C utilizan configuraciones fijas en las redes sintéticas. Las flechas representan dependencias, no el orden cronológico de las campañas; la selección final utiliza los resultados de la búsqueda.

3.2. Escenarios de simulación

La geometría de cada escenario determina los carriles, las fases y las relaciones de vecindad entre agentes, mientras que la demanda define los viajes que deben atender. Los cuatro escenarios se simulan en SUMO [DLR26]. A continuación se describen sus condiciones de simulación y la construcción del escenario urbano.

3.2.1. Preparación de las redes de tráfico

Para cada red de tráfico se definieron calzadas, carriles por aproximación y longitudes de almacenamiento mediante `netedit` o, en el escenario de Mendoza, datos cartográficos de OSM. Los programas semafóricos incluidos en los archivos de red se conservaron como referencia para las simulaciones. En las redes de tráfico sintéticas son programas de tiempo fijo y sus duraciones se resumen en la Tabla 3.1; los de Mendoza se describen en el apéndice A.4.

Las secuencias de la Tabla 3.1 y la Figura 3.2 siguen el orden de ejecución de cada ciclo de 90 segundos. En la convención adoptada, V_i designa el intervalo de verde de la fase i, A_i el intervalo de cambio amarillo asociado (3 s) y R el estado de todo rojo o despeje de intersección (1 s). C1, C4 y D1 son identificadores de intersecciones de la red de tráfico $4 \times 4_H$.

Tabla 3.1: Duraciones de los programas semafóricos de referencia en las redes de tráfico sintéticas (segundos).

Red de tráfico e intersecciones	Secuencia cronológica de fases e intervalos	Ciclo
3×1: todas (3)	$V_1 = 42,\ A_1 = 3,\ V_2 = 42,\ A_2 = 3$	90 s
3×3: todas (9)	$V_1 = 42,\ A_1 = 3,\ V_2 = 42,\ A_2 = 3$	90 s
$4 \times 4_H$: 13 intersecciones con secuencia común	$V_1 = 42,\ A_1 = 3,\ V_2 = 42,\ A_2 = 3$	90 s
$4 \times 4_H$: C1	$V_1 = 38,\ A_1 = 3,\ V_2 = 37,\ A_2 = 3,$ $V_3 = 6$ (giro protegido), $A_3 = 3$	90 s
$4 \times 4_H$: C4 y D1	$V_1 = 42,\ A_1 = 3,\ V_2 = 41,\ A_2 = 3,$ $R = 1$ (todo rojo)	90 s

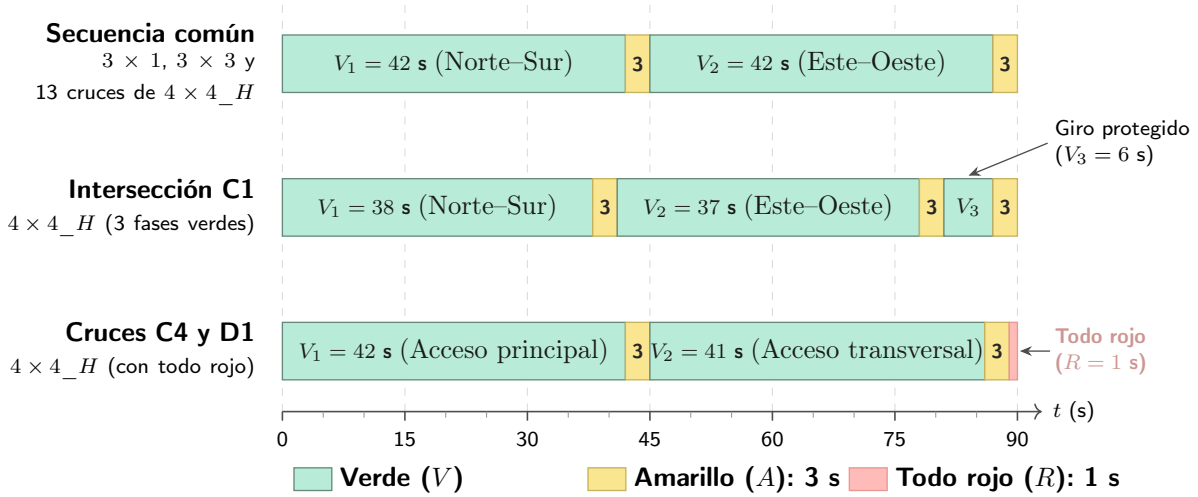


Figura 3.2: Partición temporal de los programas semafóricos de referencia en las redes de tráfico sintéticas. Cada barra representa un ciclo de 90 segundos y distingue los intervalos de verde (V), cambio amarillo (A) y todo rojo (R); las barras resumen duraciones y no desagregan el estado de cada movimiento.

En C4, un movimiento conserva el verde durante el primer intervalo amarillo. Durante el control aprendido, los agentes seleccionan las fases permitidas y las transiciones se rigen por las duraciones de la subsección 3.4.1.

3.2.2. Redes de tráfico sintéticas

Todas las redes de tráfico sintéticas comparten las mismas reglas operacionales. El límite de velocidad vial es 13,89 metros por segundo (50 kilómetros por hora), incluso si el tipo de vehículo admite una velocidad mayor. La flota del archivo `urban_vehicle_mix.add.xml` combina tres perfiles, resumidos en la Tabla 3.2. Las variables a_{veh} y d_{veh} denotan la aceleración y la desaceleración, en metros por segundo cuadrado; $v_{\text{máx}}$ es la velocidad máxima, en metros por segundo [DLR26].

Tabla 3.2: Perfiles de vehículos de las redes de tráfico sintéticas.

Perfil (tipo SUMO)	Proporción	a_{veh} (m/s^2)	d_{veh} (m/s^2)	$v_{\text{máx}}$ (m/s)
Lento (`SLOWVEHTYPE`)	25 %	1,9	4,0	8,33
Estándar (`MEDVEHTYPE`)	60 %	2,6	4,5	13,89
Rápido (`FASTVEHTYPE`)	15 %	3,1	5,0	19,44

Los vehículos miden 4,5 metros y mantienen una separación mínima de 2,5 metros (`minGap`). Se utiliza el modelo de seguimiento de Krauss, que determina cómo un vehículo ajusta su velocidad respecto del que circula delante. Su parámetro adimensional $\sigma = 0{,}5$ regula la variabilidad aleatoria de la conducción [Kra98, DLR26].

Las rutas se generaron con `random_routes_taz.py` a partir de matrices origen–destino (Origin-Destination, OD). Estas distribuyen el flujo entre zonas de análisis de tránsito (Traffic Analysis Zone, TAZ). Los archivos expresan el flujo solicitado en vehículos por hora mediante `vehsPerHour`; los registros no especifican una distribución particular de los intervalos entre llegadas. La construcción de las matrices y las asignaciones principales se detalla en el apéndice A.5.3.

Las Figuras 3.3, 3.4 y 3.5 muestran las tres redes de tráfico en SUMO. Las redes 3×1 y 3×3 tienen cuatro accesos por cruce y dos carriles por sentido; en $4 \times 4_H$, los tramos tienen entre uno y cuatro carriles por sentido de circulación y existen calles de sentido único.

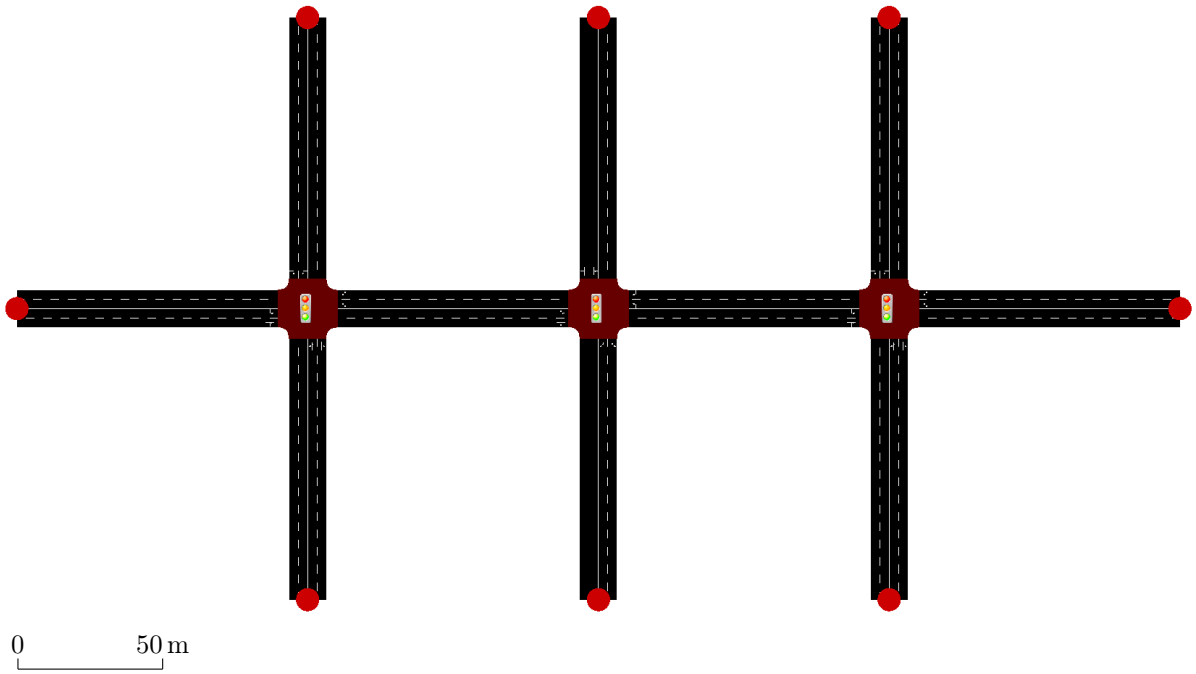


Figura 3.3: Red 3×1 en SUMO: tres intersecciones semaforizadas y dos carriles por sentido.

Las redes sintéticas tienen una separación nominal de 100 metros entre centros de

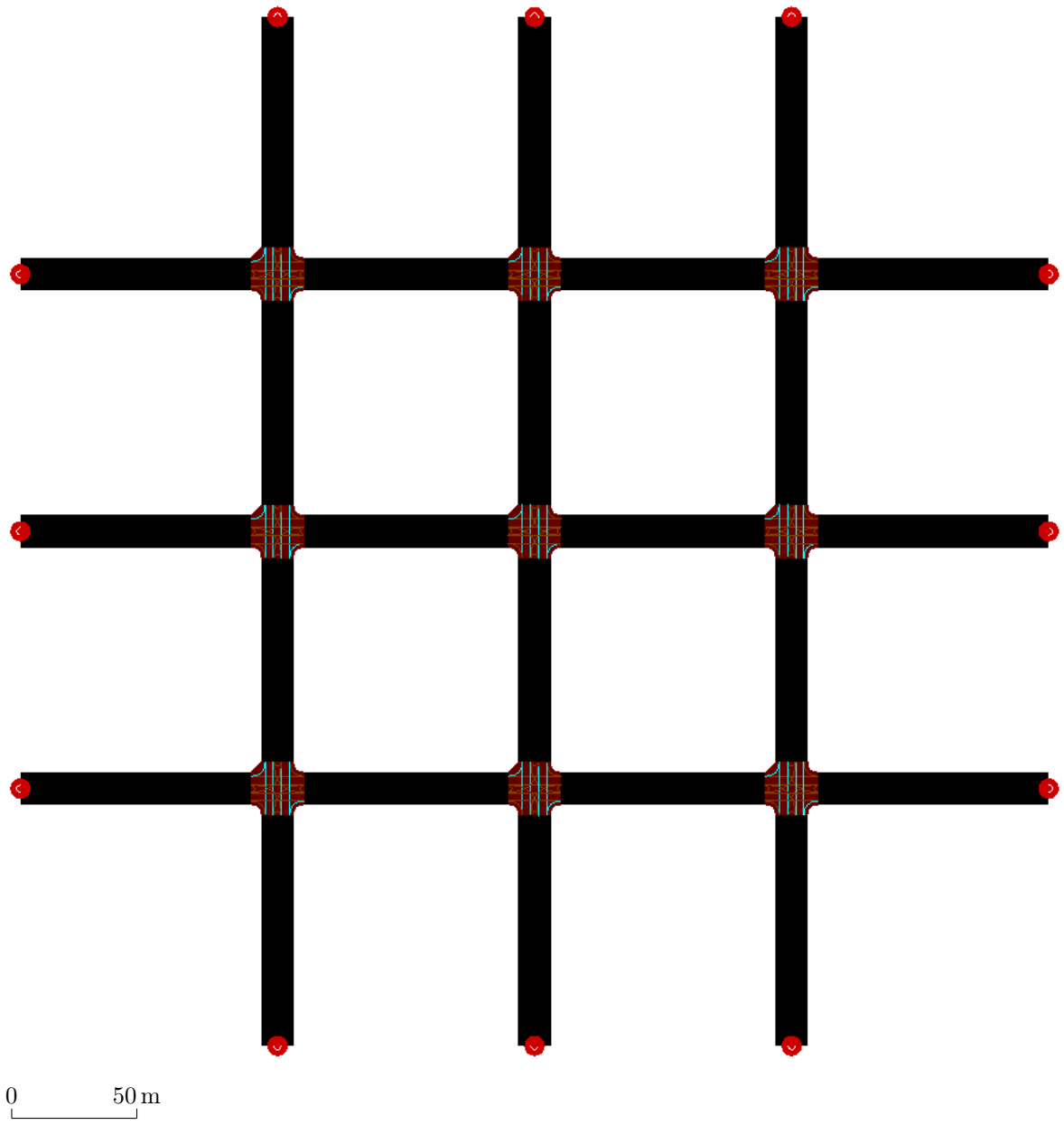


Figura 3.4: Red 3 × 3 en SUMO: nueve intersecciones semaforizadas y dos carriles por sentido.

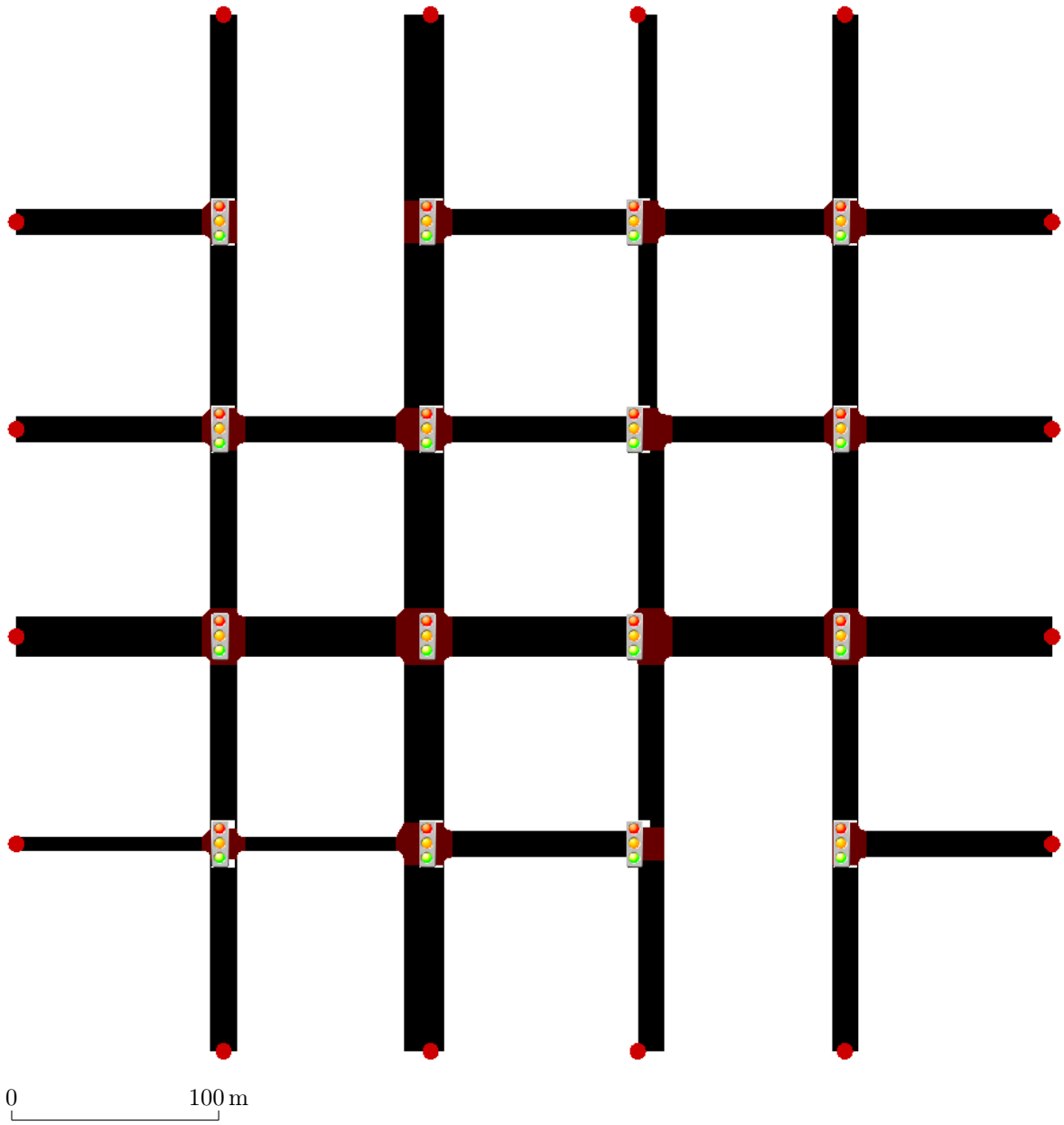


Figura 3.5: Red $4 \times 4_H$ en SUMO: dieciséis intersecciones semaforizadas y tramos de uno a cuatro carriles por sentido de circulación.

cruces consecutivos. Esta distancia no equivale necesariamente a la longitud disponible para almacenar vehículos: en 3×3 y $4 \times 4_H$, los carriles de los tramos terminan en el límite de la zona interna de cada intersección. Para comparar las redes se cuentan únicamente los tramos de circulación y sus carriles, sin los elementos internos con los que SUMO representa los movimientos dentro de los cruces.

El número de carriles entrantes controlados por el agente i se denota por m_i, y el de fases seleccionables, por K_i. El grado de vecindad es la cantidad de agentes conectados directamente con uno; el criterio de conexión se precisa en la subsección 3.3.1. La Tabla 3.3 reúne la topología, la demanda y las dimensiones del control; los intervalos describen la variación entre agentes de una misma red.

Tabla 3.3: Topología, control y demanda de las redes sintéticas. Se excluyen los tramos y carriles internos de los cruces.

Característica	3×1	3×3	$4 \times 4_H$
Agentes	3	9	16
Tramos de circulación	20	48	71
Carriles	40	96	166
Carriles por tramo y sentido	2	2	1–4
Carriles entrantes controlados (m_i)	8	8	5–12
Acciones por agente (K_i)	2	2	2 o 3
Grado de vecindad	1–2	2–4	1–4
Zonas de análisis de tránsito	8	12	16
Flujo solicitado (vehículos por hora)	3.260	7.087	7.202

El corredor 3×1 consta de tres cruces en línea, identificados como B1, C1 y D1. La demanda concentra el 75 % del flujo solicitado en los dos sentidos de la arteria y distribuye el resto entre los pares válidos. Esta configuración permite estudiar la propagación longitudinal de colas y la coordinación sobre las corrientes predominantes; las asignaciones por par se presentan en el apéndice A.5.3.

La red 3×3 dispone de nueve cruces en una cuadrícula regular. Frente al corredor, incorpora conexiones en dos direcciones y permite estudiar la coordinación bidimensional con tamaños locales de observación y acción uniformes. La distribución de demanda por par se conserva en el apéndice A.5.3.

La red $4 \times 4_H$ dispone de dieciséis cruces sobre una cuadrícula con capacidad y conectividad heterogéneas. La fila 2 es una arteria de tres carriles por sentido; el sector A1–B1–C1 tiene un carril por sentido. La columna C presenta cuatro carriles hacia el sur y dos hacia el norte, y la columna D es de sentido único. C1 tiene tres acciones seleccionables, mientras que los demás cruces tienen dos. Así, el escenario incorpora diferencias de capacidad, conectividad y tamaño de observación y acción. Las vías y asignaciones OD específicas se describen en el apéndice A.5.3.

3.2.3. Red de tráfico de la Ciudad de Mendoza

El escenario modela parte del microcentro de la Ciudad de Mendoza y utiliza la red y el archivo de flujos congelados de la versión v14. El trazado cartográfico define la red vial y los conteos aportan un perfil temporal del tránsito. Para construir la demanda se fija un volumen nominal y se distribuye el flujo entre orígenes y destinos bajo supuestos explícitos; la red y las rutas resultantes constituyen las entradas de simulación. Su identidad y procedencia se documentan en el apéndice A.4.1.

Fuentes de datos y preprocesamiento de los conteos

Los conteos disponibles cubren accesos específicos. El conteo de un acceso informa sobre el volumen y su variación horaria en ese punto, pero no identifica por sí solo los orígenes y destinos de todos los viajes. Por ello, el volumen total y la distribución espacial de la demanda incorporan supuestos explícitos [CyMR18, PW16].

Los registros de cámaras térmicas instaladas en el entorno de Costanera y Acceso Este contienen conteos clasificados en intervalos de 15 minutos y abarcan aproximadamente de octubre de 2025 a abril de 2026. Se examinaron cuatro dispositivos: tres observan corrientes fuera del área modelada; para representar el ingreso hacia el oeste por la Avenida José Vicente Zapata, sobre el canal Cacique Guaymallén, se tomó el registro de la cámara 169.

Los conteos se depuraron y redondearon al intervalo de 15 minutos más cercano, y se agregaron por cámara, intervalo, zona y clase vehicular. El registro de generación asociado a la semilla 104773 informa una selección de 32 días completos de lunes a viernes y un promedio diario de 25.175,84 vehículos. Estos valores describen el perfil registrado: no reconstruyen de forma independiente los conteos ni miden el volumen total urbano. Los criterios de limpieza y la procedencia de los registros se detallan en el apéndice A.4.1.

Representación y revisión de la red de tráfico

El trazado geométrico base se obtuvo mediante la exportación de datos de OSM y su conversión al formato de red de SUMO (`.net.xml`). El dominio modelado delimita el microcentro de la Ciudad de Mendoza entre Av. José Vicente Zapata / Colón al sur, Av. General San Martín como eje central norte-sur, Av. Manuel Belgrano al oeste y Av. Las Heras al norte. Para expresar las posiciones del mapa en coordenadas planas se utilizó el sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*, UTM), zona 19S. La referencia geodésica, que permite ubicar esas coordenadas sobre la Tierra, fue el Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*, WGS84).

El archivo `gran_mendoza2_fixed.net.xml` contiene 140 nodos de intersección; 74 están semaforizados y se agrupan en 73 controladores, cada uno asociado con un agente. La red comprende 255 tramos de circulación y 554 carriles. Los agentes controlan entre 3 y 10 carriles entrantes y tienen entre una y cuatro acciones posibles. La demanda se

organiza en 39 zonas de análisis de tránsito, con 520 pares origen–destino conectables. La escala y la diversidad de las condiciones locales distinguen este escenario de las redes sintéticas; las dimensiones detalladas de la red se presentan en el apéndice A.4.1.

La Figura 3.6 muestra la red de tráfico en SUMO. Los tramos tienen entre uno y cuatro carriles por sentido de circulación.

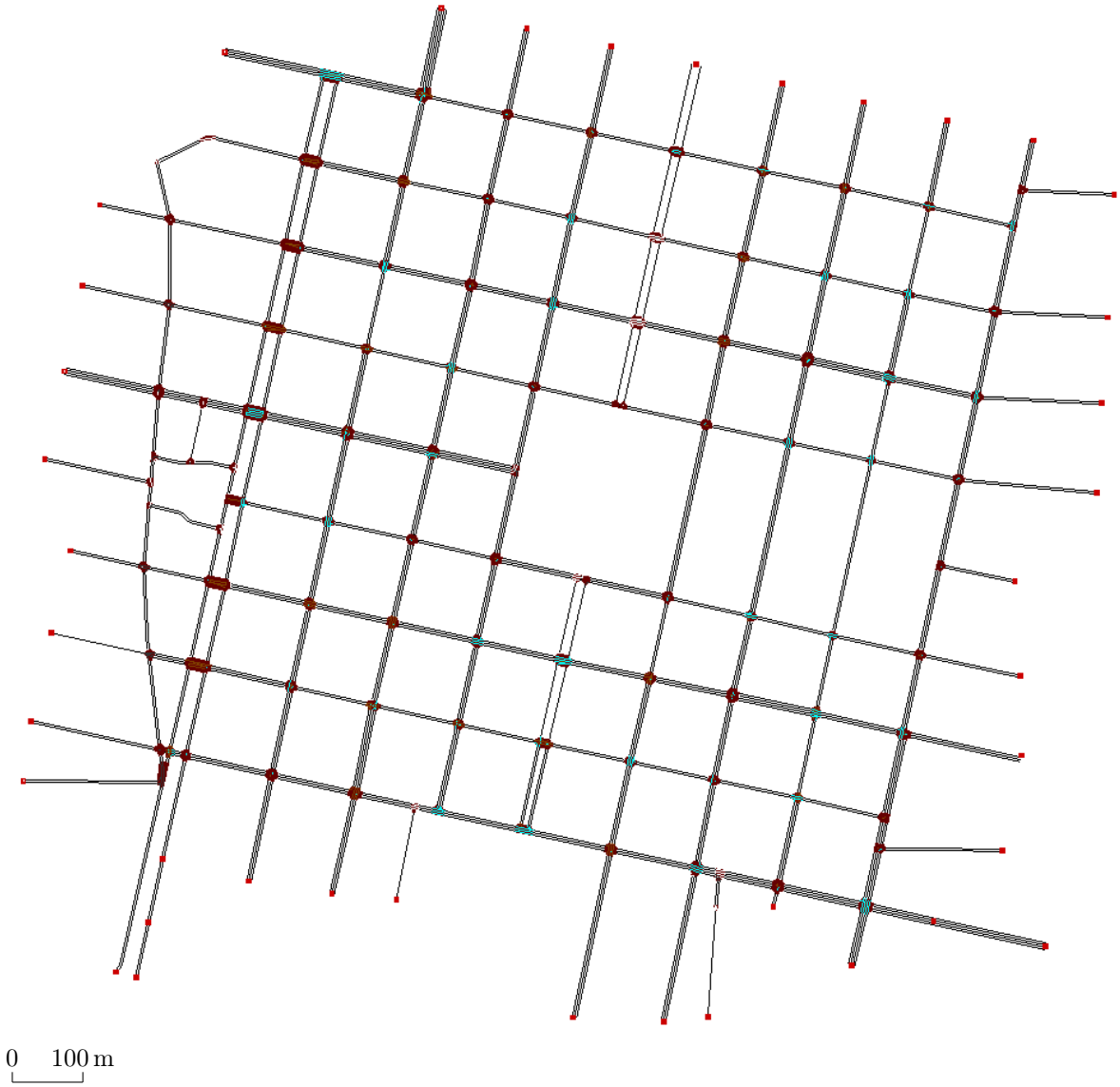


Figura 3.6: Red de tráfico de una porción del microcentro de Mendoza en SUMO.

Como la cámara 169 queda fuera del límite modelado, el ingreso que representa su conteo se asigna al primer tramo interno; la red de tráfico no se extiende hasta el punto de medición.

Tras convertir el trazado de OSM, la revisión con imágenes satelitales y vistas de calle permitió corregir sentidos de circulación, conectividad, carriles y restricciones de giro, y habilitar accesos laterales en Catamarca, Buenos Aires y Leandro N. Alem. Entre los cambios se incluyeron la prohibición de girar a la izquierda desde Av. General San Martín

hacia el sur por Av. Colón y la conservación del giro semaforizado de calle Perú hacia Av. Las Heras.

Los programas de señales incluidos en el archivo de red se utilizan como referencias de simulación dentro de una aproximación topológica del área: 17 son estáticos y 56 accionados. Las referencias conservan esos programas; sus duraciones nominales se describen en el apéndice A.4.

Construcción de la demanda

La demanda nominal se construyó a partir del perfil horario promedio de la cámara 169, $\bar{q}_{\text{obs}}(h)$, expresado en vehículos por hora, donde h identifica una hora entre 0 y 23. El perfil se escaló a un volumen diario nominal V_{diario} de 260.000 vehículos, fijado como supuesto. Sea $\Delta h = 1$ hora la duración de cada intervalo; la ecuación (3.1) define el factor adimensional f_{exp} que relaciona el volumen nominal con el total diario del perfil:

$$f_{\text{exp}} = \frac{V_{\text{diario}}}{\sum_{h=0}^{23} \bar{q}_{\text{obs}}(h)\,\Delta h} \tag{3.1}$$

El registro de generación informa un factor de 10,32735993. Se conserva como valor registrado; la Tabla A.3 distingue su procedencia de la del resumen histórico. La ecuación (3.2) aplica ese factor para obtener el flujo horario nominal de la red:

$$Q_{\text{red}}(h) = \bar{q}_{\text{obs}}(h) \cdot f_{\text{exp}} \tag{3.2}$$

$Q_{\text{red}}(h)$ se expresa en vehículos por hora. En la generación se conservó el conteo de la cámara como referencia completa; esto es un supuesto de construcción, pues no se conoce qué fracción de esos vehículos llega al área modelada ni cuántos ingresan por accesos no observados.

Distribución origen–destino y generación de rutas

La Figura 3.7 muestra los tramos asociados a las zonas de análisis de tránsito en la vista de SUMO. Las zonas se definen por sus tramos asociados; los recuadros de la captura no representan límites territoriales.

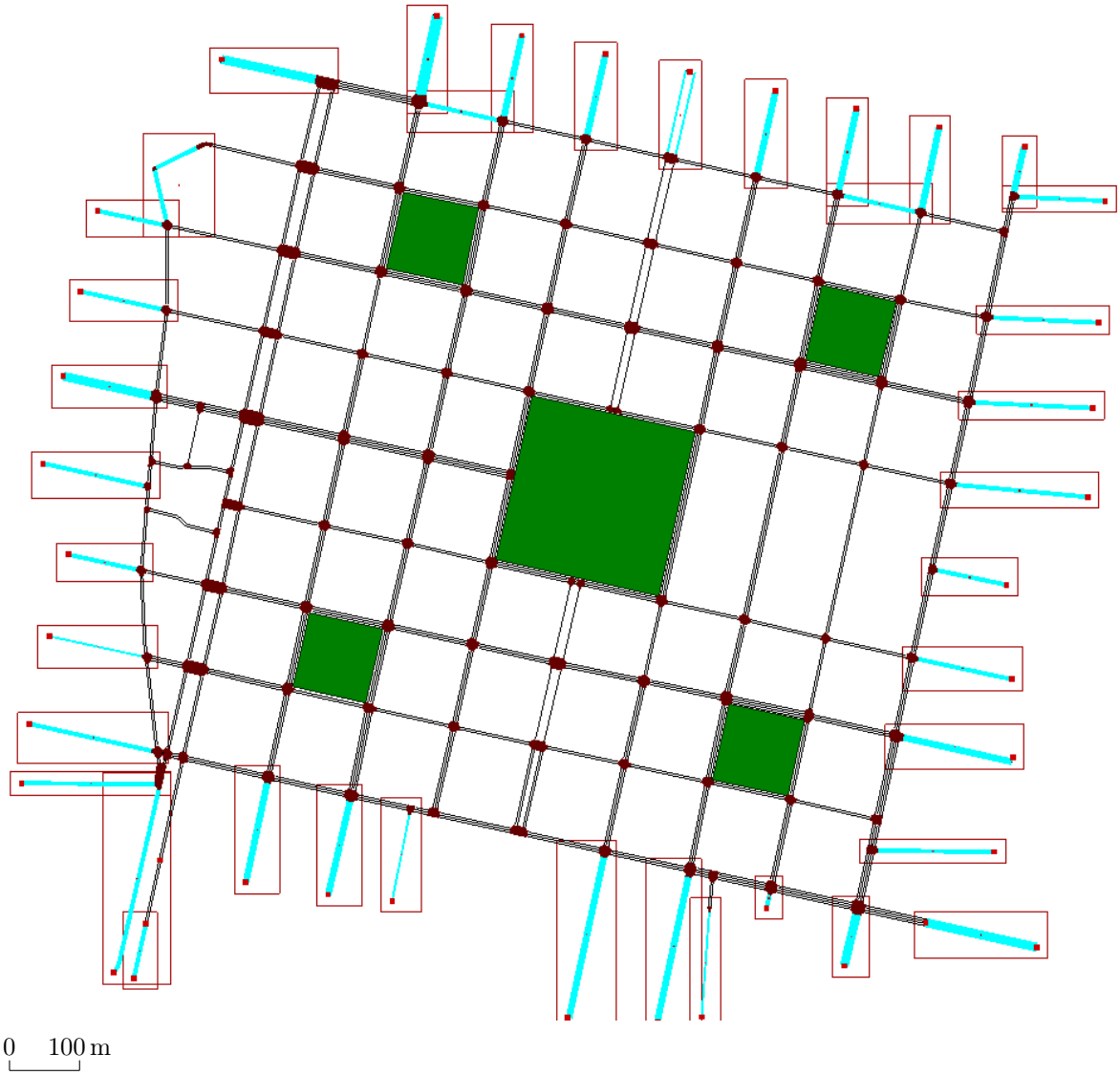


Figura 3.7: Zonas de análisis de tránsito de Mendoza en SUMO. Los tramos asociados a las zonas se destacan en celeste y los recuadros rojos permiten localizarlos en la vista.

El flujo horario nominal se distribuye entre pares origen–destino. La especificación canónica de la matriz OD asigna 60 % del flujo a pares con proporciones fijadas y 40 % a pares con proporciones aleatorias. Prioriza los corredores Vicente Zapata, General San Martín, Colón–Emilio Civit, Las Heras y Manuel Belgrano, según la revisión cartográfica y los conteos disponibles. También excluye tres pares en los que se habían observado bloqueos durante la construcción: esta decisión limita la demanda representada, pero no garantiza que otras simulaciones estén libres de bloqueos. La formulación se detalla en el apéndice A.4.1.

El registro de generación describe otra distribución, 70/30. No se conservaron la matriz asociada a ese registro ni las rutas originales de las comprobaciones; por ello, no es posible reconstruir la relación entre ambas especificaciones ni verificar que la ruta congelada fuera la utilizada en esas corridas.

La semilla 104773 controla la generación de rutas y la semilla 104729 inicializa la dinámica estocástica de SUMO. La documentación describe la elección de 104773 entre configuraciones consideradas por permitir una operación de doce horas sin teletransportes y aproximarse a la distribución espacial buscada; esta selección no constituye una validación independiente y su reconstrucción es incompleta.

Llegadas y flota. Para cada par OD con flujo horario positivo Q_{od}, el procedimiento programa llegadas con intervalos exponenciales de tasa $\lambda_{od} = Q_{od}/3600$ vehículos por segundo, según la distribución presentada en la ecuación (2.4). La transformación utilizada se conserva en el apéndice A.4.1. Estos intervalos describen llegadas programadas, no tiempos de cruce: si el acceso está ocupado, el vehículo se inserta cuando hay espacio. La flota combina un 25 % de vehículos lentos, un 60 % estándar y un 15 % con mayor aceleración.

Comprobación operacional y contraste con los conteos

La comprobación operacional examina si la red de tráfico y la demanda construidas funcionan con los programas semafóricos conservados; no evalúa políticas aprendidas. El tiempo de simulación t_{sim} se expresa en segundos desde las 06:00. Se registran dos horizontes: 12 horas diurnas ($t_{\text{sim}} = 0$–43200) y dos horas de hora pico matutina ($t_{\text{sim}} =$ 1800–9000, correspondientes a 06:30–08:30).

La simulación de hora pico con programas de referencia comienza en $t_{\text{sim}} = 1800$ y no añade calentamiento, es decir, un período previo para poblar la red antes de medir el control. La configuración MARL correspondiente comienza en $t_{\text{sim}} = 1500$ (06:25) y reserva 300 segundos para ese período antes del mismo intervalo de control. Los archivos y parámetros se describen en el apéndice A.4.1.

Los registros consultados contienen conteos de carga, inserción y viajes completados antes del cierre. Con ellos se reportan tres proporciones: completitud sobre vehículos cargados (CR), inserción sobre vehículos cargados (IR) y completitud sobre vehículos insertados (CI). En las dos medidas de completitud, el numerador cuenta los viajes completados; en la de inserción, cuenta los vehículos que ingresaron a la red. Las fórmulas se presentan en la subsección 3.6.1; los valores se muestran en la Tabla 4.1, en el capítulo de Resultados (4).

El informe documental del escenario canónico indica cero teletransportes; las salidas de simulación consultadas no conservan un contador explícito que permita verificar ese dato de forma independiente. La procedencia de los registros y sus configuraciones se documentan en el apéndice A.4.

Contraste con el conteo observado. El criterio estadístico de comparación de flujos de Geoffrey E. Havers (GEH) compara el flujo horario nominal solicitado desde la zona 28, asociada al acceso representado (V_{mod}), con el flujo horario del perfil de la cámara 169 (V_{obs}); ambos se expresan en vehículos por hora [PW16]. Su fórmula se conserva en

la ecuación (A.3) del apéndice. Como el conteo de la cámara 169 contribuyó a construir el perfil, esta comparación no constituye una validación independiente de la demanda urbana.

3.3. Sistema de control multiagente

Con las redes y sus demandas definidas, el sistema implementa el ciclo agente–entorno presentado en el marco teórico: convierte las mediciones del tránsito en información para los agentes y aplica sus decisiones a la simulación. La misma interfaz permite utilizar los cuatro algoritmos del estudio. Sus observaciones, acciones y recompensas definen la información disponible y las restricciones bajo las que se entrenan las políticas.

Cada agente i gobierna las señales de una intersección; si varios nodos comparten un mismo controlador semafórico, se representan mediante un único agente. Los tamaños m_i y K_i describen sus carriles entrantes controlados y sus fases seleccionables. Las fases con amarillo y el estado final de todo rojo no integran ese conjunto de acciones; el entorno los utiliza para las transiciones y la inicialización, respectivamente. La cantidad de acciones habilitadas en un instante también depende de las restricciones temporales del control.

El paquete `marlTLC` integra el simulador SUMO con Ray RLlib, biblioteca que coordina la recolección de experiencias y las actualizaciones de aprendizaje [LLN+18a], por medio del entorno `SumoRllibEnv`. Este entorno consulta y modifica la simulación mediante libsumo, organiza las mediciones como observaciones, aplica las acciones semafóricas y calcula las recompensas. PyTorch es la biblioteca que calcula y actualiza las redes neuronales. La Figura 3.8 sitúa estas operaciones entre la simulación y el aprendizaje; las herramientas y clases principales se describen en el apéndice A.1.

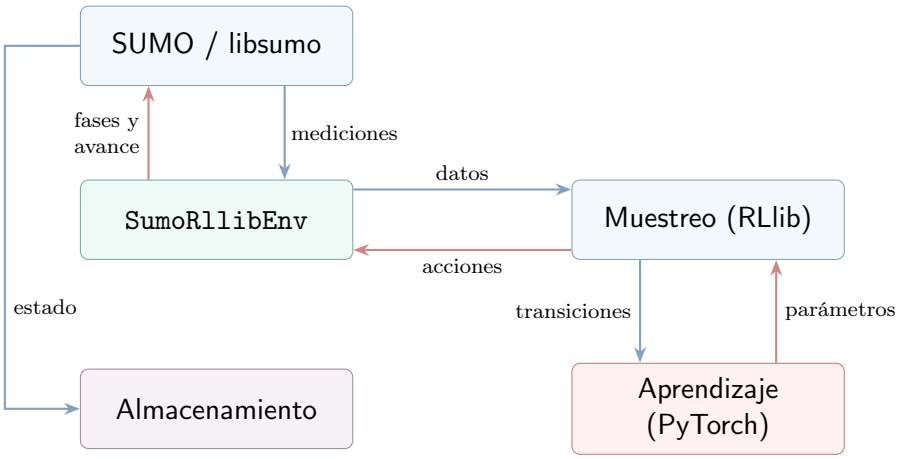


Figura 3.8: Flujo entre la simulación, el entorno y el aprendizaje. El estado de SUMO se almacena aparte del flujo de aprendizaje.

3.3.1. Entorno de simulación

El entorno guarda un estado de SUMO correspondiente al inicio del control. Cuando se añade calentamiento, la simulación avanza durante ese período antes de guardarlo; con calentamiento de duración cero, el estado corresponde al inicio de la simulación. Al llamar a `reset`, el entorno carga ese estado y reinicia los contadores, los datos temporales y la espera atribuida a cada carril. En el control aprendido, también coloca los semáforos en su estado inicial de seguridad, con todas las señales en rojo.

En el protocolo urbano, el estado guardado incluye los generadores aleatorios de SUMO; cada semilla dispone de su propio estado posterior al calentamiento [Ecl26a]. La referencia conserva los programas de la red durante el control, mientras que las políticas comienzan desde el estado de seguridad. Esta diferencia de inicialización forma parte de las condiciones de comparación.

En cada paso, `step` recibe las acciones de los agentes, ejecuta las transiciones y avanza la simulación hasta la decisión siguiente. Luego actualiza las mediciones y devuelve observaciones, recompensas e información del episodio.

Los episodios cierran al alcanzar su límite temporal; la interfaz distingue terminación y truncamiento. En la versión conservada para A/C, el límite temporal activa ambos indicadores, como se explica en el apéndice A.1.

En esta comparación con MAPPO, ambos indicadores se transmiten al cálculo de los objetivos de valor y las ventajas; su tratamiento conjunto puede afectar la incorporación del valor del estado final mediante *bootstrapping* (ecuación 2.18). Por ello, este cierre temporal forma parte de la interpretación de las actualizaciones. Las ejecuciones se realizaron sin interfaz gráfica.

En Mendoza, al alcanzar el horizonte se activa únicamente el indicador de truncamiento; el de terminación permanece falso. Así se distingue el corte temporal de un estado terminal al calcular los objetivos de aprendizaje. El episodio urbano tampoco se cierra por una ausencia momentánea de vehículos.

El entorno considera vecinos a dos agentes cuando una conexión desde un carril controlado por uno desemboca en un carril controlado por el otro, y registra la relación para ambos. Por ello, una vía de sentido único también puede generar una relación de vecindad simétrica.

3.3.2. Observación, acciones y recompensas

La formulación utilizada representa la fase activa y la espera por carril en la observación, restringe las acciones según la señalización y calcula la recompensa a partir de cambios de espera.

Observaciones y codificación de la espera

En el paso de decisión t, el entorno atribuye al carril l del agente i una espera acumulada $w_{i,l}^{(t)}$, expresada en vehículos-segundo (veh·s), que representa la suma de las esperas atribuidas a los vehículos presentes en él. Para cada vehículo presente, la implementación resta de la espera devuelta por SUMO las atribuciones almacenadas en otros carriles y limita el resultado a un mínimo de cero. La suma de estas medidas en los m_i carriles controlados es $W_i^{(t)} = \sum_{l=1}^{m_i} w_{i,l}^{(t)}$, también en veh·s.

La función de codificación $e(x)$ transforma una medida de espera x en un nivel entero. La observación local $o_i^{(t)}$ reúne el índice $p_i^{(t)}$ de la fase activa y las esperas codificadas de sus carriles, según la ecuación (3.3). El índice de fase también puede identificar el estado inicial de todo rojo, aunque no sea una acción seleccionable. En las búsquedas de hiperparámetros se usa esta entrada local; las políticas no conservan un historial de observaciones:

$$o_i^{(t)} = \big(p_i^{(t)}, e(w_{i,1}^{(t)}), \ldots, e(w_{i,m_i}^{(t)})\big), \tag{3.3}$$

La variante vecinal C añade a esa entrada un valor por cada vecino directo j: la espera acumulada de sus carriles, $W_j^{(t)}$, dividida por su cantidad de carriles, m_j. Cada promedio se codifica con la misma función y el límite de escala vecinal M_{vec}, expresado en veh·s por carril. Los valores y las semillas de calibración de ambos límites de escala se presentan en el apéndice A.6.4.

SUMO obtiene la espera acumulada mediante `getAccumulatedWaitingTime`, que considera una ventana temporal finita denominada memoria de espera [Ecl26b]. Su duración se detalla en el apéndice A.4. Esta magnitud se distingue de la espera por viaje registrada en `TripInfo`, una salida de SUMO con información individual de los viajes, como los instantes de inserción y llegada y el tiempo de espera (`waitingTime`) [Ecl26c]. La salida de vehículos, los cambios de carril y el vencimiento de la ventana pueden reducir la señal observada sin demostrar que disminuyó la espera total de los viajes.

En la versión de código conservada para A/C, las atribuciones almacenadas en otros carriles no se actualizan por el vencimiento de esa memoria, lo que puede subestimar la espera atribuida al carril actual. Este comportamiento limita la interpretación de la señal local como medida de espera.

El codificador transforma la espera acumulada por carril en uno de I niveles, entre 0 e $I-1$. El parámetro local M_{lane}, denominado `max_lane_value` en el código y expresado en veh·s, fija el límite de la escala de codificación. La transformación combina un componente logarítmico, que da mayor resolución a esperas bajas, y otro lineal. Como la salida se redondea hacia arriba mediante la función techo (ceil), el nivel máximo puede alcanzarse antes de llegar a ese límite. Las ecuaciones y la forma computada por el código se desarrollan en el apéndice A.6.1.

El número de niveles I se muestrea entre los valores permitidos para cada combinación de algoritmo y escenario. En la fórmula del apéndice, I interviene también en el peso que combina las componentes logarítmica y lineal; por ello, variarlo modifica tanto la resolución como la forma de la transformación.

Para las redes sintéticas, el límite de escala M_{lane} se estimó en simulaciones previas mediante el percentil 99 de la mayor espera entre carriles en cada paso, después de 300 segundos de calentamiento. Los valores son 1184, 1136 y 712 veh·s para 3×1, 3×3 y $4 \times 4_H$, respectivamente, y fijan la escala de codificación para esperas elevadas.

En Mendoza se fija $M_{\text{lane}} = 5396$ veh·s mediante una calibración con los programas de referencia y una memoria de espera de 1000 segundos, compartida con el entorno de aprendizaje. Se toma el percentil 99 del máximo entre los carriles registrado en cada segundo y se redondea al entero más próximo. La calibración suma la espera acumulada de los vehículos presentes; esa suma proporciona una envolvente de la espera atribuida al carril por el entorno. Las semillas, la ventana y el procedimiento se detallan en la subsección A.5.8 del apéndice.

La Figura 3.9 ilustra la codificación de la observación local con el límite de escala estimado para 3×3. La comparación entre las variantes local y vecinal utiliza $I = 17$, distinto del número de niveles del ejemplo.

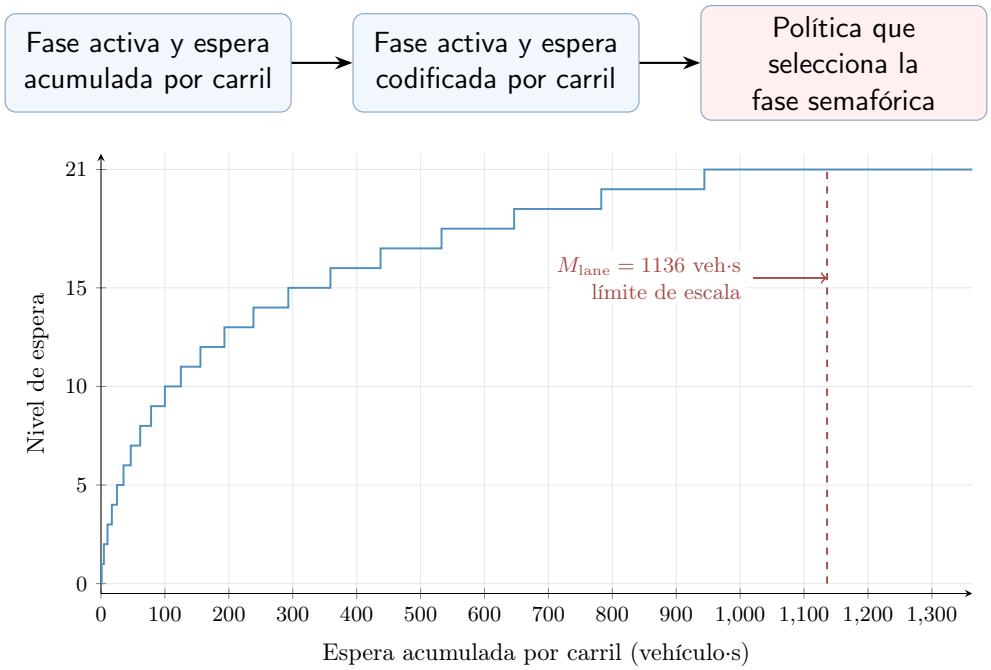


Figura 3.9: Codificación de la espera por carril con $I = 22$ y límite de escala $M_{\text{lane}} = 1136$ veh·s, como ejemplo ilustrativo. La fase activa se conserva y el nivel máximo puede alcanzarse antes de llegar a ese límite.

Una vez construida la observación, el agente selecciona una fase permitida por una máscara de acciones, regla que excluye las fases que la señalización vigente no habilita.

Acciones permitidas y cambios de fase

El agente i elige entre K_i fases verdes, numeradas de cero a $K_i - 1$. En cada decisión, la señal vigente habilita solo un subconjunto de esas acciones. Los métodos IPPO, MAPPO y HAPPO enmascaran las fases no habilitadas antes de normalizar las probabilidades de la política; IDQN las excluye al seleccionar y evaluar acciones. Se requiere al menos una fase admisible. La operación de la máscara se formaliza en el apéndice A.6.2.

Durante el verde mínimo y la transición amarilla no se admite una nueva solicitud de cambio. Las duraciones y la frecuencia de decisión se especifican en la subsección 3.4.1.

Recompensas locales y vecinales

La recompensa se calcula a partir del cambio de la espera acumulada entre decisiones. Se define primero la señal de la propia intersección y luego la contribución de sus vecinos, utilizada en la variante vecinal.

La señal local $L_i^{(t)}$ compara la suma $W_i^{(t)}$ definida para la observación entre dos pasos de decisión consecutivos. La ecuación (3.4) formaliza esta diferencia, en veh·s: la recompensa es positiva cuando la suma disminuye.

$$L_i^{(t)} = W_i^{(t-1)} - W_i^{(t)} \tag{3.4}$$

Esta señal favorece reducciones de la espera atribuida a los vehículos presentes. Su relación con una mejora operacional se examina mediante los registros de viajes y la atención a la demanda; el retorno descontado de estas diferencias no equivale directamente a la espera total de los viajes.

En la variante C, sea $\mathcal{N}(i)$ el conjunto de vecinos directos del agente i y $|\mathcal{N}(i)|$ su cantidad; m_j es el número de carriles controlados por el vecino j. La ecuación (3.5) normaliza por carriles la señal local de cada vecino y luego promedia esos valores para obtener $N_{i,\text{carril}}^{(t)}$, en veh·s por carril; si no hay vecinos, vale cero:

$$N_{i,\text{carril}}^{(t)} = \begin{cases} \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \frac{L_j^{(t)}}{m_j}, & |\mathcal{N}(i)| > 0, \\ 0, & |\mathcal{N}(i)| = 0. \end{cases} \tag{3.5}$$

En C, el coeficiente adimensional $\rho_{\text{coop}} \in [0, 1)$ determina el peso de la señal vecinal en la recompensa de la ecuación (3.6); si el agente no tiene vecinos, conserva la señal local completa:

$$R_i^{(t)} = \begin{cases} (1 - \rho_{\text{coop}}) L_i^{(t)} + \rho_{\text{coop}} N_{i,\text{carril}}^{(t)}, & |\mathcal{N}(i)| > 0, \\ L_i^{(t)}, & |\mathcal{N}(i)| = 0. \end{cases} \tag{3.6}$$

Cada agente recibe su propia recompensa; las señales no se suman en un retorno global. La finalidad del sistema es favorecer la operación coordinada de la red, mientras que el aprendizaje utiliza señales individuales cuya alineación con esa finalidad se examina

empíricamente. En la variante C, el peso vecinal se fijó en $\rho_{\text{coop}} = 0{,}1$. Como ambas señales tienen escalas distintas, ese valor pondera su influencia relativa pero no representa un porcentaje fijo de la recompensa resultante.

La Tabla 3.4 reúne las señales utilizadas en los procedimientos sintéticos. En la fase 1, las observaciones son locales y el programa promedia las señales vecinales sin normalizarlas por carriles; se explora un peso entre cero y 0,95, sin reservar un ensayo específico con $\rho_{\text{coop}} = 0$. La búsqueda refinada fija recompensa local, mientras que A/C contrasta las dos formulaciones descritas anteriormente.

Tabla 3.4: Observaciones y recompensas de las búsquedas sintéticas y de la comparación A/C.

Procedimiento	Observación	Recompensa	Peso vecinal
Fase 1 sintética	Local	Local y vecinal, sin normalización por carriles	[0; 0,95)
Búsqueda refinada sintética	Local	Local	0
Comparación: local	Local	Local	0
Comparación: vecinal	Vecinal	Local y promedio vecinal normalizado por carril	0,1

3.3.3. Agentes y algoritmos de aprendizaje

RLlib vincula un módulo de aprendizaje con cada agente. Los actores de los métodos basados en PPO tienen parámetros propios, sin compartirlos entre agentes. MAPPO y HAPPO utilizan un crítico centralizado durante el entrenamiento. El crítico reúne en un único vector las observaciones definidas para los agentes de una simulación y produce un valor para cada uno; en la variante vecinal de MAPPO, esas observaciones incluyen información de los vecinos. La entrada no es el estado físico completo de la red de tráfico.

El uso de un crítico centralizado en ambos algoritmos no exige que sus arquitecturas sean idénticas. Esta implementación sigue el paradigma CTDE presentado en el marco teórico (Figura 2.19).

En IDQN, la memoria de experiencias es un búfer priorizado: asigna distinta prioridad a las transiciones al seleccionarlas para aprender. La Figura 3.10 reúne los cuatro algoritmos y distingue la información utilizada durante el entrenamiento y la ejecución.

En MAPPO, las recompensas individuales y las estimaciones del crítico permiten calcular objetivos de valor y ventajas por agente. Las ventajas se estandarizan por agente antes de actualizar cada actor.

La implementación de HAPPO actualiza primero el crítico y después los actores, uno por uno, sobre el lote completo. Utiliza ventajas individuales y un paso de gradiente por

actor y *epoch*; tras cada actualización repondera los factores de los actores restantes. El apéndice A.6 detalla estas operaciones y sus diferencias con la formulación teórica de la sección 2.3.6.

En MAPPO y HAPPO, la pérdida del crítico limita a una cota el error cuadrático medio entre agentes para cada muestra y luego promedia las muestras válidas. A diferencia del recorte convencional de valores de PPO, esta operación satura la pérdida; por encima de la cota, esa muestra deja de aportar gradiente al crítico. La fórmula se presenta en la ecuación (A.4) del apéndice.

Como el error se promedia entre agentes antes de aplicar la cota, una muestra saturada deja de contribuir al ajuste de todas las salidas del crítico. Esto puede afectar las estimaciones de valor y las ventajas que guían a los actores. Su impacto efectivo requiere conocer la frecuencia de saturación, que no se cuantifica en los registros descritos aquí.

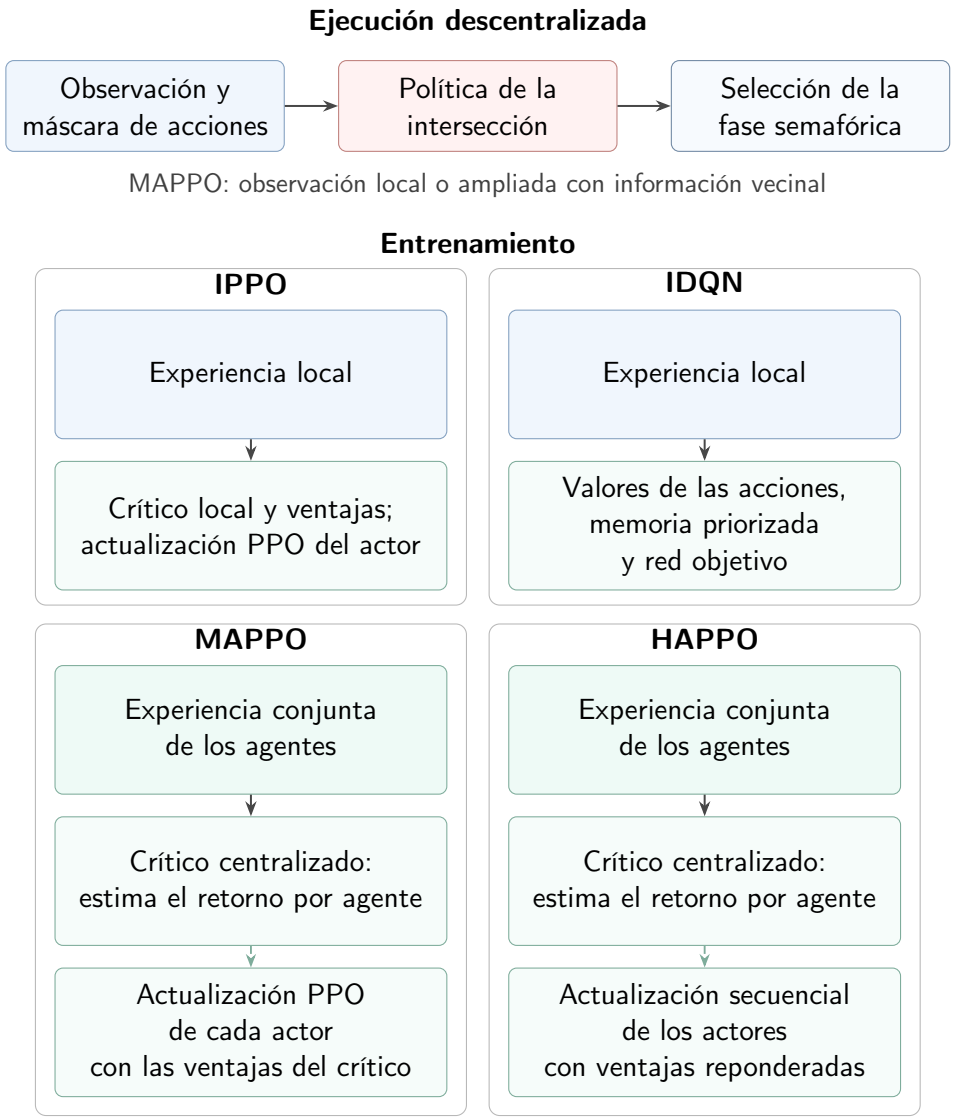


Figura 3.10: Información de los algoritmos durante el entrenamiento y la ejecución. Cada algoritmo y agente tiene su propia política; el crítico centralizado de MAPPO y HAPPO no interviene al ejecutar. En la variante vecinal de MAPPO, el actor recibe además la espera media por carril codificada de cada vecino directo.

3.4. Entrenamiento y búsqueda de hiperparámetros

Con los agentes, las acciones y las señales de aprendizaje definidos, la optimización de hiperparámetros (*Hyperparameter Optimization*, HPO) explora combinaciones de valores que regulan el entrenamiento y produce configuraciones candidatas para comparar los algoritmos. El espacio de búsqueda fija los rangos y las opciones permitidos. Cada ensayo entrena una configuración propuesta; el presupuesto limita la cantidad de ensayos y su duración máxima, medida en iteraciones de entrenamiento. Una iteración puede incluir varios episodios o fragmentos y no equivale necesariamente a una simulación completa.

La búsqueda combina propuestas basadas en el historial con detenciones anticipadas para distribuir un presupuesto de entrenamiento limitado. En las búsquedas, Optuna

propone y registra configuraciones mediante el estimador de Parzen estructurado en árbol (*Tree-structured Parzen Estimator*, TPE), que usa los resultados anteriores para proponer nuevos valores [ASY+19, BBBK11]; Ray Tune organiza la ejecución y el registro de los ensayos [LLN+18b].

El algoritmo asíncrono de reducción sucesiva a la mitad (*Asynchronous Successive Halving Algorithm*, ASHA) compara el rendimiento intermedio y detiene ensayos antes de completar su presupuesto [LJR+20]. El período de gracia fija el mínimo de iteraciones antes de considerar una detención; el factor de reducción determina la proporción de ensayos que continúan en cada nivel de comparación.

En la fase 1, los cuatro escenarios usan los mismos dominios iniciales por algoritmo y los mismos límites de búsqueda, salvo por el peso de la recompensa vecinal en Mendoza. La fase 1 de Mendoza se desarrolla después de A–E y A/C y fija la observación local y $\rho_{\text{coop}} = 0$. Algunas condiciones propias de cada red, como el límite de escala del codificador, se establecen por escenario.

En la fase 2, los dominios de cada escenario se refinan a partir de los resultados de su fase 1. Los dominios sintéticos y las reglas de refinamiento urbano se presentan por separado, en esta sección y en el apéndice A.5.8, respectivamente.

3.4.1. Condiciones de simulación y entrenamiento

Los horizontes de simulación dependen de la campaña y del escenario; la Tabla 3.5 resume el calentamiento y la duración del control. Las correspondencias horarias de Mendoza se definieron en la subsección 3.2.3.

Tabla 3.5: Calentamiento y duración del control por procedimiento.

Campaña	Calentamiento	Duración del control
HPO sintética, fase 1	600 s	3600 s
HPO sintética, fase 2	600 s	3600 s
Pruebas preliminares de MAPPO	300 s	1800 s
Comparación local y vecinal de MAPPO	300 s	3600 s
Entrenamiento y evaluación de modelos finales sintéticos	600 s	3600 s
Mendoza con programas de la red de tráfico, 12 h	No se añade	43200 s
Mendoza con programas de la red de tráfico, 2 h	No se añade	7200 s
Mendoza: fases 1 y 2, selección y entrenamiento final	300 s	7200 s
Mendoza: evaluación final	No se añade	43200 s

En las campañas de aprendizaje, el agente puede solicitar una fase cada cinco segundos. Un cambio solicitado activa cuatro segundos de amarillo; el verde debe mantenerse al menos cinco segundos antes de que pueda solicitarse otro cambio.

En las búsquedas sintéticas, la semilla de aprendizaje controla las fuentes aleatorias del algoritmo y la inicialización de sus redes; la de SUMO, la dinámica estocástica del simulador; la de rutas, la selección y generación de itinerarios; y la de TPE, la secuencia de propuestas. El código de fase 1 define la semilla de TPE como independiente de la de aprendizaje. Las semillas de entrenamiento, SUMO y rutas de fase 1 figuran en la Tabla A.9; las de fase 2, en la Tabla A.10, y las de A/C, en el apéndice A.6.4. La selección urbana y la evaluación final utilizan paneles de semillas independientes, descritos en sus respectivas subsecciones.

En los episodios de seguimiento, la política actúa sin muestrear acciones: IPPO, MAPPO y HAPPO eligen la fase permitida con mayor probabilidad; IDQN selecciona la fase permitida con mayor valor estimado. Estos registros permiten observar el comportamiento durante el entrenamiento, pero no constituyen una evaluación independiente. Su frecuencia se especifica para cada fase de búsqueda; la selección de configuraciones utiliza los criterios de la subsección 3.4.5.

3.4.2. Función objetivo de la búsqueda

La búsqueda ordena las configuraciones mediante la espera media por vehículo insertado y una penalización por sus aumentos recientes. A igualdad de espera actual, una penalización mayor produce una puntuación menos favorable. El criterio incorpora el deterioro reciente al ordenamiento; no mide por sí solo la variabilidad entre entrenamientos ni demuestra estabilidad futura.

El índice r identifica las mediciones válidas de espera w_r incorporadas al historial en los registros de cada iteración de entrenamiento. A partir de TripInfo, se agregan las esperas y las cantidades de vehículos insertados de los episodios de entrenamiento representados en ese registro, y se divide la espera agregada por las inserciones agregadas. El cociente se expresa en segundos por vehículo y pondera los episodios por su cantidad de inserciones. Solo los valores numéricos y finitos se incorporan al historial; los episodios de seguimiento descritos en la subsección 3.4.1 se registran por separado.

Para calcular la penalización se excluyen las primeras cinco mediciones válidas del historial y se utilizan hasta las diez más recientes de las restantes. Esta exclusión es distinta del calentamiento de la simulación. Las diferencias se calculan entre mediciones consecutivas del historial retenido: los descensos se sustituyen por cero y los ceros se conservan. Si quedan menos de dos mediciones, la penalización vale cero y el objetivo coincide con la espera disponible w_r.

Sea n el número de diferencias disponibles y $X_{(j)}$ el aumento situado en la posición j al ordenarlas de mayor a menor. La penalización promedia los $k = \lceil 0{,}2n \rceil$ aumentos

mayores, donde el techo redondea k hacia arriba. Esta operación finita se inspira en el valor en riesgo condicional (*Conditional Value at Risk*, CVaR), que resume una cola desfavorable, pero no estima necesariamente el CVaR de una distribución poblacional. La ecuación (3.7) define el objetivo J_r como la suma de w_r y esa penalización T_r:

$$\begin{aligned} X_r &= \text{máx}(0, w_r - w_{r-1}), \qquad k = \lceil 0{,}2n \rceil, \\ J_r &= w_r + T_r, \qquad T_r = \begin{cases} \frac{1}{k}\sum_{j=1}^{k} X_{(j)}, & n > 0, \\ 0, & n = 0. \end{cases} \end{aligned} \tag{3.7}$$

Las magnitudes w_r, X_r, T_r y J_r se expresan en segundos por vehículo. La búsqueda minimiza J_r; la recompensa del agente se calcula por separado. La longitud del historial y la fracción de aumentos retenidos son elecciones de este criterio de ordenamiento.

3.4.3. Fase 1: búsqueda amplia

La fase 1 realiza una exploración amplia en las cuatro redes. En cada escenario, los resultados de esta etapa orientan el espacio de búsqueda que se refina en la fase 2.

La fase 1 combina los cuatro algoritmos con dos arquitecturas: h128 y h256, con dos capas ocultas de 128 y 256 unidades, respectivamente. Cada combinación de algoritmo, red y arquitectura constituye un estudio; en total, se definen 32 estudios. En cada escenario se asignan 100 propuestas iniciales por estudio, ampliables hasta 150, y un máximo de 40 iteraciones por ensayo. La duración efectiva de cada ensayo depende de las detenciones anticipadas de ASHA.

En la fase 1, ASHA especifica una gracia de 10 iteraciones y un factor de reducción de 4. Cada diez iteraciones se programa un episodio de seguimiento diagnóstico. La arquitectura se mantiene fija dentro de cada estudio para evitar que RLlib altere su configuración al almacenarla. Estos resultados orientan los dominios y las arquitecturas propuestas para fase 2 en el mismo escenario.

En el espacio inicial de fase 1, los métodos basados en PPO exploran la tasa de aprendizaje, la amplitud de las actualizaciones, la estimación de ventajas, la exploración y la reutilización de experiencias. Para IDQN se incluyen además el inicio del aprendizaje, la actualización de la red objetivo y la programación de la exploración. Ambos espacios varían los niveles del codificador; en las redes sintéticas también se explora el peso de la recompensa vecinal. El descuento se fija en $\gamma = 0{,}99$ y el límite de escala del codificador de espera acumulada por carril se calibra por escenario. Las distribuciones loguniformes permiten explorar varias escalas de magnitudes positivas; los rangos completos, las alternativas y los factores fijos figuran en las Tablas A.5, A.6 y A.7 del apéndice A.5.4.

La fase 1 de Mendoza comienza después de la prueba A–E y de la comparación A/C y fija la recompensa local, con $\rho_{\text{coop}} = 0$, y la observación local con máscara de acciones. El contraste entre las formulaciones local y vecinal orienta esta elección; sus variantes modifi-

can conjuntamente observación y recompensa. Los demás dominios iniciales se conservan.

Los mecanismos incluidos en la búsqueda se apoyan en los métodos presentados en el marco teórico, y la literatura sobre HPO en aprendizaje por refuerzo fundamenta la conveniencia de explorar configuraciones y separar las semillas de búsqueda y evaluación [SWD+17, YVV+22, ELR23]. Estas fuentes fundamentan los mecanismos generales; los rangos numéricos son decisiones particulares del diseño y delimitan una exploración bajo un presupuesto finito. Los resultados se interpretan dentro de esos rangos y del presupuesto disponible.

En la implementación documentada para la búsqueda sintética, HAPPO procesa el lote completo por actor; por ello, la clave de tamaño del minilote registrada no determina una subdivisión efectiva del lote (subsección 3.3.3). Cada configuración sintética utiliza una semilla de entrenamiento por escenario: repetir episodios de evaluación de esa política no permite estimar la variabilidad entre entrenamientos independientes.

3.4.4. Fase 2: búsqueda con dominios reducidos

La fase 2 refina la búsqueda en los cuatro escenarios a partir de los resultados de fase 1 del mismo escenario. En las redes sintéticas, la búsqueda se concentra en regiones asociadas con mejores puntuaciones y conserva un margen para explorar fuera de ellas. Para cada combinación de algoritmo, escenario sintético y arquitectura, ordena por el objetivo los ensayos de fase 1 que completaron su presupuesto sin errores y con puntuaciones finitas. En parámetros numéricos, parte del intervalo de los diez mejores y lo amplía; mantiene el dominio original cuando esos ensayos están dispersos o próximos a sus límites. Las reglas para variables numéricas y categóricas, así como los dominios sintéticos resultantes, se documentan en el apéndice A.5.7.

En la búsqueda refinada sintética, esta reducción orienta una nueva exploración con recompensa local ($\rho_{\text{coop}} = 0$) y máscara de acciones; las puntuaciones de fase 1 no se incorporan como resultados de fase 2. La elección de recompensa local responde a la lectura exploratoria de que valores vecinales altos podían ser desfavorables, sin demostrar un peso óptimo ni aislar su efecto. Como también cambia la recompensa, los dominios heredados constituyen una orientación exploratoria: las configuraciones favorables en fase 1 no tienen por qué serlo bajo la señal local.

En las redes sintéticas, la fase 2 utiliza hasta 100 propuestas por combinación de algoritmo y escenario, con un máximo de 60 iteraciones por ensayo. ASHA fija 30 iteraciones de gracia y un factor de reducción de 2; el seguimiento diagnóstico se realiza cada 15 iteraciones. Las arquitecturas, las semillas y las condiciones de ejecución figuran en el apéndice A.5.6.

La fase 1 también aporta una configuración de referencia como punto inicial de la búsqueda refinada sintética: se construye con las medianas de los parámetros numéricos y las modas de los categóricos de los cinco mejores ensayos. Con estos valores se inicia un

entrenamiento desde cero bajo la señal local; no se transfieren estados de modelos ni de búsqueda. Las reglas se detallan en el apéndice A.5.7.

En Mendoza, la fase 2 conserva la observación y la recompensa locales, con $\rho_{\text{coop}} = 0$, y deriva sus dominios exclusivamente de los ensayos completos de la fase 1 de esa red. Las reglas específicas de refinamiento se presentan en el apéndice A.5.8.

La Figura 3.11 resume el paso de la exploración amplia a la búsqueda con dominios reducidos.

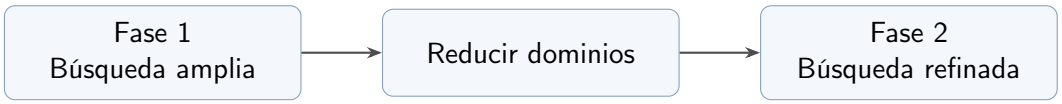


Figura 3.11: Búsqueda en dos fases para las tres redes sintéticas y Mendoza. En cada escenario, la exploración amplia orienta la reducción de sus dominios; las condiciones sintéticas y urbanas se detallan por separado en el apéndice.

3.4.5. Criterios de selección de configuraciones

La selección determina qué configuración se utiliza en el entrenamiento final desde cero. Los criterios se especifican por separado para las redes sintéticas y para Mendoza.

Redes sintéticas

Una espera baja entre los vehículos insertados debe interpretarse junto con cuántos ingresaron y cuántos completaron su viaje; por eso, la selección combina esa medida con filtros de atención a la demanda y de incidencias. El procedimiento de selección sintética se aplica a las doce combinaciones de fase 2.

La admisibilidad técnica exige 60 iteraciones, registros de métricas válidos y archivos de configuración y modelo legibles. Esta comprobación permite distinguir un candidato evaluable de un ensayo incompleto; la lista de requisitos se presenta en el apéndice A.5.6.

Para cada algoritmo y escenario se forma una lista de hasta tres candidatos técnicamente admisibles. Se priorizan los que cumplen los criterios diagnósticos de completitud y teletransportes; después se ordenan por espera diagnóstica y por el identificador de la política. Se incluye el primer candidato de cada arquitectura disponible y se completa la lista siguiendo ese orden. La Tabla A.10 indica la arquitectura prevista para cada combinación; el procedimiento no exige que ambas estén disponibles. El apéndice A.5.6 precisa la construcción de esta lista.

Los candidatos se evalúan con tres semillas SUMO comunes a los algoritmos de una misma red. El panel inicial reúne estas evaluaciones para las cuatro combinaciones de algoritmo de cada escenario. Antes de aplicar los filtros, se fija como referencia de inserciones el máximo observado en ese panel para cada red y semilla. Esta referencia empírica permite comparar el tránsito atendido por los candidatos; no mide de forma independiente la demanda total. Cada episodio debe registrar cero inicios de teletransporte, una CI de al menos 0,95 (ecuación (3.9)) y al menos el 95 % de esa referencia de inserciones.

Entre los candidatos que superan los filtros se elige el de menor espera media. Si falta evidencia, el resultado se informa como incompleto; si ningún candidato es admisible, se informa esa condición. Las semillas usadas para seleccionar no son una prueba independiente: la evaluación posterior debe conservarse como etapa separada. El candidato se interpreta como el mejor observado dentro del presupuesto, no como un óptimo global.

Mendoza

La lista urbana reúne hasta tres candidatos por algoritmo que completan el presupuesto declarado y conservan métricas finitas, una configuración íntegra y un modelo restaurable de la última iteración. Se incluye el primer candidato de cada arquitectura disponible y se completa la lista por menor espera diagnóstica e identificador. La auditoría contrasta cada configuración y modelo con su ensayo de origen, como precisa la subsección A.5.8 del apéndice.

La selección compara cada candidato con los programas semafóricos conservados en la red de Mendoza, tanto fijos como accionados (apéndice A.4). Las ejecuciones usan el escenario v14, el archivo de flujos congelado `nets/Mendoza/mendoza_12h.rou.xml` y la configuración `nets/Mendoza/mendoza_hpo_v14.sumocfg`. El control comprende $[1800, 9000)$, con 300 segundos de calentamiento desde $t = 1500$ s.

En cada comparación se mantienen la misma semilla de SUMO, demanda, calentamiento y ventana. La referencia conserva el funcionamiento de sus programas; las políticas actúan con la inicialización descrita en la subsección 3.3.1.

Se emplea un panel común de cinco semillas de SUMO para todos los candidatos urbanos y la referencia. Las semillas se generan aleatoriamente, se registran antes de la selección y son distintas de las usadas en la búsqueda y en la evaluación final. Con hasta tres candidatos por algoritmo, el panel comprende como máximo 65 simulaciones: cinco de referencia y sesenta de candidatos. Este tamaño acota el coste de la selección; es una decisión operativa, no una garantía de potencia estadística ni de estabilidad del ordenamiento.

Para cada semilla, las inserciones cuentan los vehículos cuyo instante efectivo de inserción pertenece a $[1800, 9000)$; la CI divide los vehículos de esa cohorte que completan el viaje antes del cierre por el total insertado. Se contabilizan también los inicios de teletransporte durante el control.

Una configuración es admisible solo si, en cada una de las cinco semillas, iguala o supera al programa de referencia en cantidad de vehículos insertados y en CI, y no registra más inicios de teletransporte. Los criterios se fijan en relación con el programa de referencia, sin umbrales absolutos de inserción o completitud.

La clasificación requiere simulaciones que alcancen el fin de la ventana, los tres registros para cada comparación y un denominador positivo para calcular la tasa y la espera; si alguno falta, no puede determinarse la admisibilidad del candidato.

Entre las configuraciones admisibles se elige la de menor media aritmética de las cinco esperas medias por simulación, calculadas sobre los vehículos insertados en el intervalo y ponderando por igual cada semilla. Ante un empate exacto, se prioriza la mayor media de las cinco CI, después el menor número medio de inicios de teletransporte y, finalmente, el identificador de la configuración. Si ninguna configuración satisface los criterios en las cinco semillas, no se designa una configuración urbana admisible para esa combinación.

3.5. Comparación de variantes locales y vecinales de MAPPO

Esta línea examina si una formulación que incorpora información de los vecinos y una contribución ponderada de su señal de recompensa mejora el desempeño respecto de la formulación local. La campaña comparó A y C de MAPPO con una configuración de aprendizaje transferida y fija. A utiliza observación y recompensa locales ($\rho_{\text{coop}} = 0$); C añade a la observación la espera media por carril de los vecinos y combina la recompensa local con una señal vecinal normalizada ($\rho_{\text{coop}} = 0{,}1$). Como C modifica tanto la observación como la recompensa, el contraste evalúa conjuntamente esos cambios y no permite aislar el efecto de cada uno.

El procedimiento comprende una prueba preliminar de cinco formulaciones locales y vecinales de observación y recompensa, A–E, y una comparación ampliada de las formulaciones local A y vecinal C en las tres redes sintéticas, ambas realizadas con MAPPO. La primera permite examinar alternativas; la segunda contrasta las dos variantes bajo condiciones comunes de entrenamiento y evaluación. Su alcance se restringe a MAPPO y no se generaliza a HAPPO, IPPO o IDQN.

3.5.1. Prueba preliminar de formulaciones

Se examinaron cinco formulaciones en la red $4 \times 4_H$, con MAPPO de arquitectura h128 y tres semillas de entrenamiento, durante veinte iteraciones por política. Cada episodio duró 1800 segundos, después de 300 segundos de calentamiento. Se aplicaron las mismas semillas a las cinco formulaciones; las repeticiones en el entorno local y en el clúster de cómputo Toko (grupo de servidores que ejecuta trabajos en paralelo) no se contaron como semillas independientes adicionales.

La Tabla 3.6 identifica las formulaciones como A–E. B, C y D fijaron $\rho_{\text{coop}} = 0{,}1$: la prueba comparó formulaciones y no realizó un barrido del coeficiente. C y D comparten recompensa y difieren en la observación, mientras que E examina otra recompensa local. La campaña multiescenario documentada se limitó a A y C. No se dispone de un criterio previo documentado para elegir C entre las variantes vecinales; por ello, esa elección no establece que C sea la mejor formulación vecinal.

Tabla 3.6: Variantes examinadas en la prueba preliminar.

Variante	Observación	Recompensa
A	Local: fase activa y señales de carriles propios.	Diferencia local de espera acumulada.
B	Local y suma de espera acumulada de cada vecino.	Diferencia local y vecinal sin normalización por carriles; $\rho_{\text{coop}} = 0{,}1$.
C	Local y vecinal: espera media por carril de cada vecino.	Diferencia local y vecinal normalizada por carriles; $\rho_{\text{coop}} = 0{,}1$.
D	Local, suma de espera vecinal y logaritmo natural de uno más el número de carriles del vecino.	Igual que C.
E	Local: fase activa y señales de carriles propios.	Negativo de la diferencia absoluta entre el número de vehículos detenidos en carriles de entrada y el de los carriles de salida; esta señal se denomina recompensa de presión local.

Un resultado desfavorable no se clasificó como fallo técnico si la ejecución se completó correctamente; las comprobaciones se describen en el apéndice A.5.2. La prueba preliminar no permite atribuir las diferencias a la cooperación vecinal, porque E también modifica la recompensa local.

3.5.2. Entrenamiento de las variantes en las tres redes sintéticas

La comparación ampliada utilizó MAPPO con arquitectura h128 y los hiperparámetros de una configuración histórica de $4\times4_H$, documentada en el apéndice A.6.3. Se aplicaron estos valores a las tres redes sin ajustes por variante o escenario, para mantener una configuración de aprendizaje común.

C incorpora una coordenada codificada por cada vecino directo; por eso, su primera capa tiene más pesos que la de A. Esta diferencia de capacidad forma parte de la comparación, que también modifica observación y recompensa, como se indicó arriba. El contraste caracteriza las variantes bajo esa configuración transferida, sin establecer qué desempeño alcanzarían con búsquedas propias. Las observaciones y las recompensas de ambas variantes se definen en la subsección 3.3.2.

Cada par de políticas reúne una política local y una vecinal entrenadas con la misma semilla. Se entrenaron 24 pares por red para examinar la variación del contraste entre entrenamientos. La planificación de este tamaño, basada en la dispersión histórica de las diferencias, se detalla en el apéndice A.6.3; es una previsión, no una garantía de precisión.

Cada política se entrenó durante 40 iteraciones y se evaluó con el modelo de la última, elegido de antemano (subsección 3.6.3).

3.6. Evaluación y análisis de políticas

Una vez obtenidas las políticas, la evaluación examina su desempeño con parámetros fijos durante cada episodio. La comparación de los cuatro algoritmos resume descriptivamente la variación entre simulaciones de cada modelo final. La comparación A/C utiliza pares de políticas entrenadas para estimar la incertidumbre del contraste entre las formulaciones local y vecinal de MAPPO. Ambos análisis consideran la espera y la atención a la demanda, con las unidades de observación que se precisan a continuación.

3.6.1. Métricas de desempeño y vehículos evaluados

Las métricas de evaluación se organizan en dos escalas: medidas globales a nivel de red, basadas en los registros de viaje, y medidas locales por agente, calculadas sobre las intersecciones que controlan.

Métricas globales de viaje y atención a la demanda

A nivel de red, la población evaluada es la cohorte $\mathcal{V}_{\text{ins}}$, que reúne los vehículos cuyo instante efectivo de inserción (`depart`) pertenece a la ventana de control $[t_{\text{inicio}}, t_{\text{fin}})$; t_{inicio} y t_{fin} son, respectivamente, la apertura y el cierre, en segundos de simulación. La cohorte puede variar entre políticas aun con la misma demanda programada.

Los vehículos que ingresan cerca del cierre disponen de menor tiempo observable. Para los viajes inconclusos, la espera registrada es parcial y no permite conocer la espera total que tendrían al completar el recorrido. En las evaluaciones de políticas, estos viajes se registran mediante la opción `-tripinfo-output.write-unfinished` de SUMO [Ecl26c].

El retraso previo al ingreso se registra por separado en `departDelay` y no computa en la espera de viaje. Con esta población se definen:

- **Espera media observada hasta el cierre ($\bar{w}_{\text{ins}}$):** para cada vehículo v de $\mathcal{V}_{\text{ins}}$, w_v denota el valor de `waitingTime`, que registra los segundos a velocidad $\leq 0{,}1$ m/s, excluyendo detenciones programadas. La espera media se obtiene al promediar estos valores:

$$\bar{w}_{\text{ins}} = \frac{1}{|\mathcal{V}_{\text{ins}}|} \sum_{v \in \mathcal{V}_{\text{ins}}} w_v, \qquad w_v = \texttt{waitingTime}_v. \tag{3.8}$$

  La medida se expresa en segundos por vehículo; si la cohorte está vacía, se mantiene como indefinida.

- **Completitud e inserción de la demanda:** la cantidad de vehículos insertados es el tamaño de $\mathcal{V}_{\text{ins}}$. La tasa de completitud sobre insertados (CI) mide la fracción de vehículos de $\mathcal{V}_{\text{ins}}$ que logra completar su recorrido antes del cierre t_{fin}:

$$\text{CI} = \frac{|\{v \in \mathcal{V}_{\text{ins}} : \text{completó su viaje antes de } t_{\text{fin}}\}|}{|\mathcal{V}_{\text{ins}}|}. \tag{3.9}$$

  Si $|\mathcal{V}_{\text{ins}}| = 0$, CI queda indefinida. La medida responde qué proporción de quienes ingresaron logró terminar su viaje; una tasa alta puede coexistir con un bajo volumen de inserciones si la congestión retiene vehículos fuera de la red.

- **Pérdida temporal media ($\bar{\ell}$):** media de `timeLoss` en segundos por vehículo sobre $\mathcal{V}_{\text{ins}}$. Estima la diferencia respecto del tiempo de viaje ideal según la velocidad deseada del vehículo y el límite vial; excluye las detenciones programadas [Ecl26c].

Para expresar las tasas reportadas en la comprobación operacional de Mendoza (subsección 3.2.3), se usan los conteos agregados de cada horizonte: N_{cargados} es el número de vehículos incluidos en la demanda cargada, $N_{\text{insertados}}$ cuenta los que ingresaron a la red y $N_{\text{completados}}$ los que llegaron a destino dentro del horizonte. Con estos conteos se calculan las siguientes tasas, que quedan indefinidas cuando su denominador es cero:

$$\text{CR}_{\text{ventana}} = \frac{N_{\text{completados}}}{N_{\text{cargados}}}, \qquad \text{IR} = \frac{N_{\text{insertados}}}{N_{\text{cargados}}}, \qquad \text{CI}_{\text{operacional}} = \frac{N_{\text{completados}}}{N_{\text{insertados}}}. \tag{3.10}$$

La identidad $\text{CR}_{\text{ventana}} = \text{IR} \cdot \text{CI}_{\text{operacional}}$ se cumple algebraicamente si los conteos corresponden a conjuntos anidados y sus denominadores son positivos; por sí sola no demuestra que los registros compartan una misma cohorte ni identifica el campo nativo de SUMO del que provino cada conteo. El apéndice A.4.1 resume las limitaciones de esa reconstrucción.

Métricas locales por controlador y equidad espacial

A nivel individual, la señal local proviene de `getAccumulatedWaitingTime` y de una memoria temporal finita en los carriles que controla cada agente, por lo que no equivale a la espera por viaje de `TripInfo`. Se resumen dos aspectos con ponderaciones distintas:

- **Cociente local acumulado por observación de vehículo:** para cada agente, se divide la suma de esperas reportadas durante el control por la suma de vehículos presentes en sus pasos. Un vehículo puede contribuir en varios pasos y se incluyen los presentes al comenzar el control; el denominador cuenta observaciones de vehículos, no viajes distintos. Los pasos con mayor presencia vehicular pesan más en el cociente, que se expresa en segundos. El agregado de red divide la suma de numeradores por la suma de denominadores de todos los agentes. Si no hay observaciones de vehículos, se informa cero junto con el conteo nulo.

- **Desigualdad espacial de espera (coeficiente de Gini, $G_{\mathbf{wait}}$):** para cada agente i y paso t, la espera acumulada $W_i^{(t)}$ de sus carriles se divide por el número $n_i^{(t)}$ de vehículos presentes; si $n_i^{(t)} = 0$, se asigna cero. La media temporal $\bar{w}_i$ de estos cocientes da el mismo peso a cada paso, cualquiera que sea la cantidad de vehículos presentes. El coeficiente de Gini se calcula entre agentes —cada uno corresponde a un controlador— a partir de esas medias. Sean N el número de agentes y $\bar{w}_{(i)}$ las medias temporales ordenadas de menor a mayor:

$$G_{\text{wait}} = \frac{2\sum_{i=1}^{N} i \cdot \bar{w}_{(i)} - (N+1)\sum_{i=1}^{N} \bar{w}_{(i)}}{N\sum_{i=1}^{N} \bar{w}_{(i)}}. \tag{3.11}$$

  El coeficiente es adimensional. Si la suma de las medias es cero, todas son cero y la implementación informa Gini cero; si la suma es positiva, Gini cero indica igualdad entre controladores, no ausencia de espera. Al dar igual peso a cruces con demandas desiguales, el índice también refleja disparidades de carga estructural de la red.

Los teletransportes se utilizan como indicador de incidencias operacionales severas. En la selección de candidatos, el análisis A/C y la evaluación final se cuentan sus inicios mediante TraCI durante la ventana de control; un vehículo puede contribuir con más de un evento. La salida Summary de SUMO dispone de un contador nativo `teleports`. Estos registros no se sustituyen entre sí sin comprobar su correspondencia.

3.6.2. Entrenamiento y evaluación de los modelos finales

Se entrena desde cero un modelo final para cada combinación de algoritmo y escenario que cuente con una configuración seleccionada. A cada combinación se asigna una semilla de aprendizaje nueva, separada de las fases de búsqueda, selección y evaluación. En Mendoza también se asigna una semilla nueva al simulador de entrenamiento; ambas se conservan en la receta junto con los hiperparámetros, la arquitectura y los archivos del entorno. En las redes sintéticas, el diseño conserva el calentamiento y la duración del control de la búsqueda refinada (Tabla 3.5).

El procedimiento fija para el entrenamiento final el presupuesto completo de iteraciones de la búsqueda que produjo la configuración seleccionada y toma el punto de control (*checkpoint*) de la última iteración.

El protocolo contempla veinte simulaciones por cada combinación de algoritmo y escenario. Para las dieciséis combinaciones previstas, esto representa 320 simulaciones si todas cuentan con una configuración seleccionada. Cada resumen estadístico requiere veinte observaciones válidas por métrica.

Las semillas aleatorias de SUMO se registran antes de cada evaluación y son distintas de las usadas en las etapas anteriores; no se fija un panel común entre algoritmos. En cada corrida se reconstruye el entorno con la semilla correspondiente y se comprueba que sea la

utilizada por SUMO. Los parámetros de las políticas permanecen fijos y estas actúan sin exploración. El calentamiento se aplica cuando lo establece la configuración del escenario.

En Mendoza, la configuración de búsqueda, selección y entrenamiento final establece la ventana de dos horas del escenario v14: la simulación comienza en $t = 1500$ s, incluye 300 s de calentamiento y el control se evalúa entre $t = 1800$ y $t = 9000$ s (06:30–08:30), según `nets/Mendoza/mendoza_hpo_v14.sumocfg`.

La configuración de evaluación final establece una jornada completa, de $t = 0$ a $t = 43200$ s (06:00–18:00), sin calentamiento, según `nets/Mendoza/mendoza_evaluation_v14.sumocfg`. Esta separación busca examinar el modelo final en un horizonte diario que incluye condiciones horarias distintas de las del recorte matutino.

Para las evaluaciones urbanas se utiliza el archivo congelado `nets/Mendoza/mendoza_12h.rou.xml` (apéndice A.4.1). El panel urbano de selección se especifica en la subsección 3.4.5; las veinte semillas de evaluación final excluyen la semilla 104729, reservada para la evaluación diaria canónica.

El protocolo requiere registrar por simulación las inserciones, la completitud de la cohorte insertada y los teletransportes. Como el modelo final puede diferir del comportamiento observado durante la selección, estos diagnósticos se informan de manera independiente y no operan como filtros retroactivos.

Las medidas de viajes con denominador nulo se mantienen como indefinidas; si falta una de las veinte observaciones válidas de una métrica, no se calcula su resumen para esa combinación. Un cierre anticipado se conserva en el registro y no cuenta como un horizonte completo. Las repeticiones por un fallo técnico conservan el intento original y la semilla asignada.

Para cada métrica, la media da el mismo peso a las veinte simulaciones y la desviación estándar muestral utiliza el divisor $20 - 1 = 19$. El resumen se calcula por separado para cada agente y para cada medida global de red; las métricas globales de viajes se conservan diferenciadas de la señal local por agente.

La dispersión describe la variación entre simulaciones del modelo evaluado; no estima la variabilidad entre entrenamientos independientes [ASC+21]. Por ello, la comparación descriptiva caracteriza los modelos obtenidos bajo este procedimiento y no establece una superioridad general de los algoritmos.

3.6.3. Evaluación de variantes de MAPPO

Cada política de la comparación A/C se evaluó cinco veces con el modelo de la última iteración y las semillas 71001–71005, comunes a ambas variantes y distintas de las de entrenamiento. La campaña reunió 72 pares, 144 políticas y 720 episodios; las configuraciones y semillas se conservan en el apéndice A.6.3. Los parámetros de cada modelo permanecieron fijos y, en cada decisión, el agente seleccionó la fase permitida con mayor probabilidad.

Para cada evaluación se reconstruyó el entorno con la semilla correspondiente, se restauró el estado posterior al calentamiento y se reiniciaron los contadores antes del control. La ventana de análisis fue $[t_{\text{inicio}}, t_{\text{fin}}) = [300, 3900)$ segundos de simulación. Para calcular CI, se excluyeron de numerador y denominador los vehículos presentes al terminar el calentamiento. `TripInfo` incluyó los viajes según su instante real de inserción; los que seguían inconclusos al cierre permanecieron en el denominador, pero no en el numerador.

3.6.4. Análisis por pares y criterio de decisión

El análisis utiliza los pares de políticas y las evaluaciones descritos en las subsecciones 3.5.2 y 3.6.3. Para emitir una decisión se requieren los 24 pares completos de una red; si faltan pares, el análisis se informa como incompleto. Primero se promedian las cinco evaluaciones de cada política, realizadas con semillas SUMO comunes a ambas variantes.

Sea $w_{s,b,q}$ la espera media por vehículo insertado, en segundos por vehículo, para el par de entrenamiento $s \in \{1, \ldots, 24\}$, la variante $b \in \{A, C\}$ y la evaluación $q \in \{1, \ldots, 5\}$. La ecuación (3.12) reúne la media por política $\bar{w}_{s,b}$, la diferencia absoluta d_s de C menos A y la diferencia relativa z_s respecto de A:

$$\begin{aligned} \bar{w}_{s,b} &= \frac{1}{5} \sum_{q=1}^{5} w_{s,b,q}, \\ d_s = \bar{w}_{s,C} - \bar{w}_{s,A}, &\qquad z_s = \frac{\bar{w}_{s,C} - \bar{w}_{s,A}}{\bar{w}_{s,A}}. \end{aligned} \tag{3.12}$$

La diferencia d_s se expresa en segundos por vehículo y z_s es adimensional; un valor negativo favorece C. Los efectos reportados son las medias entre pares, $\bar{d}$ y $\bar{z}$; $100\bar{z}$ expresa el cambio porcentual medio, no el cociente entre medias globales. La diferencia relativa no está definida cuando $\bar{w}_{s,A} = 0$; si esto ocurre en alguno de los 24 pares requeridos, no es posible emitir la decisión basada en dicho efecto. El análisis trata cada par de políticas entrenadas como una observación independiente; las cinco evaluaciones de una misma política no se cuentan como entrenamientos independientes [ASC+21].

La incertidumbre se estima mediante 20.000 remuestreos con reemplazo (*bootstrap*) de los 24 pares completos de cada red. Cada remuestreo conserva juntas las dos variantes del par; el efecto se recalcula en cada conjunto.

La Figura 3.12 resume la unidad de pareo y el recorrido desde las evaluaciones hasta el remuestreo.

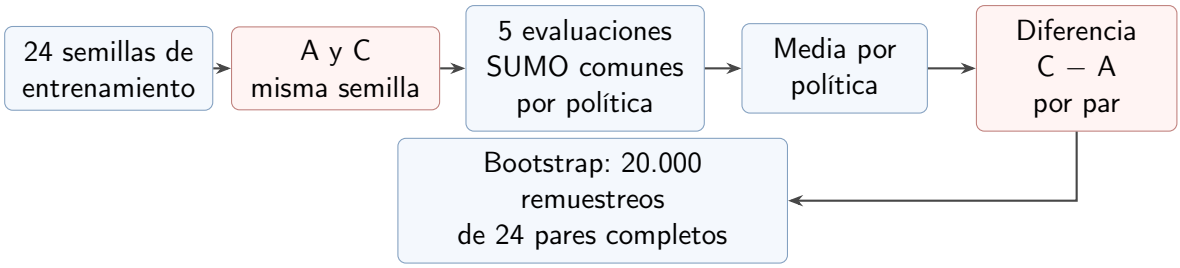


Figura 3.12: Análisis por pares: A y C comparten la semilla de entrenamiento y las cinco semillas de evaluación; el remuestreo conserva cada par completo.

Se informan intervalos percentiles del 95 % y del 98,33 %. Este último aplica la corrección de Bonferroni a los tres contrastes A/C, uno por cada red, para cada medida por separado; no ajusta simultáneamente todas las métricas.

También se calcula un intervalo basado en la distribución t de Student como análisis de sensibilidad. Ambos son aproximaciones con 24 pares y su cobertura no está garantizada; se conservan los valores extremos y se revisa su influencia. El cálculo de los percentiles y del ajuste se detalla en el apéndice A.5.9.

Para CI, se promedian las cinco evaluaciones de cada política. En cada par s, la diferencia es $d_{\mathrm{CI},s} = \overline{\mathrm{CI}}_{s,C} - \overline{\mathrm{CI}}_{s,A}$; $\bar{d}_{\mathrm{CI}}$ denota la media de las 24 diferencias pareadas y su intervalo se obtiene con el remuestreo descrito arriba.

El criterio exige una reducción práctica de la espera, admite una disminución acotada de la tasa de completitud (CI) y no permite más inicios de teletransporte que A. Una decisión favorable a C requiere cumplir conjuntamente las siguientes condiciones:

- **Espera:** el límite superior del intervalo del 98,33 % para el efecto relativo debe ser menor que $-0,05$.
- **CI:** el límite inferior del intervalo del 98,33 % para la diferencia media pareada $\bar{d}_{\mathrm{CI}}$ debe superar $-0,005$, equivalente a una disminución de 0,5 puntos porcentuales en la tasa (ecuación (3.9)).
- **Inicios de teletransporte:** el conteo de C no debe superar al de A en ninguna evaluación con la misma semilla dentro de un par. Se utiliza `teleport_starts`, que acumula los vehículos que comienzan un teletransporte en cada paso de simulación, mediante `getStartingTeleportIDList`; cuenta eventos, no vehículos distintos. Si falta el contador de alguna variante, esta condición no puede evaluarse y no se emite una decisión favorable.

Los márgenes del 5 % y de 0,5 puntos porcentuales son decisiones prácticas de este análisis, no umbrales universales. El intervalo del 95 % se informa para describir la incertidumbre y examinar cómo cambia la interpretación al utilizar otro nivel de confianza.

El criterio de A/C no impone un umbral de inserciones ni evalúa el retraso previo al ingreso; la condición sobre la CI se refiere solo a los vehículos insertados. Conservar

esa proporción no asegura conservar la cantidad de vehículos atendidos. Una decisión favorable aporta evidencia de una reducción práctica de la espera observada entre los vehículos insertados, bajo las condiciones evaluadas; para interpretarla como una mejora integral de la atención a la demanda deben examinarse también las inserciones y los retrasos de ingreso disponibles.

Los intervalos describen la variación entre entrenamientos, condicionada a las cinco semillas de evaluación compartidas, y no cubren toda demanda posible. La ausencia de evidencia de una reducción práctica no implica equivalencia estadística ni operacional. El cierre temporal de los viajes inconclusos se interpreta según la subsección 3.6.1.

El análisis de esta tesina considera por separado las tres redes: 24 pares de 3×1, 24 de 3×3 y 24 de $4 \times 4_H$; no combina los 72 pares en una única estimación. El informe histórico presentó 3×1 y $4 \times 4_H$ como análisis principal y separó 3×3 después de revisar los resultados. Esta decisión de presentación no estaba fijada de antemano y se distingue aquí del alcance de tres redes adoptado en esta tesina; el antecedente se identifica en el apéndice A.5.9.

3.7. Reproducibilidad y alcance

La reproducción se apoya en los archivos de red y rutas, las configuraciones de SUMO, las semillas, las arquitecturas y parámetros efectivos, las versiones de código y los estados de los modelos (*checkpoints*). Los manifiestos relacionan cada ejecución con sus productos y los códigos de verificación permiten comprobar la identidad de los archivos. Las fuentes y valores se reúnen en el apéndice A.5. En las comprobaciones históricas de Mendoza, la ausencia de las rutas originales limita esta reconstrucción, como se explica en la subsección 3.2.3.

3.7.1. Entorno de cómputo y envío de trabajos

Las búsquedas documentadas de fase 1 y la comparación entre las variantes local y vecinal se ejecutaron en el clúster de cómputo Toko. La prueba preliminar se realizó en un entorno local y en Toko; las exploraciones anteriores en el clúster Mendieta no forman parte de los análisis presentados.

Los trabajos se organizaron por combinación experimental y conservaron sus configuraciones y productos en almacenamiento persistente. Los programas de envío y el traslado de registros durante la ejecución se describen en el apéndice A.5.1. Esos archivos registran recursos solicitados, pero no permiten identificar los recursos físicos efectivamente asignados ni el modelo de unidad de procesamiento gráfico (*Graphics Processing Unit*, GPU) de cada trabajo.

3.7.2. Limitaciones del diseño y alcance de la inferencia

El contraste A/C conserva el alcance y la unidad de inferencia establecidos en las secciones 3.5 y 3.6.4. La planificación de su tamaño se basó en una diferencia absoluta y un intervalo del 95 %, mientras que el criterio de decisión utiliza el efecto relativo y un intervalo del 98,33 %; por tanto, aquella previsión no garantiza por sí sola la precisión de este último criterio.

La búsqueda adaptativa explora regiones distintas y consume presupuestos efectivos desiguales; igualar iteraciones no iguala pasos de gradiente ni costo computacional entre algoritmos. Las arquitecturas seleccionadas tampoco permiten atribuir causalmente sus diferencias de desempeño.

El marco HAML ofrece garantías para HAPPO bajo condiciones específicas, entre ellas una recompensa compartida. Como el entrenamiento aquí emplea recompensas individuales, esas garantías no se trasladan directamente a la formulación experimental; el apéndice A.6 precisa su alcance [ZKF+24].

La comprobación de Mendoza se interpreta con los supuestos y límites de procedencia de la subsección 3.2.3. Las conclusiones de ambas líneas experimentales se restringen a las condiciones simuladas, las poblaciones evaluadas y los presupuestos descritos.

4

Análisis y discusión de resultados

Este capítulo organiza los resultados por procedimiento: búsqueda de hiperparámetros, evaluación de los modelos finales, comparación pareada A/C de MAPPO y comprobación operacional de Mendoza. Sus métricas y criterios se interpretan según las unidades experimentales definidas en la metodología.

La comprobación de Mendoza corresponde a la red urbana con los programas semafóricos incluidos en ella. Sus conteos y tasas describen esa operación y no se interpretan como una evaluación de políticas aprendidas.

4.1. Resultados del ajuste de hiperparámetros

La presentación de HPO sigue las dos fases de búsqueda en las tres redes sintéticas y en Mendoza. Los dominios y criterios se interpretan según el escenario al que corresponden; la comparación A/C y la comprobación operacional urbana se presentan como procedimientos distintos de la búsqueda de hiperparámetros.

4.1.1. Estado de la consolidación y de las búsquedas

4.1.2. Resultados del ajuste y análisis de sensibilidad

4.1.3. Selección de configuraciones de HPO

4.2. Evaluación de políticas y comprobación de escenarios

La comparación A/C se presenta por separado de la selección de configuraciones mediante HPO. Esta tesina analiza las tres redes sintéticas por separado y conserva los pares de entrenamiento de cada una. El informe histórico separó la red 3×3 del análisis principal después de observar los resultados; ese antecedente se distingue del análisis adoptado aquí, como se explica en la sección 3.6.4.

4.2.1. Criterios de evaluación y métricas de comparación

4.2.2. Desempeño en escenarios sintéticos: 3×1, 3×3, 4×4 heterogéneo

4.2.3. Comprobación operacional del escenario de la Ciudad de Mendoza

Se comprobó el funcionamiento de la red modelada de la Ciudad de Mendoza en los dos horizontes definidos en la sección 3.2.3. La Tabla 4.1 presenta conteos y tasas de las corridas con programas semafóricos, que caracterizan la operación de la red. La fila de dos horas corresponde al intervalo de control de 06:30 a 08:30, sin calentamiento. La configuración MARL `mendoza_marl_2h.sumocfg` comienza a las 06:25 e incluye 300 segundos de calentamiento antes de ese intervalo. Las fuentes y límites de procedencia se detallan en la sección A.4.1.

Tabla 4.1: Métricas operacionales del escenario urbano de la Ciudad de Mendoza con los programas semafóricos de la red.

Ventana temporal	Cargados	Insertados	Completados	CR	IR	CI	Teletransportes
2 horas (06 : 30–08 : 30)	30.629	23.735	22.702	0,7412	0,7749	0,9565	0
12 horas (06 : 00–18 : 00)	180.163	143.819	143.022	0,7938	0,7983	0,9945	0

Los registros primarios, sus ventanas y la versión de SUMO se documentan en el apéndice A.4.1; los campos de carga, inserción y estadísticas sustentan los conteos de la tabla.

Los logs revisados no contienen un contador explícito de teletransportes: los ceros de la tabla están informados en la documentación del escenario, no extraídos de un contador primario conservado. Tampoco se conservaron los archivos de rutas originales para

confirmar que el archivo congelado descrito en el apéndice fuera exactamente la entrada consumida por estas dos corridas; los conteos describen las operaciones registradas y no validan por sí solos la construcción completa de la demanda.

4.3. Discusión e interpretación de resultados

En esta sección se integran los resultados presentados y se discuten sus principales implicancias en relación con los objetivos del trabajo.

5

Conclusiones

Este capítulo sintetiza los hallazgos del estudio, relaciona la evidencia experimental con los objetivos planteados y delimita el alcance de las conclusiones. También identifica las limitaciones del modelo y propone líneas de trabajo futuro. Las afirmaciones finales deben interpretarse dentro de los escenarios, métricas y protocolos descritos en la metodología.

A

Arquitectura de software y reproducción experimental

Este apéndice describe el diseño orientado a objetos del paquete de software `marlTLC` mediante diagramas de clases UML y presenta los patrones previstos para separar la simulación, el aprendizaje por refuerzo y el cálculo de recompensas. Los diagramas representan el diseño inicial y no una correspondencia exhaustiva entre clases y archivos ejecutables. Los apartados finales reúnen parámetros, dominios y semillas necesarios para reproducir los procedimientos descritos.

A.1. Diseño orientado a objetos del entorno de simulación

El diseño del entorno separa las responsabilidades de sus componentes. En la especificación, la clase abstracta `BaseSumoEnvironment` encapsula la inicialización de SUMO, el control temporal y la interacción de bajo nivel con la API `libsumo` / `traci`. A partir de ella, `SumoRllibEnv` se define como adaptador de la simulación a la interfaz `MultiAgentEnv` de Ray RLlib.

La Tabla A.1 resume las responsabilidades de las herramientas que intervienen en el flujo de información.

Tabla A.1: Herramientas y responsabilidades en el sistema experimental.

Herramienta	Función
SUMO y libsumo	Simulación microscópica del tránsito.
`SumoRllibEnv`	Adaptación de la simulación a la interfaz multiagente.
Ray RLlib	Organización de los procesos de muestreo y aprendizaje.
PyTorch	Cálculo de redes neuronales y sus actualizaciones.

La interfaz distingue dos indicadores de cierre: la terminación señala que se alcanzó una condición final del entorno, y el truncamiento indica una interrupción impuesta por un límite externo, como la duración máxima del episodio. En la versión conservada para la comparación entre variantes, alcanzar el límite temporal activa ambos indicadores.

La Figura A.1 presenta las entidades del entorno y sus relaciones con los agentes semafóricos. En un diagrama de clases, cada recuadro contiene el nombre de la clase, sus atributos y sus operaciones. El signo "−" identifica miembros privados, "+" públicos y "#" protegidos, disponibles en la clase y sus derivadas. El texto después de los dos puntos indica el tipo de dato. La cursiva señala una clase u operación abstracta, que debe concretarse en otra clase. La flecha "hereda de" apunta desde la clase especializada hacia la general; la multiplicidad $1..*$ indica "uno o más" objetos relacionados.

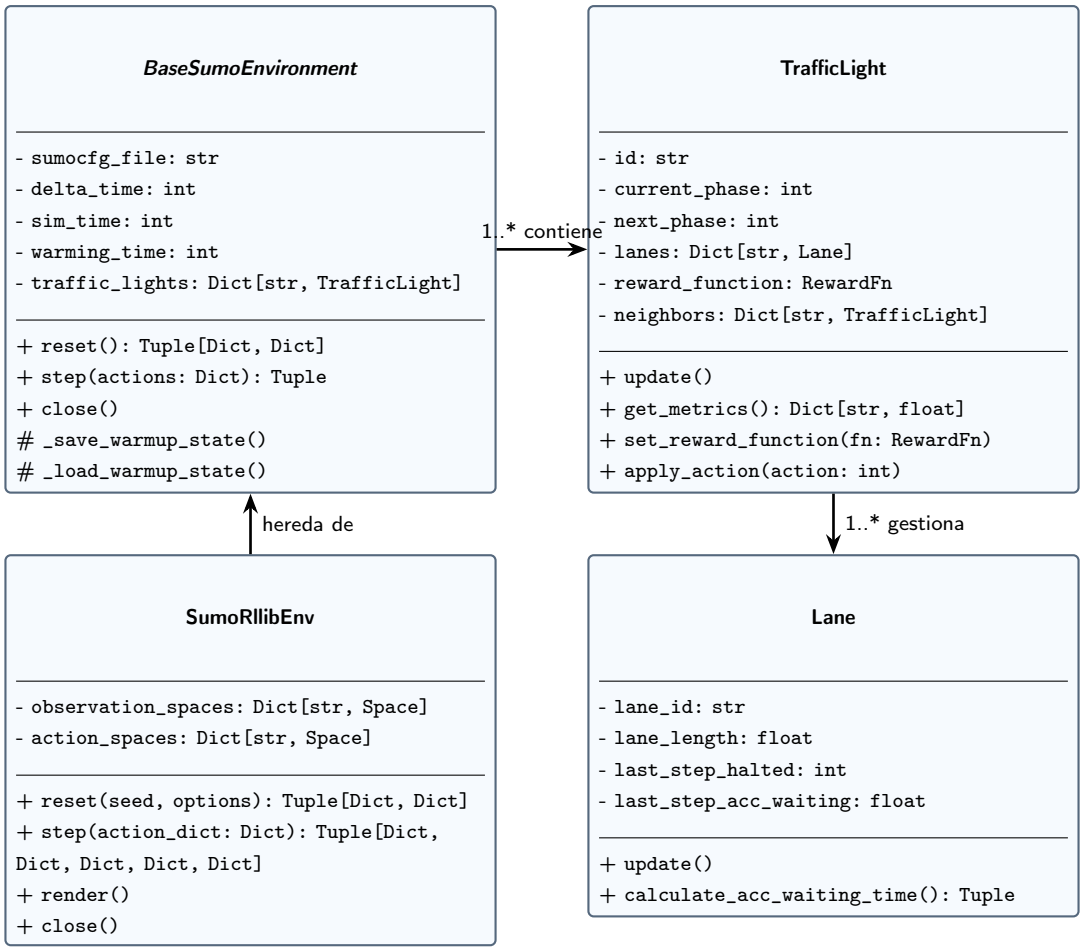


Figura A.1: Diagrama de clases UML del entorno. `SumoRllibEnv` adapta la clase abstracta `BaseSumoEnvironment`; el entorno contiene uno o más objetos `TrafficLight`, y cada uno gestiona objetos `Lane`. En cada clase se muestran sus principales atributos y métodos.

La Figura A.1 se organiza desde la abstracción general hacia las entidades de la red. En la columna izquierda, `BaseSumoEnvironment` concentra el archivo de configuración, los parámetros temporales, los semáforos y las operaciones generales de reinicio, avance y cierre. Debajo, `SumoRllibEnv` especializa esa clase para recibir un diccionario de acciones de los agentes y devolver las observaciones, recompensas y estados definidos por la interfaz de Ray RLlib. A la derecha, el diseño asigna a `TrafficLight` la fase actual, la siguiente fase, sus carriles, su función de recompensa y sus vecinos; también le asigna la actualización de métricas y la aplicación de acciones. Finalmente, `Lane` representa la longitud y las medidas recientes de detención y espera. Las flechas muestran las dependencias previstas entre la interfaz de aprendizaje, el entorno, los semáforos y sus carriles.

A.2. Jerarquía de funciones de recompensa y patrón *Strategy*

En la especificación, el cálculo de la recompensa se desacopla mediante el patrón ***Strategy***. Esta organización permite sustituir la métrica de optimización sin modificar el

bucle principal de simulación.

La clase base abstracta `RewardFn` define la interfaz común `compute_reward(tl_id)`. En el diseño se contemplan formulaciones locales basadas en tres señales: demora acumulada, longitud de cola y presión de red. La coordinación distribuida se representa mediante el patrón ***Composite***: una clase de recompensa compuesta combina la señal local y la señal promedio de los semáforos adyacentes.

La Figura A.2 detalla las clases que intervienen en la recompensa cooperativa. Las demás estrategias locales no se dibujan para conservar la legibilidad. Se mantienen las convenciones de la Figura A.1; en este segundo diagrama, la flecha discontinua "compone" indica que una clase usa otra en su cálculo, no que herede sus atributos y métodos.

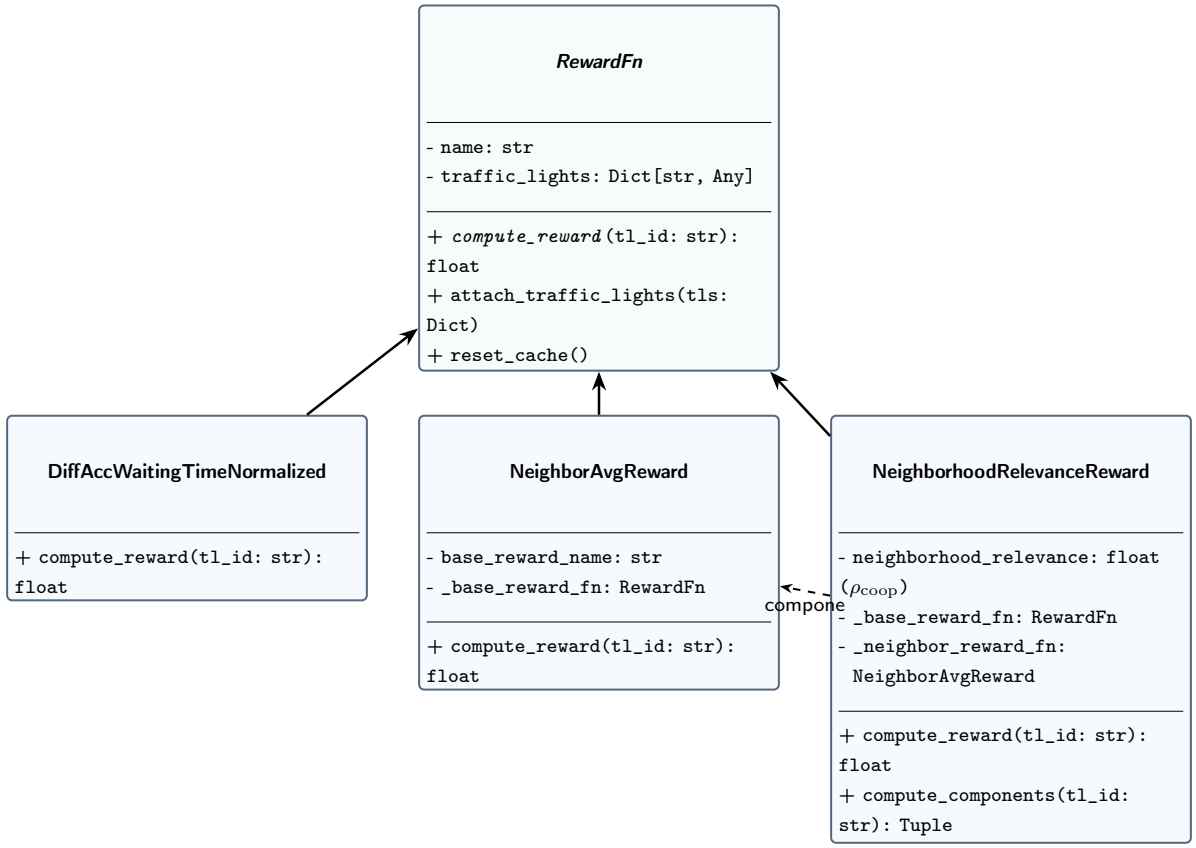


Figura A.2: Especificación UML del módulo de recompensas. Las clases inferiores realizan la interfaz `RewardFn`; la clase compuesta incorpora la señal promedio de los agentes vecinos.

La Figura A.2 sitúa la clase abstracta `RewardFn` en la parte superior; esta define la operación abstracta `compute_reward`, que recibe un identificador de semáforo y devuelve un valor numérico. Las flechas indican las clases que implementan variantes de esa operación. `DiffAccWaitingTimeNormalized` representa una señal local normalizada a partir de la variación de la espera acumulada; `NeighborAvgReward` promedia la señal base de los vecinos; y `NeighborhoodRelevanceReward` combina ambas según ρ_{coop}. La flecha discontinua indica la composición entre las dos últimas clases. Esta separación permite sustituir

la formulación de la recompensa sin cambiar el bucle de simulación.

A.3. Patrones de diseño de la arquitectura

A continuación se resumen los patrones definidos en la especificación de `marlTLC`. La lista organiza las responsabilidades y relaciones de diseño entre sus componentes.

1. **Patrón de estrategia (*Strategy*):**
   - **Propósito:** encapsular los algoritmos de recompensa para intercambiarlos sin modificar el módulo que aplica las señales.
   - **Rol en el diseño:** interfaz base `RewardFn` y variantes concretas de recompensa.
2. **Método de fábrica y registro (*Factory Method* y *Registry*):**
   - **Propósito:** instanciar funciones de recompensa y módulos neuronales a partir de cadenas de configuración o especificaciones compuestas (`RewardSpec`). También permite registrar nuevas funciones sin modificar la fábrica, de acuerdo con el principio abierto/cerrado.
   - **Rol en el diseño:** registro mediante `RewardFactory` y `@register_reward`, que identifica el mecanismo previsto para asociar nombres de configuración con funciones de recompensa.
3. **Patrón de objeto compuesto (*Composite*):**
   - **Propósito:** combinar recompensas locales y vecinales mediante una interfaz uniforme.
   - **Rol en el diseño:** clases de recompensa compuesta que agregan señales locales y vecinales.
4. **Protocolos y tipado estructural (*Protocol* / *Structural Subtyping*):**
   - **Propósito:** desacoplar el proveedor de métricas vecinales del tipo concreto `TrafficLight`, facilitando las pruebas unitarias con objetos simulados (*mocks*) y evitando dependencias circulares.
   - **Rol en el diseño:** protocolo `NeighborInfoProvider` para describir la información vecinal requerida sin depender de una clase concreta. El detalle del decorador de comprobación en tiempo de ejecución no forma parte de la evidencia disponible en este repositorio.
5. **Patrón de adaptador (*Adapter*):**
   - **Propósito:** adaptar la API procedural y basada en comandos de TraCI / `libsumo` a la interfaz orientada a objetos de `MultiAgentEnv` (Ray RLlib).

- **Rol en el diseño:** `SumoRllibEnv` como punto de adaptación entre el simulador y la interfaz de aprendizaje.

6. **Patrón de decorador (*Decorator*):**

   - **Propósito:** extender dinámicamente las capacidades de los módulos sin alterar su jerarquía de herencia.
   - **Rol en el diseño:** decoradores de registro y una envoltura para los módulos de acción en PyTorch; los nombres concretos de esas piezas se mantienen como identificadores de la especificación.

A.4. Parámetros de SUMO y programas semafóricos de referencia

Los archivos de red contienen las secuencias de señales utilizadas antes de aplicar las decisiones de los agentes. En las redes sintéticas, todos los programas son de tiempo fijo y tienen un ciclo de 90 segundos; sus duraciones aparecen en la Tabla 3.1.

La red de tráfico de Mendoza contiene 73 programas: 17 de tiempo fijo y 56 accionados, es decir, que responden a la detección de vehículos. La Tabla A.2 agrupa las secuencias de duraciones nominales del archivo `gran_mendoza2_fixed.net.xml`. Cada fila reúne programas con la misma secuencia de duraciones, aunque sus movimientos habilitados pueden diferir.

Tabla A.2: Duraciones nominales de las fases en el archivo de red de Mendoza, en segundos.

Tipo	Cantidad	Secuencia de duraciones
Accionado	1	33, 6, 6, 6, 33, 6
Accionado	5	39, 6, 39, 6
Accionado	19	40, 5, 40, 5
Accionado	5	41, 4, 41, 4
Accionado	24	42, 3, 42, 3
Accionado	1	81, 4, 5
Accionado	1	82, 3, 5
Fijo	1	24, 3, 24, 3, 6, 3, 24, 3
Fijo	2	39, 6, 39, 6
Fijo	3	40, 5, 40, 5
Fijo	1	41, 4, 41, 4
Fijo	8	42, 3, 42, 3
Fijo	2	82, 3, 5

En los programas accionados, la duración efectiva puede variar. Los atributos `minDur` y `maxDur` establecen los límites de cada verde: los mínimos registrados son 5, 7, 10 o 13

segundos según la fase, y el máximo es de 50 segundos. Por eso, sumar las duraciones nominales de la Tabla anterior no permite afirmar que todos los ciclos ejecutados duren lo mismo.

A.4.1. Identificadores y procedencia del escenario urbano de Mendoza

En la Figura 3.6, el acceso de Avenida José Vicente Zapata se representa mediante el tramo interno `93904440#0`, ubicado en la TAZ 28. La continuidad de los carriles de ingreso se restablece entre los tramos `93904440#0` y `93904440#1`. La configuración de búsqueda y entrenamiento urbano parte de la demanda congelada con el perfil de doce horas, cuyo origen es 06:00. La simulación comienza en $t = 1500$ segundos (06:25), incluye 300 segundos de calentamiento y controla entre $t = 1800$ y $t = 9000$ (06:30–08:30). Se utiliza `nets/Mendoza/mendoza_hpo_v14.sumocfg`; la configuración histórica `mendoza_0630_0830.sumocfg` no incluye el calentamiento.

La comprobación de doce horas controla desde $t = 0$ hasta $t = 43200$ (06:00–18:00). La de dos horas comienza en $t = 1800$ y controla hasta $t = 9000$, sin calentamiento previo. Los registros primarios confirman esas ventanas, pero no se conservaron los archivos de rutas originales de las corridas; por ello, no se afirma que las comprobaciones y la configuración MARL usaran el mismo archivo. Las condiciones de las búsquedas urbanas y la excepción al dominio inicial del peso vecinal se precisan en la subsección A.5.8.

Los dos perfiles de conteo documentados corresponden a versiones distintas y no deben combinarse como si describieran una única entrada canónica. La Tabla A.3 compara sus valores. En la Tabla A.4, se identifican los archivos; las carpetas se detallan después de ella y se señala qué archivos registrados no se conservaron.

Tabla A.3: Resumen de dos versiones documentadas del perfil de demanda de Mendoza.

Dato	Resumen histórico	Generación
Días incluidos	33	32
Media diaria observada (vehículos/día)	25 283,85	25 175,84
Desviación estándar interdiaria (vehículos/día)	2 316,54	No informada
Volumen nominal (vehículos/día)	290 000	260 000
`camera_retention`	0,95	1,00
Flujo horario asociado (vehículos/h)	16 744	14 986
Factor de expansión (redondeado)	11,4698	10,3274

El resumen histórico identificado en la Tabla A.4 corresponde a una versión distinta de la entrada de generación; su factor de expansión registrado es 11,46977289. El log de generación registra 181 pares OD explícitos, una asignación 70/30 y un factor de expansión de 10,327359932077748. La guía histórica advierte que sus parámetros no deben aplicarse a ciegas para regenerar la entrada canónica. Como no se conservaron la matriz ni las

rutas originales, la transformación desde ese registro hasta las rutas consumidas no puede reconstruirse por completo. La regla canónica y sus 520 pares conectables se detallan en la subsección A.4.1.

Los logs canónicos de SUMO sustentan los conteos y las ventanas de la Tabla 4.1; ambos identifican la versión `v1_25_0+0570-f59cd14e561`. La Tabla A.4 identifica el archivo de rutas congelado; su hash completo se presenta después de ella. Este hash coincide con la documentación canónica y el encabezado del archivo declara 14 986,32 vehículos por hora. El registro de generación informa 14 986 vehículos por hora redondeados. Son valores de artefactos distintos y no identifican la ruta consumida por las corridas.

Tabla A.4: Archivos que sustentan la procedencia del escenario urbano de Mendoza.

Fuente	Nombre del archivo
Resumen de conteos	`generation_summary.json`
Registro de generación	`gen.log`
Comprobación de dos horas	`sumo.log`
Comprobación de doce horas	`sumo_12h.log`
Documento canónico	`SCENARIO_CANONICO_V14.md`
Guía de generación	`MENDOZA_SCENARIO_GENERATION_PROCEDURE.md`
Informe de candidatos	`MENDOZA_CANDIDATES_REPORT.md`
Ruta original (no conservada)	`mendoza_camera_calibrated.rou.xml`
Matriz OD (no conservada)	`od_calibrated_7030.json`
Ruta congelada	`mendoza_12h.rou.xml`

Los archivos de la Tabla A.4 se ubican en las siguientes carpetas, relativas a la raíz de `marl-tlc`. El documento canónico, la guía y el informe están en `experiments/mendoza_scenario/`. El resumen histórico está en `experiments/mendoza_scenario/results/scenario_12h/`. El registro de generación, la comprobación de doce horas y la ruta original registrada corresponden a `experiments/mendoza_scenario/results/scenario_canonico_v14/seed_search/s104773/`. La comprobación de dos horas está en `experiments/mendoza_scenario/results/scenario_canonico_v14/validation_2h/`; la matriz OD registrada, en `experiments/mendoza_scenario/results/scenario_canonico_v14/`. La ruta congelada se conserva en `nets/Mendoza/`. La ruta original y la matriz OD registradas no se conservaron.

El hash SHA-256 de la ruta congelada es:
`e7380990e07dbaea1df2cf1013d3ee886b92820ad74f6064786f1082b6716055`.

La ruta registrada para la entrada evaluada original no se conservó, por lo que no puede afirmarse que el archivo congelado produjera las corridas. El informe de candidatos y la documentación canónica registran cero teletransportes, sin que los logs primarios incluyan un contador explícito para confirmarlo.

En `Summary`, `loaded`, `inserted` y `arrived` cuentan los vehículos cargados, insertados

y llegados a destino, respectivamente. `loaded` puede incluir vehículos cuya salida está programada para más adelante. `ended` incluye los que llegaron o fueron retirados de la simulación, por lo que no equivale por sí solo a viajes completados [DLR26].[1] El material de la comprobación operacional de Mendoza no identifica una salida `Summary` con el mapeo exacto de los conteos reportados. Por ello, en la metodología se los define por su significado operacional y no se atribuyen a campos nativos concretos.

Detalle de preprocesamiento y generación de la demanda

El preprocesamiento de los conteos descartó fechas no interpretables, valores negativos, identificadores `ZoneID` fuera del rango 1–19 y códigos `ClassID` distintos de 1, 2 y 3. `ZoneID` identifica una región virtual configurada en la cámara. `ClassID` clasifica los vehículos por longitud: hasta 5 metros, más de 5 y hasta 10 metros, y más de 10 metros. Los tiempos se redondearon al intervalo de 15 minutos más cercano y los valores se agregaron por cámara, intervalo, zona y clase.

La suma de las longitudes de carril es 66,3 km; cuenta cada carril por separado y no equivale a la extensión geográfica del área. Las longitudes de carril varían aproximadamente entre 11 y 276 m, con una media cercana a 120 m; los límites viales de velocidad registrados van de 10 a 100 km/h. Los agentes controlan de 3 a 10 carriles entrantes. De los 73 agentes descritos en el cuerpo, 67 tienen dos acciones, cuatro tienen una, uno tiene tres y uno tiene cuatro. El grado de vecindad varía entre dos y cuatro, con una media de 3,23.

El registro de generación asociado a la semilla 104773 incluye el factor de expansión 10,327359932077748 y `camera_retention`=1. Este último conserva el conteo de cámara completo como referencia para el ingreso modelado; es una decisión de construcción, no una proporción observada de vehículos que alcanzan el área urbana. La Tabla A.3 mantiene separados este registro y el resumen histórico, que informa `camera_retention`=0,95.

Las 39 zonas, identificadas de 0 a 38, permiten 1482 pares OD entre zonas distintas. Una búsqueda en anchura (*breadth-first search*, BFS) determinó que 520 pares son conectables. En la especificación canónica, 473 pares reciben proporciones fijas cuya suma es 0,60, 44 reciben proporciones aleatorias que suman 0,40 y tres se excluyen. Para un par conectable, o y d denotan origen y destino, p_{od} su proporción, y p_{od}^{fijo} la fracción especificada para los pares fijos. Los conjuntos disjuntos $\mathcal{P}_{\text{fijo}}$, $\mathcal{P}_{\text{aleat}}$ y $\mathcal{P}_{\text{excl}}$ identifican, respectivamente, pares fijos, aleatorios y excluidos. Para cada par aleatorio, u_{od} se muestrea

[1] `https://sumo.dlr.de/docs/Simulation/Output/Summary.html`, consultada el 13 de octubre de 2026.

uniformemente entre cero y uno. La regla canónica es:

$$p_{od} = \begin{cases} p_{od}^{\text{fijo}}, & (o,d) \in \mathcal{P}_{\text{fijo}}, \\ 0, & (o,d) \in \mathcal{P}_{\text{excl}}, \\ \dfrac{0,40\, u_{od}}{\sum_{(o',d') \in \mathcal{P}_{\text{aleat}}} u_{o'd'}}, & (o,d) \in \mathcal{P}_{\text{aleat}}. \end{cases} \tag{A.1}$$

La normalización requiere que la suma de los pesos aleatorios sea positiva. Esta regla canónica de 60/40 es distinta del 70/30 informado en el registro de generación; no se conservó la matriz original de ese registro.

Para un flujo positivo Q_{od} en vehículos por hora, la tasa de llegadas es $\lambda_{od} = Q_{od}/3600$ vehículos por segundo. Si U tiene distribución uniforme en $(0, 1)$, el intervalo programado H_{od}, en segundos, se obtiene mediante:

$$H_{od} = -\frac{\ln(U)}{\lambda_{od}}, \tag{A.2}$$

donde ln es el logaritmo natural. El intervalo representa una llegada programada; si no hay espacio en el acceso, la inserción efectiva ocurre más tarde.

El estadístico GEH compara el flujo horario nominal programado, V_{mod}, y el flujo horario observado, V_{obs}, ambos en vehículos por hora:

$$\text{GEH} = \sqrt{\frac{2(V_{\text{mod}} - V_{\text{obs}})^2}{V_{\text{mod}} + V_{\text{obs}}}}. \tag{A.3}$$

Se aplica cuando $V_{\text{mod}} + V_{\text{obs}} > 0$, vale cero cuando los flujos coinciden y no es un porcentaje. Como el conteo de la cámara 169 participó en la construcción del perfil, esta comparación no constituye validación independiente de la demanda urbana.

El parámetro `waiting_time_memory` determina cuántos segundos de historia conserva SUMO al consultar la espera acumulada de un vehículo. En los experimentos de aprendizaje se fija en 1000 segundos. Esta ventana afecta la observación del agente, mientras que el atributo `waitingTime` de TripInfo acumula la espera durante el viaje [Ecl26b, Ecl26c].

En los episodios de aprendizaje, el entorno guarda el estado de SUMO al terminar el calentamiento y lo carga al reiniciar el episodio. La versión conservada para A/C no activa la opción de guardar los generadores aleatorios; la restauración del estado vehicular, por sí sola, no acredita que se repitan sus secuencias. El protocolo urbano activa `-save-state.rng` y conserva un estado posterior al calentamiento para cada semilla de SUMO. Esta opción permite restaurar también los generadores aleatorios [Ecl26a]; su uso no altera los archivos de demanda congelados.

A.5. Especificaciones para reproducir los experimentos

Este apéndice distingue los valores observados en configuraciones y archivos de ejecución de los parámetros, dominios y semillas que aparecen en los diseños de búsqueda. En este contexto, fase 1 y fase 2 designan etapas de búsqueda de hiperparámetros.

A.5.1. Infraestructura y almacenamiento de las ejecuciones

Los programas de envío de trabajos, o *scripts*, especifican recursos solicitados; se distinguen de los registros de asignación efectiva. La metodología resume las plataformas utilizadas en la subsección 3.7.1.

Los trabajos se envían con `sbatch`, comando del gestor de trabajos herramienta simple de Linux para gestión de recursos (*Simple Linux Utility for Resource Management*, SLURM). En las búsquedas sintéticas, fase 1 utiliza `submit_phase1.sh` y fase 2 especifica `submit_phase2_refined.sh`.

Las configuraciones de los scripts corresponden a una combinación de algoritmo, red de tráfico y arquitectura, y solicitan un servidor del clúster (nodo), una tarea de ejecución, diez núcleos de la unidad central de procesamiento (*Central Processing Unit*, CPU) y una partición —grupo de recursos que el sistema asigna para ejecutar trabajos— con unidades de procesamiento gráfico (GPU). El manual institucional describe una partición de Toko con hasta ocho núcleos; por ello no permite confirmar los recursos físicos asignados a cada trabajo [Tok24].

El script refinado también especifica el uso exclusivo del nodo y un límite de tiempo. La configuración de Ray limita a un ensayo simultáneo por trabajo; ASHA define las detenciones anticipadas según los parámetros de cada búsqueda.

Los registros de las ejecuciones documentadas se guardaron en almacenamiento persistente y se copiaron al espacio temporal del nodo durante la ejecución. Aunque los scripts configuraron un recurso de GPU para Ray, no solicitaron una tarjeta física mediante un identificador de recurso de SLURM; por ello, los archivos del proyecto no permiten asociar cada trabajo con un modelo específico de GPU.

A.5.2. Comprobaciones preliminares

En una comprobación de 3×3, cerca del 70 % de los vehículos cargados —incluidos en la demanda de SUMO— completó el viaje; este dato no se extrapola a las otras redes. Esa comprobación incluyó 600 segundos de calentamiento y 3600 segundos de control.

En la prueba preliminar de formulaciones de MAPPO (subsección 3.5.1), B y D incluyen ejecuciones completadas y no se clasifican como fallos técnicos. Esta comprobación distingue el estado de ejecución de las diferencias de desempeño.

A.5.3. Detalles de las redes sintéticas

La Tabla 3.3 reúne las dimensiones comparables de las redes. Este apartado conserva las excepciones topológicas y las asignaciones de demanda por par origen–destino (OD) que explican la función de cada escenario.

En el corredor 3×1, los cruces B1, C1 y D1 se disponen en línea. El 75 % del flujo solicitado se asigna a los pares de zonas $3 \rightarrow 4$ (40 %) y $4 \rightarrow 3$ (35 %); el 25 % restante se distribuye de manera reproducible entre los pares válidos.

La red 3×3 forma una cuadrícula regular. Sus asignaciones principales son 20 % para $5 \rightarrow 6$, 17,5 % para $6 \rightarrow 5$, 17,5 % para $10 \rightarrow 1$ y 20 % para $1 \rightarrow 10$; el 25 % restante se reparte entre los pares válidos.

En $4 \times 4_H$, la fila 2 tiene tres carriles por sentido, mientras que el sector A1–B1–C1 de la fila 1 tiene uno por sentido. La columna C es asimétrica, con cuatro carriles hacia el sur y dos hacia el norte. La columna D solo permite circular hacia el norte: tiene cuatro carriles hasta D3 y tres desde D3 hasta el borde norte. Las demás vías tienen dos carriles por sentido donde existe circulación. Faltan los enlaces horizontales B4–C4 y D1–E1, por lo que la red no es una malla completa.

C1 tiene tres acciones seleccionables. C4 y D1 tienen dos, aunque sus programas de referencia contienen cinco fases, porque las fases amarillas y el estado final de todo rojo no son acciones de los agentes. B4 y E1 tienen un vecino; los demás agentes tienen entre dos y cuatro. La demanda asigna 20 % a $5 \rightarrow 9$, 17,5 % a $6 \rightarrow 5$, 17,5 % a $13 \rightarrow 1$ y 20 % a $1 \rightarrow 13$; el 25 % restante se distribuye entre pares válidos. Estas diferencias de capacidad, conectividad y demanda se presentan como características del escenario, no como una variación controlada de un único factor.

A.5.4. Espacio de búsqueda inicial de fase 1

Esta sección especifica los dominios iniciales por algoritmo y los factores que se mantienen fijos. Los rangos se contrastaron con el código de `experiments/marl_experiments/tuning/shared/search_space.py` y con la revisión documental del 10 de octubre de 2026 conservada en `docs/initial_search_space_criteria.md`, dentro de `marl-tlc`. Mendoza utiliza estos dominios, salvo el peso vecinal, que fija en cero después de A/C. Las tablas posteriores de presupuestos y semillas corresponden a las búsquedas sintéticas; las condiciones urbanas se reúnen en la subsección A.5.8. Esta reconstrucción permite identificar la especificación consultada; una clave almacenada en un archivo de configuración no demuestra por sí sola cómo operó cada ejecución histórica.

Las Tablas A.5 y A.6 reúnen los dominios por familia algorítmica. Cada fila identifica el parámetro por su función; la Tabla A.8 conserva su correspondencia con las claves del código. Uniforme continua y loguniforme tienen el significado explicado en la sección 3.4.3; uniforme entera permite cualquiera de los enteros del rango, incluidos ambos extremos, y categórico restringe la búsqueda a las alternativas enumeradas. Los intervalos continuos

conservan la notación de las distribuciones declaradas; la inclusión de cero como extremo no garantiza una propuesta exactamente nula. En las llamadas enteras del código, el límite superior es exclusivo: por ejemplo, `randint(4,65)` permite los enteros de 4 a 64. La excepción urbana del peso vecinal también se aplica a estas tablas. Otros factores, como el límite de escala del codificador de espera acumulada por carril, se fijan por escenario. La Tabla A.7 especifica esos factores para los estudios sintéticos documentados.

En la familia PPO, el margen de recorte limita la razón de probabilidades del actor y el parámetro de GAE regula la estimación de ventajas. La entropía favorece la exploración; los parámetros de divergencia KL configuran la penalización y su adaptación. El límite del gradiente acota la magnitud del gradiente utilizado para actualizar los parámetros. Las cantidades de episodios, recorridos de optimización y minilotes controlan la recolección y reutilización de experiencias. En MAPPO y HAPPO, la cota de valor limita la pérdida cuadrática del crítico, según la sección A.6; no se interpreta como un límite físico de espera. En HAPPO, la probabilidad de ordenación determina con qué frecuencia se elige el siguiente actor según la magnitud de su ventaja, en lugar de elegirlo al azar; su aplicación se precisa después de la tabla.

Tabla A.5: Espacio inicial de fase 1 para MAPPO, IPPO y HAPPO. Las excepciones por algoritmo se indican en las filas.

Parámetro	Dominio	Muestreo
Peso de recompensa vecinal	[0; 0,95)	Uniforme continua
Tasa de aprendizaje	$[3 \times 10^{-5}; 10^{-3}]$	Loguniforme
Recorte del actor	[0,10; 0,22)	Uniforme continua
Parámetro de GAE	[0,88; 0,99)	Uniforme continua
Peso de entropía	[0; 0,01)	Uniforme continua
Límite del gradiente	[0,25; 5]	Loguniforme
Coeficiente de penalización KL	[0,05; 0,5]	Loguniforme
Objetivo de divergencia KL	[0,005; 0,03]	Loguniforme
Cota de pérdida de valor	[5; 20]	Loguniforme
Recorridos de optimización	5–15; HAPPO: 5–18	Uniforme entera
Episodios por lote	2–12	Uniforme entera
Niveles del codificador	4–64	Uniforme entera
Tamaño del minilote*	{128; 256; 512}	Categórico
Probabilidad de ordenación (HAPPO)	[0,20; 0,90)	Uniforme continua

El asterisco de `minibatch_size` señala una diferencia de implementación: el módulo de aprendizaje de HAPPO consultado procesa el lote completo por actor, aunque la configuración registre un tamaño de minilote. En ese módulo, la clave registrada no determina una subdivisión efectiva del lote. La probabilidad de ordenación, en cambio, interviene al elegir el siguiente actor: selecciona al de mayor magnitud media de ventaja con la probabilidad configurada y, en caso contrario, elige al azar entre los restantes. Su dominio

excluye tanto la ordenación completamente aleatoria como la selección siempre guiada por ese criterio.

En las redes sintéticas, para PPO, entre 2 y 12 episodios nominales por lote equivalen a un objetivo de 1440 a 8640 pasos de decisión, al multiplicar por $3600/5 = 720$ pasos por episodio controlado. La recolección de episodios completos puede producir un tamaño efectivo distinto. Los niveles del codificador modifican la representación de las observaciones; el peso vecinal modifica la recompensa. No son parámetros del optimizador y se distinguen de la inclusión de información vecinal en la observación, que permanece desactivada durante esta búsqueda.

En IDQN, la probabilidad de exploración desciende desde uno hasta el valor final propuesto, durante el número de pasos configurado. La red objetivo se actualiza con la frecuencia indicada, y el horizonte de varios pasos fija cuántas transiciones intervienen en el retorno utilizado para aprender. Sus frecuencias y contadores corresponden a pasos de entrenamiento o muestreo según la interfaz de RLlib, no a segundos de SUMO. El lote de IDQN contiene transiciones y no equivale al número de episodios utilizado para los métodos PPO.

Tabla A.6: Espacio inicial de fase 1 para IDQN. Los contadores se expresan en pasos según la interfaz del algoritmo.

Parámetro	Dominio	Muestreo
Peso de recompensa vecinal	[0; 0,95)	Uniforme continua
Tasa de aprendizaje	$[10^{-5}; 10^{-3}]$	Loguniforme
Transiciones por lote	{64; 128; 256; 512}	Categórico
Límite del gradiente	{0,5; 1; 2; 5; 10}	Categórico
Actualización de red objetivo	{500; 1000; 2000; 5000}	Categórico
Pasos para reducir exploración	{5000; 10000; 20000}	Categórico
Exploración final	[0,02; 0,10)	Uniforme continua
Horizonte del retorno	{1; 3; 5}	Categórico
Pasos antes de aprender	{1000; 2000; 5000}	Categórico
Niveles del codificador	4–64	Uniforme entera

La Tabla A.7 separa los factores de estudio y los valores fijos de los parámetros anteriores. La arquitectura se conserva dentro de cada estudio para evitar que RLlib 2.53 altere la arquitectura al almacenarla dentro de otros campos de configuración, según la decisión registrada en `experiments/marl_experiments/tuning/shared/architectures.py`. El límite de escala del codificador de espera acumulada por carril se calibra por escenario; su fijación no implica que el percentil utilizado sea superior a otras alternativas.

Tabla A.7: Factores de estudio y valores fijos documentados para la fase 1 sintética.

Factor	Valor o alternativa
Arquitectura por estudio	h128 o h256: dos capas de 128 o 256 unidades
Descuento	$\gamma = 0{,}99$
Medición de espera	Espera acumulada por carril (veh·s)
Límite de escala del codificador, 3×1	1184 veh·s
Límite de escala del codificador, 3×3	1136 veh·s
Límite de escala del codificador, $4 \times 4_H$	712 veh·s
Información observada	Observación local y máscara de acciones
Penalización KL	Activada en los métodos PPO

La arquitectura permanece fija por estudio, y el límite de escala del codificador de espera acumulada por carril varía según el escenario. La medición de espera y el descuento son comunes a los cuatro algoritmos. Variar el peso de recompensa vecinal no activa la observación vecinal. En los métodos PPO, la penalización KL permanece activada, aunque su coeficiente y su objetivo se exploran.

Los rangos y distribuciones documentan cómo se acotó la exploración. No se recuperó una justificación contemporánea para todos sus límites exactos, ni una comparación que validara las alternativas de cada fila. La calibración de espera y la separación de arquitecturas tienen un respaldo local identificable; los demás límites se conservan como decisiones del diseño exploratorio. Las propuestas adaptativas de TPE, las detenciones de ASHA y la única semilla de entrenamiento por escenario limitan las inferencias sobre sensibilidad y variabilidad. La especificación no garantiza cobertura exhaustiva del espacio ni optimalidad global.

Tabla A.8: Correspondencia entre los parámetros de búsqueda y sus claves de configuración.

Parámetro	Clave en el código
Peso de recompensa vecinal	`neighborhood_relevance`
Tasa de aprendizaje	`lr`
Recorte del actor	`clip_param`
Parámetro de GAE	`lambda_gae`
Peso de entropía	`entropy_coeff`
Límite del gradiente	`grad_clip`
Coeficiente de penalización KL	`kl_coeff`
Objetivo de divergencia KL	`kl_target`
Cota de pérdida de valor	`vf_clip_param`
Recorridos de optimización	`num_epochs`
Episodios por lote	`train_batch_episodes`
Niveles del codificador	`encoder_intervals`
Tamaño del minilote	`minibatch_size`
Probabilidad de ordenación (HAPPO)	`greedy_selection_prob`
Transiciones por lote	`train_batch_size_per_learner`
Actualización de red objetivo	`target_network_update_freq`
Pasos para reducir exploración	`epsilon_timesteps`
Exploración final	`epsilon_final`
Horizonte del retorno	`n_step`
Pasos antes de aprender	`num_steps_sampled_before_learning_starts`

A.5.5. Semillas de fase 1 en las redes sintéticas

La Tabla A.9 reúne las semillas observadas en las configuraciones serializadas de fase 1 y las asociadas a los perfiles canónicos de generación de rutas. La semilla de entrenamiento inicializa el aprendizaje y la simulación SUMO durante el entrenamiento; la de evaluación controla SUMO en las evaluaciones diagnósticas.

Tabla A.9: Semillas por escenario sintético en la fase 1.

Escenario	Entrenamiento	Evaluación	Rutas
3×1	101	100101	1333938385
3×3	137	100137	810840844
$4 \times 4_H$	179	100179	1871007461

El código de fase 1 define 7001 como semilla TPE por defecto, independiente de las semillas de entrenamiento.

A.5.6. Arquitecturas y semillas de la búsqueda refinada sintética

Este apartado detalla la búsqueda refinada sintética, posterior a la prueba A–E y a la comparación A/C de MAPPO. La fase 2 abarca los cuatro escenarios; en Mendoza, los dominios y condiciones de búsqueda son propios de la red urbana y se derivan de su fase 1.

El protocolo de búsqueda refinada sintética fija un máximo de 100 configuraciones por combinación de algoritmo y escenario y de 60 iteraciones por ensayo. La configuración de ASHA fija 30 iteraciones de gracia y un factor de reducción de 2; además, establece una ejecución concurrente por combinación y un episodio de seguimiento sin exploración cada 15 iteraciones. Los fallos, las detenciones anticipadas y las interrupciones consumen parte del presupuesto. Los valores sintéticos de presupuesto, semillas y arquitectura no se trasladan a la búsqueda urbana. Las fuentes históricas con rangos o arquitecturas distintos conservan su identidad.

El estado del modelo al final de un ensayo se guarda en un archivo *checkpoint*, que puede servir para reanudar o evaluar el entrenamiento. Su existencia no libera presupuesto si el ensayo está incompleto ni garantiza que la política sea admisible para selección. En la búsqueda sintética, la fase 1 adapta las propuestas mediante TPE y las detenciones mediante ASHA, sin reservar una prueba en $\rho_{\text{coop}} = 0$; por eso sus registros no identifican la causa de las diferencias asociadas al peso vecinal ni demuestran que el valor local sea un óptimo.

La selección de configuraciones, el entrenamiento final desde cero y la evaluación comparativa son etapas posteriores a las búsquedas. Las semillas 950001, 950003 y 950007 se utilizan en la evaluación de candidatos sintéticos descrita en el apartado de admisibilidad y selección de la subsección A.5.6.

Tabla A.10: Arquitecturas y semillas de la búsqueda refinada para las doce combinaciones sintéticas. La semilla de evaluación es común a los algoritmos del mismo escenario; las semillas de TPE no son réplicas de entrenamiento.

Algoritmo	Escenario	Arquitectura	Entrenamiento	Evaluación	TPE
IPPO	3×1	h256	910001	920001	930001
IPPO	3×3	h128	910002	920003	930002
IPPO	$4 \times 4_H$	h128	910003	920004	930003
MAPPO	3×1	h128	910004	920001	930004
MAPPO	3×3	h128	910005	920003	930005
MAPPO	$4 \times 4_H$	h128	910006	920004	930006
HAPPO	3×1	h256	910007	920001	930007
HAPPO	3×3	h256	910008	920003	930008
HAPPO	$4 \times 4_H$	h128	910009	920004	930009
IDQN	3×1	h256	910010	920001	930010
IDQN	3×3	h128	910011	920003	930011
IDQN	$4 \times 4_H$	h256	910012	920004	930012

Las arquitecturas h128 y h256 contienen dos capas ocultas de 128 y 256 unidades, respectivamente. La arquitectura asignada a cada combinación prevista se indica en la Tabla A.10. Son preferencias de búsqueda derivadas de la fase 1, no evidencia de superioridad bajo la recompensa local. Las semillas TPE controlan las propuestas; las semillas SUMO, la simulación. Ninguna sustituye una réplica de entrenamiento.

Admisibilidad y selección de candidatos

Para los candidatos sintéticos, la selección exige que el ensayo haya alcanzado 60 iteraciones, que sus métricas sean numéricas y finitas y que existan al menos 15 registros finitos de métricas de tránsito. También se comprueba que la configuración efectiva y el archivo del modelo final estén completos y puedan leerse. Por sí solo, el estado `COMPLETE` de Optuna indica que el ensayo terminó sin registrar un fallo, pero no garantiza el cumplimiento de estos requisitos.

La lista inicial de candidatos sintéticos se construye con `build_shortlist`, en `experiments/marl_experiments/tuning/phase2_refined/closure_audit.py`, de la copia `phase2-cluster` de `marl-tlc`. Entre los ensayos técnicamente admisibles se consideran aquellos con espera diagnóstica disponible. Se priorizan los que registran cero inicios de teletransporte y una CI diagnóstica de al menos 0,95; después se ordenan por menor espera diagnóstica y por el identificador de la política. Se toma el primero de cada arquitectura disponible y se completa la lista, hasta tres candidatos, con los restantes en ese orden. Este ordenamiento preliminar no sustituye los filtros del panel común de selección. En Mendoza se aplican los criterios urbanos descritos en la subsección 3.4.5.

Cada candidato se evalúa con las semillas SUMO 950001, 950003 y 950007, comunes a los algoritmos de una misma red. Los tres episodios deben registrar cero inicios de teletransporte, una CI de al menos 0,95 y una cantidad de vehículos insertados igual o superior al 95 % de la referencia empírica para esa red y semilla.

Para cada red y semilla, la referencia de inserciones es el máximo observado entre los episodios del panel inicial, que reúne los candidatos de los cuatro algoritmos. Esta referencia no es una medición independiente de la demanda total. La lista de candidatos y las semillas se conservan antes de iniciar el panel mediante `closure_panel.py`. Una vez reunidas sus evaluaciones, `select_recipes`, en `experiments/marl_experiments/tuning/phase2_refined/closure.py`, calcula ese máximo antes de aplicar los filtros y lo utiliza para todos los candidatos del panel. El campo `teleports` de estas evaluaciones corresponde a `teleport_starts`, obtenido mediante TraCI.

Entre quienes cumplen los tres filtros se elige el candidato con menor espera media en las tres evaluaciones. En caso de empate exacto, se priorizan la mayor CI media, el menor tiempo de entrenamiento registrado y, por último, el identificador de la política. Si no se puede completar la auditoría o faltan evaluaciones, la combinación se informa como incompleta; si ningún candidato cumple los filtros, se informa que no hay candidato admisible. El procedimiento de selección termina al identificar una configuración admisible. El entrenamiento final desde cero y su evaluación se describen en la subsección 3.6.2 de la metodología. Las semillas de selección no reemplazan una evaluación independiente posterior.

El criterio `final_declared_budget` del contrato de entrenamiento identifica el estado de la última iteración del presupuesto completo declarado en `training_iterations`. La comprobación del modelo restaurado exige que su número de iteración coincida con ese valor. El entrenamiento final adopta este criterio para elegir el *checkpoint*; los filtros de selección de las redes sintéticas se aplican al candidato que aporta la configuración, mientras que los diagnósticos del modelo final se interpretan en su evaluación independiente.

A.5.7. Dominios declarados y significado de los parámetros

Las Tablas A.11–A.14 presentan los doce dominios sintéticos declarados para la búsqueda refinada, uno por combinación de algoritmo, escenario sintético y arquitectura prevista. Los valores corresponden al archivo `experiments/marl_experiments/tuning/phase2_refined/informed_domains_rho0_v3.json`, localizado en la copia de trabajo `phase2-cluster` del proyecto `marl-tlc`. Mendoza también forma parte de la fase 2 de HPO, con dominios propios del escenario urbano; no integra estas doce combinaciones sintéticas. La columna «Muestreo» indica el tipo de selección sin abreviaturas: uniforme continua, loguniforme, uniforme entera o categórico, con los significados descritos al presentar el espacio inicial. En el muestreo entero, $[a; b)$ permite los enteros desde a hasta $b-1$: el extremo superior queda excluido. En los dominios continuos, los corchetes indican

los límites declarados; no garantizan que se propongan sus extremos. Las llaves enumeran las únicas alternativas del muestreo categórico. Cada columna de valores corresponde al escenario y a la arquitectura indicados.

Tabla A.11: Dominios declarados para la búsqueda `phase2_refined` de HAPPO por escenario y arquitectura.

Parámetro	Muestreo	3×1 (h256)	3×3 (h256)	$4 \times 4_{\mathrm{H}}$ (h128)
Recorte del actor	Uniforme continua	[0,1; 0,22]	[0,1; 0,22]	[0,1; 0,22]
Niveles del codificador	Uniforme entera	[4; 27)	[5; 37)	[4; 65)
Peso de entropía	Uniforme continua	[0,0043479; 0,0079967]	[0; 0,01]	[0; 0,01]
Límite del gradiente	Loguniforme	[0,64487; 4,7566]	[0,26841; 1,3289]	[0,25; 5]
Probabilidad de ordenación	Uniforme continua	[0,40484; 0,82866]	[0,44174; 0,82189]	[0,48405; 0,77601]
Coeficiente de penalización KL	Loguniforme	[0,05; 0,5]	[0,066096; 0,19175]	[0,05; 0,5]
Objetivo de divergencia KL	Loguniforme	[0,015783; 0,03]	[0,005; 0,03]	[0,005; 0,03]
Parámetro de GAE	Uniforme continua	[0,88683; 0,93862]	[0,90514; 0,9443]	[0,88; 0,99]
Tasa de aprendizaje	Loguniforme	$[2{,}707 \times 10^{-4};\ 1 \times 10^{-3}]$	$[3 \times 10^{-5};\ 1 \times 10^{-3}]$	$[3 \times 10^{-5};\ 1 \times 10^{-3}]$
Tamaño del minilote*	Categórico	{256; 512}	{128; 512}	{256; 512}
Recorridos de optimización	Uniforme entera	[5; 19)	[5; 19)	[5; 19)
Episodios por lote	Uniforme entera	[2; 13)	[2; 13)	[2; 13)
Cota de pérdida de valor	Loguniforme	[8,402; 18,471]	[6,674; 20]	[5; 11,218]

Tabla A.12: Dominios declarados para la búsqueda `phase2_refined` de IDQN por escenario y arquitectura.

Parámetro	Muestreo	3×1 (h256)	3×3 (h128)	$4 \times 4_{\mathrm{H}}$ (h256)
Niveles del codificador	Uniforme entera	[10; 62)	[7; 33)	[4; 65)
Exploración final	Uniforme continua	[0,02; 0,047523]	[0,023555; 0,066094]	[0,02; 0,1]
Pasos para reducir exploración	Categórico	{10 000; 20 000}	{10 000; 20 000}	{10 000; 20 000}
Límite del gradiente	Categórico	{0,5; 1; 2; 5; 10}	{0,5; 1; 2}	{1; 2; 5; 10}
Tasa de aprendizaje	Loguniforme	$[9{,}879 \times 10^{-5};\ 1 \times 10^{-3}]$	$[1 \times 10^{-5};\ 1 \times 10^{-3}]$	$[8{,}135 \times 10^{-5};\ 7{,}889 \times 10^{-4}]$
Horizonte del retorno	Categórico	{3; 5}	{1; 3}	{1; 3; 5}
Pasos antes de aprender	Categórico	{1 000; 2 000; 5 000}	{1 000; 2 000; 5 000}	{1 000; 2 000; 5 000}
Actualización de red objetivo	Categórico	{500; 1 000; 2 000; 5 000}	{500; 2 000; 5 000}	{500; 1 000; 2 000}
Transiciones por lote	Categórico	{64; 128; 256}	{256; 512}	{64; 128; 256}

Tabla A.13: Dominios declarados para la búsqueda refinada de IPPO por escenario y arquitectura.

IPPO				
Parámetro	Muestreo	3×1 (h256)	3×3 (h128)	$4 \times 4_{\mathrm{H}}$ (h128)
Recorte del actor	Uniforme continua	[0,1; 0,22]	[0,1; 0,22]	[0,15327; 0,19443]
Niveles del codificador	Uniforme entera	[4; 65)	[4; 65)	[4; 65)
Peso de entropía	Uniforme continua	[0,0038889; 0,0095621]	[0,00067906; 0,0074136]	[0,0036258; 0,01]
Límite del gradiente	Loguniforme	[0,25; 5]	[0,25; 5]	[0,42722; 1,5143]
Coeficiente de penalización KL	Loguniforme	[0,055016; 0,21664]	[0,05; 0,2904]	[0,05; 0,5]
Objetivo de divergencia KL	Loguniforme	[0,005; 0,03]	[0,005; 0,03]	[0,005; 0,03]
Parámetro de GAE	Uniforme continua	[0,88; 0,99]	[0,89174; 0,96008]	[0,88; 0,99]
Tasa de aprendizaje	Loguniforme	$[3 \times 10^{-5}; 1 \times 10^{-3}]$	$[1{,}136 \times 10^{-4}; 9{,}113 \times 10^{-4}]$	$[1{,}322 \times 10^{-4}; 4{,}772 \times 10^{-4}]$
Tamaño del minilote	Categórico	{128; 512}	{128; 512}	{128; 512}
Recorridos de optimización	Uniforme entera	[5; 16)	[5; 16)	[5; 16)
Episodios por lote	Uniforme entera	[2; 13)	[2; 13)	[2; 13)
Cota de pérdida de valor	Loguniforme	[5; 20]	[5; 20]	[5; 20]

Tabla A.14: Dominios declarados para la búsqueda refinada de MAPPO por escenario y arquitectura.

MAPPO				
Parámetro	Muestreo	3×1 (h128)	3×3 (h128)	$4 \times 4_{\mathrm{H}}$ (h128)
Recorte del actor	Uniforme continua	[0,1; 0,22]	[0,1; 0,22]	[0,1; 0,22]
Niveles del codificador	Uniforme entera	[4; 65)	[8; 45)	[4; 65)
Peso de entropía	Uniforme continua	[0,00015809; 0,0062478]	[0; 0,01]	[0,0021511; 0,0080125]
Límite del gradiente	Loguniforme	[0,28422; 0,80645]	[0,25; 5]	[1,1032; 5]
Coeficiente de penalización KL	Loguniforme	[0,05; 0,13242]	[0,05; 0,5]	[0,14905; 0,38494]
Objetivo de divergencia KL	Loguniforme	[0,005; 0,03]	[0,01384; 0,028938]	[0,005; 0,01363]
Parámetro de GAE	Uniforme continua	[0,88703; 0,92246]	[0,88; 0,99]	[0,88; 0,99]
Tasa de aprendizaje	Loguniforme	$[4{,}285 \times 10^{-5}; 1{,}695 \times 10^{-4}]$	$[1{,}947 \times 10^{-4}; 1 \times 10^{-3}]$	$[3 \times 10^{-5}; 1 \times 10^{-3}]$
Tamaño del minilote	Categórico	{256; 512}	{128; 512}	{128; 256; 512}
Recorridos de optimización	Uniforme entera	[5; 16)	[6; 13)	[5; 16)
Episodios por lote	Uniforme entera	[2; 13)	[2; 13)	[2; 13)
Cota de pérdida de valor	Loguniforme	[5; 20]	[5; 20]	[5; 20]

Para los dominios sintéticos de las tablas, la documentación describe su derivación a partir de los ensayos de fase 1 que completaron el presupuesto sin errores y cuyo objetivo es numérico y finito, ordenados de menor a mayor por ese objetivo. Como vector inicial, propone la mediana de los parámetros numéricos y la moda de los categóricos entre los cinco mejores; esa combinación puede no coincidir con un ensayo histórico. También plantea entrenar desde cero, sin incorporar puntuaciones, archivos de estado del modelo (*checkpoint*) ni bases Optuna de fase 1. La búsqueda urbana sigue la misma relación entre fases con sus propios dominios.

En parámetros numéricos se parte del intervalo de los diez mejores ensayos, se amplía cada extremo en un 10 % del ancho del dominio original y se limita a sus fronteras. Los parámetros loguniformes se tratan en escala logarítmica. Se conserva el dominio original si el intervalo de los veinte mejores ensayos cubre al menos el 70 % de su ancho o si alguno de los diez mejores queda a una distancia de un extremo no mayor que el 5 % de ese ancho. Para los parámetros enteros se calcula el ancho entre el menor y el mayor valor admisibles; las distancias del 5 % y del 10 % se redondean hacia arriba, con un margen mínimo de una unidad para la ampliación. El extremo superior del dominio resultante se expresa como exclusivo. En parámetros categóricos se consideran las alternativas de los veinte mejores; si solo aparece una, se añade la siguiente en el orden del dominio original o, si ocupa la última posición, la anterior.

El asterisco de `minibatch_size` en la Tabla A.11 indica que el parámetro se registra, pero HAPPO procesa el lote completo por actor. En PPO, `encoder_intervals` fija los niveles del codificador, `clip_param` limita la razón de probabilidades del actor y `entropy_coeff` pondera la entropía. `kl_coeff` pondera la penalización KL entre la política recolectora y la política actual. `kl_target` registra el objetivo asociado a la adaptación de ese coeficiente y se distingue de la ponderación que aplica `kl_coeff`. `num_epochs` indica los recorridos de optimización del lote; `minibatch_size` fija el tamaño de sus subdivisiones cuando se usan.

En las redes sintéticas, para PPO, el código multiplica `train_batch_episodes` por los pasos de control del episodio ($3600/5 = 720$) para calcular el tamaño del lote por componente de aprendizaje; ese cálculo no incluye un factor adicional por el número de procesos de simulación. En IDQN, `num_steps_sampled_before_learning_starts` fija los pasos muestreados antes de iniciar el aprendizaje; `target_network_update_freq` fija el intervalo de actualización de la red objetivo; `train_batch_size_per_learner` es el tamaño del lote de transiciones; `n_step`, el horizonte del valor objetivo temporal; `epsilon_final`, el nivel final de exploración; y `epsilon_timesteps`, el horizonte de su programación. $c_v =$ `vf_clip_param` es la cota superior de la pérdida cuadrática de valor, no un recorte PPO convencional de predicciones.

A.5.8. Protocolo urbano de Mendoza

El protocolo `mendoza_rho0_v1` reúne las condiciones urbanas en `experiments/marl_experiments/tuning/urban/protocol.json`. El manifiesto `docs/mendoza_preparation/preparation_manifest.json` identifica los recursos congelados de v14, sus hashes y las versiones de software. Este protocolo utiliza libsumo 1.26.0; sus condiciones se distinguen de las comprobaciones históricas con el ejecutable SUMO 1.25.0.

Los archivos `nets/Mendoza/mendoza_hpo_v14.sumocfg` y `nets/Mendoza/mendoza_evaluation_v14.sumocfg` conservan la red y la demanda congeladas y definen las ventanas indicadas en la metodología. Ambos fijan `time-to-teleport`=300 segundos y `max-num-teleports`=−1. El primero regula el umbral de espera por bloqueo que puede iniciar un teletransporte; el segundo permite registrarlos sin detener la simulación al ocurrir uno [DLR26, Ecl26d]. La selección compara sus inicios con la referencia, según la subsección 3.4.5. La inicialización, la restauración de los generadores aleatorios y el cierre por truncamiento se describen en la subsección 3.3.1.

La calibración del codificador se registra en `docs/mendoza_preparation/encoder_calibration.json`. Utiliza los programas de referencia, las semillas 5001 y 5002, 300 segundos de calentamiento, 7200 segundos de control y una memoria de espera de 1000 segundos. Cada segundo se calcula el máximo, entre los carriles controlados, de la suma de `getAccumulatedWaitingTime` de sus vehículos. Las dos simulaciones aportan 14.400 muestras; su percentil 99 es 5396,04 veh · s y se redondea al entero más próximo para fijar $M_{\text{lane}} = 5396$ veh · s. Esta suma conserva esperas anteriores de los vehículos, mientras que el entorno atribuye la espera a cada carril; por ello, la calibración proporciona una envolvente conservadora de la señal utilizada. El valor permanece fijo durante las búsquedas y el entrenamiento final.

La fase 1 urbana comienza después de A–E y A/C y utiliza observación local y $\rho_{\text{coop}} = 0$, conforme a la decisión explicada en la subsección 3.4.3. Comparte con los demás escenarios las dos arquitecturas, de dos capas ocultas con 128 o 256 unidades por capa, y los límites de propuestas, iteraciones, ASHA y seguimiento diagnóstico. Los cuatro algoritmos suman ocho estudios urbanos y 800 propuestas iniciales.

En los métodos de la familia PPO, cada episodio controlado aporta $7200/5 = 1440$ pasos nominales; el tamaño objetivo del lote por componente de aprendizaje es esa cantidad multiplicada por `train_batch_episodes`. Las condiciones propias de fase 2 se reúnen en el protocolo urbano.

La fase 2 urbana deriva sus dominios exclusivamente de la fase 1 de Mendoza mediante `experiments/marl_experiments/tuning/urban/refinement.py`; no incorpora dominios de los escenarios sintéticos. Para cada algoritmo y arquitectura, exige al menos diez ensayos que completen el presupuesto, pertenezcan al dominio inicial y tengan objetivo finito. Los ordena por menor objetivo y, en caso de empate, por identificador; utiliza los

diez primeros.

Los parámetros numéricos se expresan como coordenadas entre cero y uno dentro del dominio inicial, en escala logarítmica para los loguniformes. Se conserva el intervalo original si algún valor queda a un 5 % o menos de un extremo o si el intervalo observado ocupa al menos la mitad de su ancho. En los demás casos, se centra un intervalo en el punto medio de los extremos observados, se toma como ancho base el mayor entre el ancho observado y el 10 % del dominio inicial, y se amplía un 25 % a cada lado. El resultado se limita al dominio original.

En enteros se redondean los extremos hacia afuera y se expresa el superior como exclusivo; se conservan todas las alternativas categóricas y los parámetros fijos. Conservar las categorías evita descartarlas a partir de los diez ensayos seleccionados. Las reglas numéricas mantienen la exploración amplia cuando los mejores valores están cerca de los extremos o dispersos. El ancho mínimo evita reducir el dominio a un único valor y la ampliación conserva un margen de exploración fuera del intervalo observado. Los porcentajes son parámetros heurísticos de esta regla. El archivo de dominios conserva los identificadores de los ensayos y los hashes de sus fuentes.

El registro `experiments/marl_experiments/tuning/urban/seeds.json` separa las semillas del aprendiz, de SUMO de entrenamiento, de SUMO diagnóstico y de Optuna para cada fase, algoritmo y arquitectura. También registra la selección, el entrenamiento final y la evaluación final. Se genera con `random.Random`, semilla maestra 20261013 y muestreo sin reemplazo de enteros desde 1.000.001 hasta $2^{31} - 1$, excluyendo las semillas históricas identificadas y las de calibración. La semilla de SUMO se pasa mediante `-seed`; el reinicio del entorno recupera el estado asociado a ella. Las semillas del aprendiz y de SUMO del entrenamiento final son nuevas respecto de las búsquedas y la selección.

La lista de candidatos se construye con `experiments/marl_experiments/tuning/urban/selection.py`. La auditoría comprueba el presupuesto completo, la ausencia de errores técnicos, el objetivo y la espera diagnóstica finitos, y la procedencia del protocolo y de las semillas. Contrasta el algoritmo, la arquitectura y los hiperparámetros con el estudio de origen y verifica la iteración del modelo restaurado.

El panel común de selección utiliza cinco semillas de SUMO: 1205571043, 1020770583, 1021545419, 927020871 y 984623645. Se aplican tanto a los candidatos como a la referencia.

El manifiesto reserva veinte semillas de evaluación final por algoritmo, sin imponer un panel común entre ellos. Así se evita reutilizar para la evaluación final las simulaciones que intervienen en la elección de configuraciones. El criterio de checkpoint se mantiene en la última iteración del presupuesto completo de la búsqueda que aporta la configuración.

A.5.9. Métricas y análisis por pares de A/C

En la implementación, el campo `mean_acc_waiting_time` registra el cociente por paso descrito en la metodología: espera acumulada en los carriles del controlador dividida por los vehículos presentes. Cuando no hay vehículos, el valor del paso es cero. El código promedia estos valores por controlador y calcula el Gini dando el mismo peso a cada uno.

Para cada una de las tres redes, las cinco evaluaciones de una política se promedian antes de calcular las diferencias entre las 24 parejas de entrenamiento. El remuestreo conserva cada pareja completa y genera 20.000 conjuntos de 24 parejas con reemplazo. Para el intervalo percentil del 95 % se usan los percentiles 2,5 y 97,5; para el intervalo del 98,33 %, aproximadamente 0,83 y 99,17. Este nivel aplica la corrección de Bonferroni al nivel nominal de 0,05 dividido entre los tres contrastes A/C, uno por cada red (0,05/3), para cada medida por separado. No se ajusta conjuntamente entre métricas distintas.

Como análisis de sensibilidad, se calcula un intervalo basado en t de Student con 24 diferencias pareadas, bajo el supuesto de diferencias independientes y aproximadamente normales. Los intervalos percentiles y el intervalo t son aproximaciones; su cobertura no está garantizada, y los valores extremos se conservan para revisar su influencia.

El protocolo original de la campaña, identificado en `docs/PROTOCOLO_COOPERACION_MULTIESCENARIO.md`, incluye las tres redes. El informe histórico, en `cooperation_multiscenario_20260920/report/INFORME.md`, presenta como análisis principal 48 parejas de 3×1 y $4 \times 4_H$. El mismo informe separa los datos de 3×3 después de revisar los resultados porque su entrenamiento tuvo una dinámica distinta; señala que esa decisión de presentación no estaba fijada de antemano y que los registros de esa red se conservaron aparte. El criterio histórico de presentación se distingue del alcance metodológico adoptado aquí, que analiza por separado las tres redes; por tanto, no implica una exclusión previa de 3×3.

A.6. Detalles del aprendizaje y de la codificación

En los módulos de aprendizaje propios de MAPPO y HAPPO, $V_{b,i}$ es la predicción del crítico para el agente i en la muestra b y $y_{b,i}$ es su valor objetivo. Se denotan por N el número de agentes, por B la cantidad de muestras válidas del lote y por $c_v = \texttt{vf_clip_param}$ la cota superior configurada. Primero se calcula el error cuadrático promedio entre agentes de cada muestra, ℓ_b; después se limita al intervalo $[0, c_v]$ y se promedian las pérdidas de las muestras válidas para obtener la pérdida final del crítico, $\mathcal{L}_V$:

$$\ell_b = \frac{1}{N}\sum_{i=1}^{N}(V_{b,i} - y_{b,i})^2, \qquad \mathcal{L}_V = \frac{1}{B}\sum_{b=1}^{B}\text{mín}\{c_v, \text{máx}(0, \ell_b)\}. \tag{A.4}$$

Las muestras válidas son las que la máscara del lote no excluye del cálculo. Este recorte del error cuadrático no es el *value clipping* convencional de PPO, que compara predicciones

de valor recortadas y no recortadas. Su consecuencia para el aprendizaje y la falta de una frecuencia de activación cuantificada se explican en la subsección 3.3.3.

IDQN reutiliza transiciones de un búfer de experiencia priorizado y aprende valores para las acciones; no tiene un actor PPO, y la selección se realiza entre acciones permitidas. Los métodos tipo PPO recolectan un lote de trayectorias. En IPPO y MAPPO, un *minibatch* es una subdivisión del lote que produce una actualización; HAPPO procesa el lote completo por actor. Los críticos y optimizadores son componentes de entrenamiento, mientras que en PPO la ejecución de cada actor depende de su observación y de la máscara vigente.

En MAPPO, los objetivos de valor y las ventajas se calculan por separado con las recompensas individuales y las estimaciones del crítico. Las ventajas se estandarizan por agente: se resta su media y se divide por su desviación estándar.

En el aprendiz de HAPPO de la versión congelada inspeccionada, el crítico se actualiza al comienzo de cada *epoch* con el lote completo. Se aplica un paso de gradiente por actor y época. Las ventajas individuales estandarizadas ya calculadas inicializan los factores de reponderación M; estos se reinician en cada *epoch*, y las ventajas no se recalculan durante la secuencia de actores. Antes de actualizar actores, el código construye un orden entero de agentes. En cada turno, `greedy_selection_prob` determina la probabilidad de elegir, entre los agentes pendientes, el que tiene la mayor media del valor absoluto de su factor inicial; de lo contrario, elige uno al azar. Esta regla ordena a los actores para su actualización, no selecciona la fase semafórica. El orden no se vuelve a sortear después de reponderar factores.

Tras actualizar un actor, el código calcula para la acción de la muestra la razón entre su probabilidad bajo el actor actualizado y su probabilidad bajo la política recolectora. Esa razón multiplica los factores de los agentes que aún esperan su turno. La reponderación no ejecuta una actualización de gradiente adicional para esos actores. Estas reglas describen la adaptación de HAPPO utilizada en la tesina.

Zhong et al. encuadran HAPPO como una instancia de HAML y demuestran mejora monótona del retorno conjunto y convergencia bajo las condiciones del marco, que incluyen una recompensa compartida en un juego cooperativo. Las ventajas por agente no constituyen por sí solas una diferencia con ese planteo: HAPPO utiliza una descomposición por agentes de la ventaja conjunta. En esta adaptación, los objetivos de valor y las ventajas se calculan a partir de recompensas específicas por agente, en un juego parcialmente observable. Al faltar el retorno común, las garantías teóricas no se aplican directamente a la formulación experimental [ZKF+24].

A.6.1. Desarrollo algebraico del codificador

Para la espera local $x \geq 0$, sea $u = 1\,\text{veh} \cdot \text{s}$ la unidad de vehículo-segundo. El valor numérico de la espera es $x_{\text{num}} = x/u$ y el del límite local de escala, $M_{\text{num}} = M_{\text{lane}}/u > 0$.

La entrada normalizada $\hat{x} = \text{mín}(1, x_{\text{num}}/M_{\text{num}})$ queda entre cero y uno. Para $I \geq 2$, se define $L = (I-1)/M_{\text{num}}$; en los dominios utilizados, $M_{\text{num}} > I-1$, por lo que $0 < L < 1$. La transformación mezcla una componente logarítmica y otra lineal:

$$g(\hat{x}) = (1-L)\frac{\log_2\left(\frac{\hat{x}}{L}+1\right)}{\log_2\left(\frac{1}{L}+1\right)} + L\hat{x}, \tag{A.5}$$

El parámetro adimensional L regula la curvatura y el peso lineal. La salida se escala por el nivel máximo y se redondea hacia arriba, de modo que el máximo puede alcanzarse antes de llegar al límite de escala. La codificación se limita a $I-1$:

$$e(x) = \text{mín}\{I-1, \text{ceil}((I-1)g(\hat{x}))\}. \tag{A.6}$$

Para la observación vecinal, se aplica la misma función a la espera media por carril, con el límite de escala vecinal expresado en veh·s por carril.

La Figura A.3 separa las contribuciones del codificador utilizado en la sección 3.3.2. Permite comprobar la equivalencia algebraica, sin atribuir al ejemplo un resultado de control.

El número de niveles I se registra como `encoder_intervals` y el límite local de escala M_{lane} como `max_lane_value`. Este parámetro fija la escala de la espera acumulada por carril; la dependencia de L respecto de I explica por qué cambiar los niveles también modifica la transformación.

Sea $F = (1-L)(I-1)/\log_2(1/L+1)$ el factor que pondera el componente logarítmico. Para $0 \leq x_{\text{num}} < M_{\text{num}}$, la mezcla escalada de la sección 3.3.2 se expresa en la forma implementada por el código:

$$e(x) = \text{ceil}\left(F \cdot \log_2\left(\frac{x_{\text{num}}}{M_{\text{num}} \cdot L}+1\right) + L^2 \cdot x_{\text{num}}\right), \tag{A.7}$$

Para $x_{\text{num}} \geq M_{\text{num}}$, el codificador devuelve directamente $I-1$. Para valores inferiores al límite de escala, esta expresión es algebraicamente equivalente a la combinación de la ecuación (A.5), escalada por $I-1$ y redondeada hacia arriba. La saturación en $I-1$ al llegar a ese límite se expresa en la ecuación (A.6).

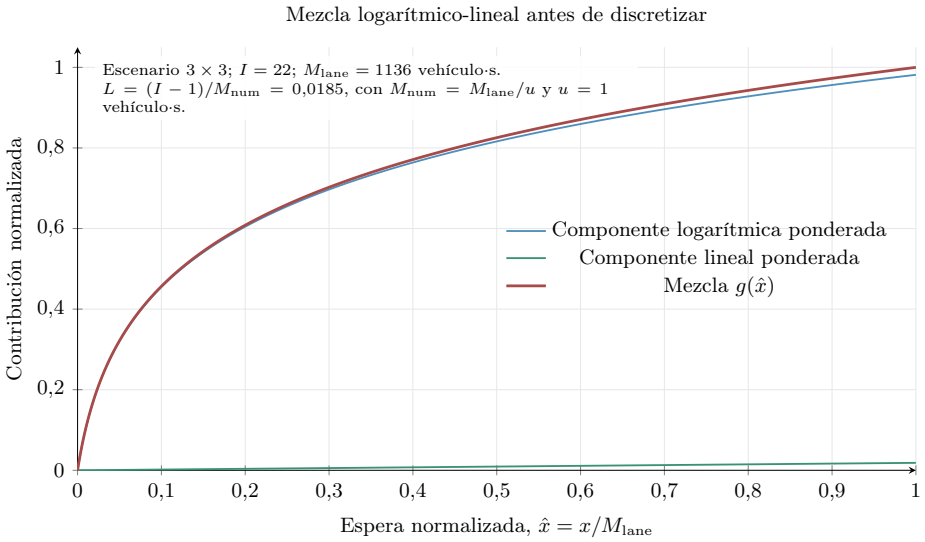


Figura A.3: Ejemplo ilustrativo con $I = 22$ y límite de escala $M_{\text{num}} = 1136$ calibrado para el escenario 3×3; $I = 22$ no corresponde a la configuración de la comparación A/C.

A.6.2. Aplicación de la máscara de acciones

En cada decisión, $\mathcal{A}_i^{\text{adm}}(t)$ es el subconjunto de acciones permitidas por la señal vigente. La máscara binaria $b_i(a,t)$ vale uno si a pertenece a ese conjunto y cero en caso contrario. Si $z_i(a,t)$ es la puntuación no normalizada (*logit*) asignada por el actor, la puntuación enmascarada $\widetilde{z}_i(a,t)$ se obtiene sumando el logaritmo natural de la máscara:

$$\widetilde{z}_i(a,t) = z_i(a,t) + \log b_i(a,t), \qquad b_i(a,t) = \begin{cases} 1 & \text{si } a \in \mathcal{A}_i^{\text{adm}}(t) \\ 0 & \text{si } a \notin \mathcal{A}_i^{\text{adm}}(t) \end{cases} \tag{A.8}$$

Con $\log(0) = -\infty$, las acciones no permitidas reciben probabilidad cero después de normalizar la política. La implementación sustituye el infinito por una puntuación finita muy baja y requiere al menos una acción admisible. Los algoritmos PPO aplican la máscara a las puntuaciones de política; IDQN filtra las acciones al seleccionarlas y evaluarlas.

La Tabla 3.4, en la metodología, distingue las señales por procedimiento. En el programa de fase 1 consultado, la normalización por carriles está desactivada por defecto; esa configuración del programa no verifica cada ensayo histórico. La comparación A/C fija la señal local en A y activa la contribución vecinal normalizada en C, según la ecuación (3.6).

A.6.3. Configuración fija de la comparación multiescenario

La Tabla A.15 conserva los hiperparámetros de MAPPO h128 seleccionados en un ensayo histórico de $4 \times 4_H$ y aplicados sin cambios entre las tres redes de tráfico. El ensayo `49cfbba0` está registrado en `rho_screening_recipes/mappo_4x4_H_h128.json` e incorporado a los manifiestos de campaña. Los valores continuos se muestran redondeados;

la fuente conserva precisión completa.

Se entrenaron 24 parejas por red. La planificación usó una desviación de referencia cercana a dos segundos por vehículo y un intervalo del 95 % basado en t de Student, bajo supuestos de diferencias independientes y aproximadamente normales. Con 24 parejas, se esperaba una semiamplitud cercana a 0,85 segundos por vehículo. Esta previsión no garantiza detectar una diferencia ni determina la amplitud de los intervalos observados.

Tabla A.15: Hiperparámetros fijos de aprendizaje de la comparación multiescenario.

Parámetro	Valor
Arquitectura	dos capas de 128 unidades
Tasa de aprendizaje (`lr`)	$4,95235885 \times 10^{-5}$
Recorte del actor (`clip_param`)	0,20148266
GAE (`lambda_gae`)	0,90059980
Coeficiente de entropía	0,00483113
Recorte del gradiente	1,48846291
Coeficiente KL	0,24180060
Objetivo KL	0,01053851
Cota de pérdida de valor	10,11299284
Recorridos de optimización	8
Episodios por lote	3
Tamaño del minilote	128
Niveles del codificador (I)	17
Descuento (γ)	0,99

La comparación fija el peso de la recompensa vecinal en cero para la variante local y en 0,1 para la vecinal; también asigna observación local o vecinal y normaliza por carriles, según la condición. Los límites de escala del codificador corresponden a cada escenario. Los tres episodios de cada lote representan $3 \times (3600/5) = 2160$ pasos de entorno por componente de aprendizaje. El número de procesos de simulación no multiplica este tamaño de lote.

Los manifiestos de la comparación registran SUMO 1.26.0, Ray 2.54.1 y PyTorch 2.10.0 con CUDA 12.8 (*Compute Unified Device Architecture*), plataforma que permite ejecutar cálculos en las unidades de procesamiento gráfico. Estas versiones corresponden a esa campaña y no se atribuyen automáticamente a las búsquedas históricas. Los archivos `manifest.json` de cada par conservan su configuración y versiones efectivas.

La calibración vecinal específica se conserva junto con la configuración de la comparación; otros archivos de calibración pueden corresponder a diseños distintos.

A.6.4. Calibraciones y semillas de cooperación

El límite de escala local se calculó con las semillas 42–44; para 3×1, 3×3 y $4 \times 4_H$ se obtuvieron 1184, 1136 y 712, respectivamente. La calibración de la señal de espera vecinal incorporada en la observación C utilizó las semillas 51001–51003 y produjo límites de escala de 9660, 5329 y 4652 en esas redes. Los límites locales se expresan en veh·s y los vecinales en veh·s por carril. En ambos casos se empleó el percentil 99 y 300 segundos de calentamiento. Para la señal vecinal, el percentil se calculó sobre la medida media agrupada por intersección y paso. Estos valores corresponden a escalas de señales codificadas, no a tiempos máximos de viaje ni a valores de recompensa.

La comparación en las tres redes utiliza las semillas 61001–61024 para el entrenamiento y 71001–71005 para la evaluación. Las semillas de evaluación son comunes a las dos variantes y se mantienen separadas de las de entrenamiento.

A.6.5. Preparación de las figuras de escenarios

Las figuras de topología utilizan capturas de las redes sintéticas y urbanas en SUMO. El encuadre elimina márgenes blancos para ampliar la red y conserva una escala de distancia. La Figura 3.6 muestra el dominio urbano modelado; el punto de conteo de la cámara 169 queda fuera de ese dominio y no aparece en la captura.

Bibliografía

[ACS24] Albrecht, Stefano V., Filippos Christianos y Lukas Schäfer: *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. `https://www.marl-book.com/`, Consultado en 2025.

[AHL16] Abel, David, D. Ellis Hershkowitz y Michael L. Littman: *Near Optimal Behavior via Approximate State Abstraction*. En *Proceedings of the 33rd International Conference on Machine Learning*, volumen 48 de *Proceedings of Machine Learning Research*, páginas 2915–2923. PMLR, 2016.

[AOS20] Amato, Christopher, Frans A. Oliehoek y Matthijs T.J. Spaan: *A Survey of Decentralized Partially Observable Markov Decision Processes and Applications*. Journal of Artificial Intelligence Research, 69:1–72, 2020.

[AS22] Ault, James y Guni Sharon: *Cooperative Multi-agent Reinforcement Learning Applied to Multi-intersection Traffic Signal Control*. En *Proceedings of the Third Workshop on Data-driven Intelligent Transportation, co-located with CIKM '22*, 2022. `https://people.engr.tamu.edu/guni/papers/CIKM22-MATCS.pdf`, Fuente: [7, 8] Destaca el "éxito limitado"de SARL y los enfoques independientes en redes.

[ASC+21] Agarwal, Rishabh, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville y Marc G. Bellemare: *Deep Reinforcement Learning at the Edge of the Statistical Precipice*. En *Advances in Neural Information Processing Systems 34*, páginas 29304–29320, 2021. `https://proceedings.neurips.cc/paper_files/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html`.

[Aso18] Asociación de Fábricas Argentinas de Componentes: *Flota circulante en Argentina 2017*. Informe técnico, Asociación de Fábricas Argentinas de Componentes, mayo 2018. `https://cdn.motor1.com/pdf-files/afac-informe-parque-circulante-2017pdf.pdf`, Informe elaborado con datos de AFAC y Promotive.

[Aso26] Asociación de Fábricas Argentinas de Componentes: *Flota vehicular circulante en Argentina*. Informe técnico, Asociación de Fábricas Argentinas de

Componentes, 2026. https://www.automotrix.com.ar/adm/enviosmasivos/Plantillas/PRENSA/2026/AFAC_PRENSA_FLOTACIRCULANTE_2025.pdf, Datos al cierre de 2025; informe elaborado en el marco del convenio entre AFAC y Promotive S.A.

[ASY+19] Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takeru Ohta y Masanori Koyama: *Optuna: A next-generation hyperparameter optimization framework*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 2623–2631, 2019.

[BB24] Bishop, Christopher M. y Hugh Bishop: *Deep Learning: Foundations and Concepts*. Springer, Cham, 2024.

[BBBK11] Bergstra, James S., Rémi Bardenet, Yoshua Bengio y Balázs Kégl: *Algorithms for Hyper-Parameter Optimization*. En *Advances in Neural Information Processing Systems 24*, 2011. https://proceedings.neurips.cc/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html.

[BV02] Bowling, Michael y Manuela Veloso: *Multiagent learning using a variable learning rate*. Artificial Intelligence, 136(2):215–250, 2002.

[CWCL19] Chu, Tianshu, Jie Wang, Lara Codecà y Zhaojian Li: *Multi-agent deep reinforcement learning for large-scale traffic signal control*. IEEE Transactions on Intelligent Transportation Systems, 21(3):1086–1095, 2019.

[CYL+24] Chen, Wubing, Shangdong Yang, Wenbin Li, Yujing Hu, Xiao Liu y Yang Gao: *Learning Multi-Intersection Traffic Signal Control via Coevolutionary Multi-Agent Reinforcement Learning*. IEEE Transactions on Intelligent Transportation Systems, 25(11):15947–15963, 2024.

[CyMR18] Mayor R., Rafael Cal y: *Ingeniería de tránsito: Fundamentos y aplicaciones*. Alfaomega, 9na edición, 2018.

[DLR26] DLR – Institute of Transportation Systems: *SUMO User Documentation*, 2026. https://sumo.dlr.de/docs/, Documentación web consultada en 2026.

[DLSZ26] Deng, Maojiang, Shoufeng Lu, Jiazhao Shi y Wen Zhang: *Adaptive Traffic Signal Control Optimization Using a Novel Road Partition and Multi-Channel State Representation Method*. Urban Lifeline, 4(1):9, 2026.

[dWGM+20] Witt, Christian Schroeder de, Tarun Gupta, Denys Makoviichuk, Viktor Maksimov, Edward Ivanov, Michael Schmid, Gregory Yakovlev, Philip Torr

y cols.: *Is independent learning all you need in the starcraft multi-agent challenge?* arXiv preprint arXiv:2011.09533, 2020.

[Ecl26a] Eclipse SUMO: *SUMO Documentation: Save and Load*. `https://sumo.dlr.de/docs/Simulation/SaveAndLoad.html`, 2026. Documentación en línea, consultada el 13 de octubre de 2026.

[Ecl26b] Eclipse SUMO: *SUMO Documentation: TraCI: Vehicle Value Retrieval*. `https://sumo.dlr.de/docs/TraCI/Vehicle_Value_Retrieval.html`, 2026. Documentación en línea, consultada el 8 de octubre de 2026.

[Ecl26c] Eclipse SUMO: *SUMO Documentation: TripInfo*. `https://sumo.dlr.de/docs/Simulation/Output/TripInfo.html`, 2026. Documentación en línea, consultada el 8 de octubre de 2026.

[Ecl26d] Eclipse SUMO: *SUMO Documentation: Why Vehicles Are Teleporting*. `https://sumo.dlr.de/docs/Simulation/Why_Vehicles_are_teleporting.html`, 2026. Documentación en línea, consultada el 13 de octubre de 2026.

[ELR23] Eimer, Theresa, Marius Lindauer y Roberta Raileanu: *Hyperparameters in Reinforcement Learning and How To Tune Them*. En *Proceedings of the 40th International Conference on Machine Learning*, volumen 202 de *Proceedings of Machine Learning Research*, páginas 9104–9149. PMLR, 2023. `https://proceedings.mlr.press/v202/eimer23a.html`.

[FA11] Fernández A., Rodrigo: *Elementos de la teoría del tráfico vehicular*. Fondo Editorial de la Pontificia Universidad Católica del Perú, 2011.

[Fed08] Federal Highway Administration: *Traffic Signal Timing Manual*. Informe técnico FHWA-HOP-08-024, Federal Highway Administration, Office of Operations, 2008. Capítulo 4: Traffic Signal Design. Disponible en: `https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter4.htm`.

[FNF+17] Foerster, Jakob, Nantas Nardelli, Gregory Farquhar, Philip H.S. Torr, Pushmeet Kohli y Shimon Whiteson: *Stabilising Experience Replay for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, páginas 1146–1155. PMLR, 2017.

[FYX+23] Fang, Jie, Ya You, Mengyun Xu, Juanmeizi Wang y Sibin Cai: *Multi-Objective Traffic Signal Control Using Network-Wide Agent Coordinated Reinforcement Learning*. Expert Systems with Applications, 229:120535, 2023.

[GBC16] Goodfellow, Ian, Yoshua Bengio y Aaron Courville: *Deep Learning*. MIT Press, Cambridge, MA, 2016. https://www.deeplearningbook.org/.

[GR18] Genders, Wade y Saiedeh Razavi: *Evaluating Reinforcement Learning State Representations for Adaptive Traffic Signal Control*. Procedia Computer Science, 130:26–33, 2018.

[HHA22] Haddad, Tarek Amine, Djalal Hedjazi y Sofiane Aouag: *A Deep Reinforcement Learning-Based Cooperative Approach for Multi-Intersection Traffic Signal Control*. Engineering Applications of Artificial Intelligence, 114:105019, 2022.

[JLW+24] Jiang, Haoyuan, Ziyue Li, Hua Wei, Xuantang Xiong, Jingqing Ruan, Jiaming Lu, Hangyu Mao y Rui Zhao: *X-Light: Cross-City Traffic Signal Control Using Transformer on Transformer as Meta Multi-Agent Reinforcement Learner*. En *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, páginas 94–102, 2024.

[KCW+22] Kuba, Jakub Grudzien, Ruiqing Chen, Muning Wen, Ying Wen, Fuchun Sun, Jun Wang y Yaodong Yang: *Trust region policy optimisation in multi-agent reinforcement learning*. En *International Conference on Learning Representations (ICLR)*, 2022. https://arxiv.org/abs/2109.11251.

[KKBA23] Kolat, Máté, Bálint Kővári, Tamás Bécsi y Szilárd Aradi: *Multi-Agent Reinforcement Learning for Traffic Signal Control: A Cooperative Approach*. Sustainability, 15(4):3479, 2023.

[KL02] Kakade, Sham y John Langford: *Approximately optimal reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 267–274, 2002.

[Kra98] Krauß, Stefan: *Microscopic Modeling of Traffic Flow: Investigation of Collision Free Vehicle Dynamics*. DLR-Forschungsbericht 98-08, Deutsches Zentrum für Luft- und Raumfahrt, 1998. https://elib.dlr.de/8380/.

[KT03] Konda, Vijay R. y John N. Tsitsiklis: *On Actor-Critic Algorithms*. SIAM Journal on Control and Optimization, 42(4):1143–1166, 2003.

[LFYZ25] Lu, Qiong, Haoda Fang, Zhangcheng Yin y Guliang Zhu: *HAPS-PPO: A Multi-Agent Reinforcement Learning Architecture for Coordinated Regional Control of Traffic Signals in Heterogeneous Road Networks*. Applied Sciences, 15(20):10945, 2025.

[LJR+20] Li, Liam, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-Tzur, Moritz Hardt, Benjamin Recht y Ameet Talwalkar: *A system for massively parallel hyperparameter tuning.* En *Proceedings of Machine Learning and Systems (MLSys)*, volumen 2, páginas 230–246, 2020.

[LLN+18a] Liang, Eric, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan y Ion Stoica: *RLlib: Abstractions for distributed reinforcement learning.* En *International Conference on Machine Learning (ICML)*, páginas 3053–3062. PMLR, 2018.

[LLN+18b] Liaw, Richard, Eric Liang, Robert Nishihara, Philipp Moritz, Joseph E Gonzalez y Ion Stoica: *Tune: A research platform for distributed model selection and training.* arXiv preprint arXiv:1807.05118, 2018.

[LML+25] Liang, Jiaxin, Haotian Miao, Kai Li, Jianheng Tan, Xi Wang, Rui Luo y Yueqiu Jiang: *A Review of Multi-Agent Reinforcement Learning Algorithms.* Electronics, 14(4):820, 2025. https://doi.org/10.3390/electronics14040820.

[LWT+17] Lowe, Ryan, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel y Igor Mordatch: *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments.* En *Advances in Neural Information Processing Systems (NeurIPS)*, páginas 6379–6390, 2017.

[LZF+25] Liu, Wanting, Chengwei Zhang, Wanqing Fang, Kailing Zhou, Yihong Li, Furui Zhan, Qi Wang, Wanli Xue y Rong Chen: *Vehicle-Level Fairness-Oriented Constrained Multi-Agent Reinforcement Learning for Adaptive Traffic Signal Control.* IEEE Transactions on Intelligent Transportation Systems, 26(4):4878–4890, 2025.

[MKS+15] Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg y D. Hassabis: *Human-level control through deep reinforcement learning.* Nature, 518(7540):529–533, 2015. https://doi.org/10.1038/nature14236.

[MMLK25] Michailidis, Panagiotis, Iakovos Michailidis, Charalampos Rafail Lazaridis y Elias Kosmatopoulos: *Traffic Signal Control via Reinforcement Learning: A Review on Applications and Innovations.* Infrastructures, 10(5):114, 2025. Fuente: [4, 5] Revisión que documenta la evolución de RL como framework para TSC.

[PW16] Pande, Anurag y Brian Wolshon: *Traffic Engineering Handbook*. John Wiley & Sons, Inc., 7th edición, 2016.

[RDZ+24] Ren, Fuyue, Wei Dong, Xiaodong Zhao, Fan Zhang, Yaguang Kong y Qiang Yang: *Two-Layer Coordinated Reinforcement Learning for Traffic Signal Control in Traffic Network*. Expert Systems with Applications, 235:121111, 2024.

[RSDW+18] Rashid, Tabish, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster y Shimon Whiteson: *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, páginas 4295–4304. PMLR, 2018.

[SB18] Sutton, R.S. y A.G. Barto: *Reinforcement learning: An introduction*. The MIT Press, Cambridge, MA, 2nd edición, 2018.

[Sha53] Shapley, Lloyd S: *Stochastic games*. Proceedings of the National Academy of Sciences, 39(10):1095–1100, 1953.

[SLA+15] Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan y Philipp Moritz: *Trust region policy optimization*. En *International Conference on Machine Learning (ICML)*, páginas 1889–1897. PMLR, 2015.

[SLG+18] Sunehag, Peter, Guy Lever, Audrunas Gruslys, Wojciech M. Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls y Thore Graepel: *Value-Decomposition Networks For Cooperative Multi-Agent Learning*. En *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, páginas 2085–2087. International Foundation for Autonomous Agents and Multiagent Systems, 2018.

[SML+16] Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan y Pieter Abbeel: *High-dimensional continuous control using generalized advantage estimation*. En *International Conference on Learning Representations (ICLR)*, 2016.

[SWD+17] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov: *Proximal policy optimization algorithms*. arXiv preprint arXiv:1707.06347, 2017.

[TMK+17] Tampuu, Ardi, Tambet Matiisen, Dorian Kodelja, Ilya Zauss, Kristjan Korjus, Juhan Aru, Jaan Birk y Raul Vicente: *Multiagent cooperation and competition with deep reinforcement learning*. PloS one, 12(4):e0172395, 2017.

[Tok24] Toko: *Manual de uso del clúster Toko*. https://toko.uncu.edu.ar/files/Manual_Toko-20240401.pdf, 2024.

[W+19] Wei, H. *y cols.*: *A survey on traffic signal control methods*, 2019. Available at: https://arxiv.org/abs/1904.08117.

[WCZ+19] Wei, Hua, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash Gayah, Kai Xu y Zhenhui Li: *PressLight: Learning Max Pressure Control to Coordinate Traffic Signals in Arterial Network*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 1290–1298, 2019.

[Wie00] Wiering, Marco A: *Multi-agent reinforcement learning for traffic light control*. Machine learning, 2000.

[WWS+22] Wu, Qiang, Jianqing Wu, Jun Shen, Bo Du, Akbar Telikani, Mahdi Fahmideh y Chao Liang: *Distributed Agent-Based Deep Reinforcement Learning for Large Scale Traffic Signal Control*. Knowledge-Based Systems, 241:108304, 2022.

[WXZ+19] Wei, Hua, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu y Zhenhui Li: *CoLight: Learning Network-level Cooperation for Traffic Signal Control*. En *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, páginas 1913–1922, 2019.

[WZK+23] Wang, Xihuai, Lingpan Zheng, Jakub Grudzien Kuba, Ruiqing Chen, Li Shen, Yidong Cao, Jun Wang y Yaodong Yang: *Order matters: Agent-by-agent policy optimization*. En *International Conference on Learning Representations (ICLR)*, 2023.

[Yan23] Yang, Shantian: *Hierarchical Graph Multi-Agent Reinforcement Learning for Traffic Signal Control*. Information Sciences, 634:55–72, 2023.

[YVV+22] Yu, Chao, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen y Yi Wu: *The surprising effectiveness of ppo in cooperative multi-agent games*. Advances in Neural Information Processing Systems, 35:24611–24624, 2022.

[ZKF+24] Zhong, Yifan, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji y Yaodong Yang: *Heterogeneous-Agent Reinforcement Learning*. Journal of Machine Learning Research, 25:1–67, 2024. https://www.jmlr.org/papers/v25/23-0488.html.

[ZLFS24] Zhang, Yifeng, Peizhuo Li, Mingfeng Fan y Guillaume Sartoretti: *HeteroLight: A General and Efficient Learning Approach for Heterogeneous Traffic Signal Control*. En *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, páginas 1010–1017, 2024.

[ZLYZ23] Zhou, Mengdi, Xiaoteng Li, Yaodong Yang y Weinan Zhang: *A Survey on Centralized Training for Decentralized Execution in Multi-Agent Reinforcement Learning*. Artificial Intelligence Review, 2023.

[ZWL24] Zhang, Haipeng, Zhiwen Wang y Na Li: *MATLight: Traffic Signal Coordinated Control Algorithm Based on Heterogeneous-Agent Mirror Learning With Transformer*. En *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, páginas 2582–2584, 2024.